

MICR 336

Microbial Ecology

2021

Table of Contents

1	General Information for MICR336 Laboratory Class	8
1.1	Laboratory Hours	8
1.2	Laboratory attendance is a requirement for terms	9
1.3	Internal Assessments	9
1.3.1	Research Project Design assessment	9
1.3.2	Lab assessments	11
1.3.3	Late submission of internal assessments	12
1.3.4	Special consideration for internal assessments	12
1.4	Academic integrity and copyright	13
1.4.1	Plagiarism	13
1.4.2	Unauthorised collaboration	13
1.4.3	Copyright	13
1.5	Useful contacts for MICR336 laboratory classes	14
1.6	Disabilities Support	15
1.7	Department Kaiāwhina	15
1.8	Department Pacific Island Support	15
1.9	Laboratory Timetable	17
2	MICR336 Laboratory Class	19
2.1	Overview	19
2.1.1	Essential background	20
2.2	Week 1	21
2.2.1	Introduction to R and R Studio	21
2.2.2	Setting up your class data environment	23
2.2.3	Using R	26
2.2.4	A quick orientation with R Studio	26
2.2.5	How to make an R markdown	27
2.2.6	Your very first R exercises	30
2.3	Using R to analyze 16S rRNA data	34

2.3.1	What kind of data comes back from the sequencing facility? _____	36
2.3.2	Downloading the data we will use today _____	37
2.3.3	Installing and loading R packages _____	38
2.3.4	Setting your working directory and sorting file paths _____	39
2.3.5	Making amplicons with dada2 _____	40
2.3.6	Assigning taxonomy and analysing output _____	43
2.4	Comparing Dada2, Qiime, and Mothur _____	44
2.5	Rarefaction curves _____	49
2.6	Week 2 Phyloseq objects _____	52
2.7	Getting started _____	53
2.7.1	Load phyloseq ³ _____	53
2.8	Data analysis-HMP Case study _____	54
2.8.1	Investigate Some Basic Features of HMPv35 _____	54
2.8.2	Exploring basic HMPv35 dataset features _____	56
2.8.3	Calculate alpha diversity _____	60
2.8.4	Compare beta diversity _____	68
2.8.5	Identifying differences in community composition _____	72
2.8.6	ANOSIM: The ANALysis Of SIMilarity (ANOSIM) test _____	73
2.8.7	NPMANOVA (ADONIS): non-parametric multivariate analysis of variance _____	74
2.8.8	Working with continuous data _____	75
2.8.9	Week 3 Exploring community composition _____	82
2.8.10	Differential pattern analysis _____	85
2.8.11	Differential pattern analysis using edgeR package _____	88
2.8.12	R coded subsetting of important OTUs _____	92
2.8.13	Plotting taxonomic changes along continuous data _____	94
3	Capstone: Your assessment _____	102
4	Appendix _____	104
4.1	Installing Software for Class _____	104
4.1.1	How to install R and R Studio _____	104
4.2	Creating an R Markdown (not from the class template). _____	108

4.3	Some simple R exercises	110
4.3.1	Beginning to use R	110
4.3.2	A conceptual view of R – The Big Picture	110
4.3.3	Getting help in R Studio ²	113
4.3.4	Understanding Variables in R ²	113
4.3.5	Reading and Writing Data ²	115
4.4	Source of R tutorials	116
4.4.1	Murray Cazdow vignettes	116
4.4.2	Bioconductor Vignettes	116
4.5	Additional R Concepts	117
4.5.1	The Working Directory ²	117
4.5.2	How to install R packages manually	118
4.5.3	How to download HMPv35 R data from original source	118
4.6	Hypothesis testing in categorical and continuous metadata	119
4.6.1	Categorical hypothesis testing	119
4.6.2	Continuous data hypothesis testing	122
5	Glossary	127
6	References	132

Figure 1. How to download the data from Gitlab.....	24
Figure 2. How to extract Micro336-master file	25
Figure 3. An overview of the R studio environment.....	26
Figure 4. Making an R Markdown.....	28
Figure 5. Knitting an R Markdown.....	29
Figure 6. Example output from knitting an R markdown.....	30
Figure 7. A schematic view of Illumina sequencing data. Source: https://www.illumina.com/documents/products/appnotes/appnote_miseq_16S.pdf	35
Figure 8. Construction of a phyloseq object.....	52
Figure 9. Distributions of OTU abundance and coverage in the HMP dataset (all samples).....	57
Figure 10. Detection of new species with increasing read depth. Statistics: Spearman correlation. .	59
Figure 11. Distributions of OTU abundance and coverage in HMP data, saliva samples only.....	60
Figure 12. Boxplots of alpha diversity in HMP saliva samples.....	61
Figure 13. Rarefaction curves showing species detected along levels of sampling for all HMPbodysubsites.....	63
Figure 14. Rarefaction curves faceted by HMPbodysubsite	63
Figure 15. Alpha diversity of HMP saliva samples after rarefaction.....	65
Figure 16. Refer to Figure 15. Plot re-drawn now showing results for <code>kruskal.test</code> for changes based on global mean.	67
Figure 17. Refer to Figure 16. Plot re-drawn now showing results specific comparisons between categories.....	68
Figure 18. Stressplot for NMDS ordination	70
Figure 19. NMDS plot based on raw (unrarified) data comparing samples based on sex.....	71
Figure 20. NMDS plot based on rarified data comparing samples based on sex.....	72
Figure 21. NMDS ordination of HMP saliva and stool samples	73
Figure 22. Changes in alpha diversity (Observed and Shannon) along a pH gradient for vaginal samples with results for Spearman correlation.	76
Figure 23. NMDS plot based vaginal samples showing relationship to pH.....	78
Figure 24. Correlation between NMDS1 (from Figure23) and pH based on a linear model.	79
Figure 25. Stacked barplot showing mean relative abundance of phyla	83
Figure 26. Stacked barplot showing mean relative abundance of phyla, faceted by sex	84
Figure 27. Faceted barplot, shown at genus level	85

Figure 28. Stacked barplot showing mean relative abundance of phyla after glomming data at Class level.....	86
Figure 29. Differentially abundant taxa between stool and saliva by Class and Phylum. Taxa identified using the <code>kruskal_test</code>	87
Figure 30. EdgeR results - taxa differentially abundant between body sites	90
Figure 31. EdgeR results: genera differentially abundant between body site, coloured by phylum....	91
Figure 32. Differentially abundant taxa between stool and saliva by Phylum. Taxa identified using the <code>exactTest</code>	93
Figure 33. Differentially abundant taxa between stool and saliva by Genus and Phylum. Taxa identified using the <code>kruskal_test</code>	94
Figure 34. Taxonomic (Phylum level) changes along a pH gradient for vaginal samples with results for Spearman correlation.	95
Figure 35. Taxonomic (Phylum level) changes along a pH gradient for vaginal samples with only significant Phyla ($\text{corr} > 0.5$) plotted. Statistic results for Spearman correlation shown.	98
Figure 36. Differentially abundant taxa between stool and saliva by Genus and Phylum. Taxa identified using a custom linear model. Plots show the results of a separate Spearman correlation.	101
Figure 37. How to download R.....	105
Figure 38. Downloading R on a Mac. Choose the correct version for your OS.....	106
Figure 39. Downloading R on Windows.....	107
Figure 40. Downloading R Studio.....	108
Figure 41. Making a new R Markdown.....	109
Figure 42. Naming your R Markdown.	109
Figure 43. Output from example R commands.....	111
Figure 44. Example: using the console.....	112
Figure 45. Example: applying a statistical function to data.....	112
Figure 46. Using the R Studio help menu	113
Figure 47. Installing an R package from the gui.....	118

Table of Tables

Table 1. Laboratory Hours and CAL Rooms.....	8
Table 2. Summary of Assessments	11
Table 3. MICR336 Laboratory Timetable.....	17

1 General Information for MICR336 Laboratory Class

1.1 Laboratory Hours

Welcome to the MICR336 laboratory classes. The lab classes for MICR336 run for 4 weeks, from 10th August -8th September, 2021.

MICR336 labs are made up of four weeks of computer-based laboratory work. The lab classes are held in the Computer Resource Rooms (CAL labs) on campus in the following locations:

Hercus CAL- Hercus Building, School of Medicine ([Click here for link to Hercus](#))

Arts CAL – Arts 1 Building, Ground Floor ([Click here for link to Arts](#))

North CAL- Science 3 Building, Ground Floor ([Click here for link to North](#))

Due to student numbers the lab days will be streamed as below:

Table 1. Laboratory Hours and CAL Rooms

Week	Day	Stream	Time	CAL room
August 10 th – 12 th				
1	Tuesday	All in one stream	2:00 PM – 6:00 PM	Hercus
	Wednesday	Stream 1	9:00 AM – 12:00 PM	Hercus
	Wednesday	Stream 2	2:00 PM – 5:00 PM	Hercus
	Thursday	Drop in help session (45 students max)	9:00 AM – 11:00 AM	Arts
August 17 th – 19 th				
2	Tuesday	Streamed**	2:00 PM – 6:00 PM	Hercus & North
	Wednesday	Stream 1	9:00 AM – 12:00 PM	Hercus
	Wednesday	Stream 2	2:00 PM – 5:00 PM	Arts
	Thursday	Drop in help session (45 students max)	9:00 AM – 11:00 AM	Arts
August 24 th – 26 th				

3	Tuesday	All in one stream	2:00 PM – 6:00 PM	Hercus
	Wednesday	Stream 1	9:00 AM – 12:00 PM	Hercus
	Wednesday	Stream 2	2:00 PM – 5:00 PM	Arts
	Thursday	Drop in help session (45 students max)	9:00 AM – 11:00 AM	Arts
Mid-Semester Break				
September 7 th – 8 th				
4	Tuesday	All in one stream	2:00 PM – 6:00 PM	Hercus
	Wednesday	Stream 1	9:00 AM – 12:00 PM	Hercus
	Wednesday	Stream 2	2:00 PM – 5:00 PM	Hercus

**** Week 2** - On Tuesday, the class will be split between Arts and North CAL. We will let you know which CAL lab room you should attend on this day, prior to the lab.
Note: This may differ to the timetabled room on eVision.

Additional computer lab time has been booked on Tuesdays from 9:00 AM – 2:00 PM in Arts Cal. This is for students who wish extra time to work independently. There will be no demonstrator support during this time.

1.2 Laboratory attendance is a requirement for terms

Attendance and satisfactory completion of the MICR336 labs is a requirement for Terms in MICR336. Please make sure you have been marked off as present on both Tuesday and Wednesday of each week.

If you are unable to attend the laboratory sessions, you must notify Jen Robson (jennifer.robson@otago.ac.nz; 479-8475), prior to class.

1.3 Internal Assessments

1.3.1 Research Project Design assessment

You will submit two reports as part of this assessment.

1.3.1.1 Formative assessment

This report will not be graded. You will need to write a minimum 1-page (max 1000 words/4-pages double spaced Times size 12) report outlining your experimental design for a microbial ecology study. The study will be based on one of three ecosystems: soil, aquatic, host-associated. For your ecosystem of choice design an experiment that mimics a disturbance and provide a testable hypothesis that explains how that disturbance may influence your microbiome of interest. Be sure to include how you would measure the effect of the disturbance and any challenges you might encounter.

Your Project Design assessment must be submitted electronically through Blackboard as a .docx file, under the Assignments section of Blackboard. We will use Turnitin for plagiarism detection for this assessment. Assessment submission is a requirement for Terms but will not be graded. This assessment is due by **12 noon on Monday 16th of August**.

1.3.1.2 Summative assessment

This report **will** be graded. You will need to write a 4-page (max 1000 words, double spaced, Times size 12) report based on your prior formative report. The goal of the report is to assess your integration of concepts learned across both lecture and lab. You should reflect on your first report and indicate how you would change your study based on your improved knowledge. Things that you should consider:

- Are my hypothesis based on known and supported ecological theories?
- Is my experimental design fit for purpose?
- Is my data analysis justified and able to support testing of my hypothesis (this should include stats)?
- What were the weaknesses of your first report?
- How have you addressed them?

The Critique of your Project Design must be submitted electronically through Blackboard as a .docx file, under the Assignments section of Blackboard. We will use Turnitin for plagiarism detection for this assignment. The summative report is worth 10% of your final mark for MICR336. This assessment is due by **12 noon on Monday 11th of October**.

1.3.2 Lab assessments

1.3.2.1 *Laboratory notebook*

At the end of each week you will submit R Markdown (RMD) files. These will be compiled to form a laboratory notebook that you can refer to as a guide when working on your written laboratory report. In the first week, you will need to complete a short Blackboard quiz as well as submit an R Markdown. The weekly lab assessments will be due at **5 pm on Friday** of the first 3 lab weeks on the following dates **August 13th, 20th, 27th**. You can submit your weekly knitted R Markdown file directly to Blackboard from the web as an HTML file.

1.3.2.2 *Oral lab presentation*

As a group you will present the analysis you have undertaken. This will be valuable opportunity to receive feedback on your understanding prior to writing your lab report. The group presentations will be held during the MICR336 lecture times on the **13th, 15th and 20th of September**. All presentations need to be submitted to Blackboard by **12 noon on the 13th of September**.

1.3.2.3 *Written laboratory report*

You will submit a formal laboratory report detailing the investigation you have undertaken. The laboratory report is worth 25% of your final mark for MICR336. Laboratory reports are due by **Monday 27th of September by 12 noon**.

Your laboratory report must be submitted electronically through Blackboard as a .docx file, under the Assignments section of Blackboard. You will also submit an R Markdown RMD file containing all the code used to analyse your data. We will use Turnitin for plagiarism detection for this assignment. You can submit one draft version of your lab report to check for plagiarism against material available on the internet prior to submitting your final lab report. Your draft can be submitted in the “MICR336 lab report draft version” section, and your final file can be submitted in “MICR336 lab report final version” section. In each of these cases, you may submit only one file. A sample assessment report will be provided on Blackboard.

Further information on this assessment can be found in [Section 7](#).

Table 2. Summary of Assessments

Internal Assessments	Due dates	Weight
Weekly Laboratory notebooks	5 pm on 13 th , 20 th , 27 th August	10%
Research Project Design (Formative)	12 pm on 16 th August	Not graded
Group Oral Lab presentations	Due on 12 pm on 13 th September; Presentations held on 13 th , 15 th , 20 th September	Not graded
Written Laboratory report	12 pm on 27 th September	25%
Critique on Research Project Design	12 pm on 11 th October	10%

1.3.3 Late submission of internal assessments

Late submission for any internal assessments will be penalised at 5% PER DAY (including weekends) for up to 5 days. A zero mark will be given to any assessment that is more than 5 days late.

1.3.4 Special consideration for internal assessments

The Department of Microbiology and Immunology has outlined guidelines for students seeking special consideration for internal assessments due to an unexpected or significant impairment. If you would like to request either an extension for an internal assessment, then you will need to apply for special consideration.

Please read the procedure for application of special consideration, which can be found on Blackboard under Course Information. In all instance's students seeking special consideration need to complete Part A of the Health Declaration for Internal Assessment form, which should be sent to the MICR336 course convenor Dr Sergio Morales. Barring exceptional circumstances students are required to notify course convenors of late submission of internal assessments **prior** to the due date if special consideration is required. Please note the department reserves the right to decline special consideration if insufficient notification was given.

The Department of Microbiology and Immunology guidelines are in accordance with the University's policy. Full information on the University's policy is available on its website under [Special Consideration in Student Assessment Procedures](#). Special Consideration in internal assessment is covered specifically by clauses 1, 2 and 7 of the policy.

1.4 Academic integrity and copyright

1.4.1 Plagiarism

Students should make sure that all submitted work is their own. Plagiarism is a form of dishonest practice and is taken seriously by the Department of Microbiology and Immunology. Plagiarism (including being party to someone else's plagiarism) is defined as: copying or paraphrasing another's work, whether intentionally or otherwise, and presenting it as one's own. All students have a responsibility to be aware of acceptable academic practice in relation to the use of material prepared by others, and for taking all reasonable steps necessary to ensure that no breach of acceptable practice occurs. In practice, plagiarism includes any attempt in any piece of submitted work (e.g. an assignment or test) to present as one's own work the work of another (whether of another student or a published authority). Any student found responsible for plagiarism in any piece of work submitted for assessment shall be subject to the University's dishonest practice regulations which may result in various penalties, including forfeiture of marks for the piece of work submitted, a zero grade for the paper, or in extreme cases exclusion from the University. The University of Otago reserves the right to use plagiarism detection tools.

1.4.2 Unauthorised collaboration

Unauthorised collaboration occurs when you work with, or share work with, others on an assessment which is designed as a task for individuals and in which individual answers are required. This does not include assessment tasks where students are expressly required or permitted to present the results of collaborative work. Unless it is explicitly stated otherwise, each student's answers should be in their own words.

1.4.3 Copyright

All teaching materials in this paper (lecture material, material in the laboratory manual and lecture podcasts) are subject to copyright and are University- or staff-owned or are made available to you under licences to which the University subscribes. These materials are provided only for your own use while studying this paper and cannot be further copied in any way. Where material is provided to

you electronically you may download and print copies for your own use but you must not copy or share them without authorisation (for example uploading them to note sharing websites). A breach of copyright may expose you to legal action for copyright infringement and/or disciplinary action by the University.

1.5 Useful contacts for MICR336 laboratory classes

Sergio Morales (convener)	sergio.morales@otago.ac.nz	479-3140
Xochitl Morgan	xochitl.morgan@otago.ac.nz	479-7738
Jen Robson	jennifer.robson@otago.ac.nz	479-8475
Leighton Payne	leighton.payne@postgrad.otago.ac.nz	

1.6 Disabilities Support

The Department of Microbiology and Immunology encourages students to seek support if they find they are having difficulty with their studies due to disability, temporary or permanent impairment, injury, chronic illness or deafness.

Contact

Either: Department of Microbiology and Immunology Disabilities Officer

Warren McBurney

Email: warren.mcburney@otago.ac.nz

Or: Disabilities Information and Support

Phone: 479-8235

Email: disabilities@otago.ac.nz

Website: <http://www.otago.ac.nz/disabilities>

1.7 Department Kaiāwhina

Megan Hall (megan.hall@otago.ac.nz) is the Department of Microbiology and Immunology Kaiāwhina (Māori Student Support). Megan is here to provide access to academic and pastoral support for Māori students, including tutorials through Te Huka Matauraka (the Māori Centre), and to provide help and advocacy with scholarships.

For more information see <http://maori.otago.ac.nz/>

1.8 Department Pacific Island Support

Jennifer Robson (jennifer.robson@otago.ac.nz) and Htin Aung (htin.aung@otago.ac.nz) are the Department of Microbiology and Immunology Pacific Island Support people. Jennifer and Htin are here to ensure that Pacific Island students maximise their full potential by providing academic guidance,

helping them access pastoral support and providing assistance with advocacy and scholarships. They work closely with the Division of Sciences, the Division of Health Sciences and the Pacific Islands Centre.

For more information regarding support for Pacific Island students at the University of Otago, see www.otago.ac.nz/pacific.

1.9 Laboratory Timetable

Table 3. MICR336 Laboratory Timetable

Week 1 – August 10th – 11th
<i>Tuesday</i> • Learn to get around in R studio / student desktop • Basic R function • Tips and tricks for R analyses • How to work using markdown (RMD) files Microbiome sequencing data processing using dada2 pipeline
<i>Wednesday</i> • Work on first R Markdown • Sign up for a dataset for your final report
Week 2 – August 17th – 18th
<i>Tuesday</i> • <i>Microbiome data analysis: Alpha and beta diversity using phyloseq</i> ○ Visuals ○ Stats
<i>Wednesday</i> • Continue troubleshooting from Tuesday
Week 3 – August 24th – 25th
<i>Tuesday</i> • <i>Exploring community composition using phyloseq</i> ○ Visuals

○ Stats
<i>Wednesday</i> • Continue troubleshooting from Tuesday
Week 4 – September 7th – 8th
<i>Tuesday</i> • Work on assessment
<i>Wednesday</i> • Work on assessment

2 MICR336 Laboratory Class

2.1 Overview

In this series of practical exercises, you will learn to sample a microbial community and to analyse microbial sequencing data to understand how quality control affects your results. You should carefully plan your laboratory work and prepare a written record of all the work you have undertaken. Every week, you will turn in an R markdown of your lab work which contains your code and your answers to weekly assessment questions.

Week 1: Introduction to R, markdowns and amplicon data processing using dada2

Week 2: Microbiome data analyses: Exploring Alpha diversity

Week 3: Microbiome data analyses: Exploring Beta diversity

Week 4: Troubleshooting and work on assessment

For your laboratory report / assessment, you will receive sequencing data and perform an analysis of the microbial community.

2.1.1 Essential background

2.1.1.1 A history of 16S rRNA

Modern microbiology relies on high throughput sequencing and molecular approaches to examine the invisible living world around us. However, early microbial ecologists relied on culture-based approaches to study microbes. With over 1 trillion species predicted to live on Earth (Locey and Lennon, 2016) it was impossible to effectively study large communities or to even detect most microorganisms in nature. This changed when modern leaders in microbial ecology established a unique approach to cataloguing life by using 16S rRNA or DNA gene sequence analysis (Woese, 1987; Pace, 1997). This approach relies on molecular clocks (gene sequences that can be used to track the evolution of organisms) to study microbial communities. Of these clocks, the most studied is the 16S ribosomal RNA gene encoding for a small RNA within the ribosome. Since all living organisms must create proteins, all organisms have ribosomes and hence must carry a copy of this gene. The gene was a useful tool because:

- Every living creature has at least one copy of the rRNA gene (16S in prokaryotes or 18S in eukaryotes)
- The gene encodes a conserved function (ribosomal RNA) across all organisms
- It has both highly-conserved regions (which you can use for targeting primers for PCR) and highly variable regions (which you can use for comparing their evolution and hence assign taxonomy)
- It is a large enough sequence to allow evolutionary changes to be recorded.

In addition, by targeting DNA instead of attempting to culture individuals we can study both culturable and unculturable, as well as known vs. unknown organisms. Since its first use scientist have amassed millions of sequences that serve as databases listing all detected organisms across the planet (Hug *et al.*, 2016). These databases (Silva and Greengenes) allow us to collect DNA and accurately identify which prokaryotes are present in any sample, even if we cannot see, culture or even know these organisms. This has revolutionized our understanding of the diversity and distribution of bacteria, archaea and microbial eukaryotes. However, with increasing information we now rely on 'big data' tools to process these large datasets. This lab will provide you with the tools to jump right into this rapidly expanding field.

Hug LA, Baker BJ, Anantharaman K, Brown CT, Probst AJ, Castelle CJ, *et al.* (2016). A new view of the tree of life. *Nature Microbiology* **1**: 1–6.

Locey KJ, Lennon JT. (2016). Scaling laws predict global microbial diversity. *P Natl Acad Sci Usa* **113**: 5970–5975.

Pace NR. (1997). A Molecular View of Microbial Diversity and the Biosphere. *Science* **276**: 734–740.

Woese CR. (1987). Bacterial evolution. *Microbiological Reviews* **51**: 221–271.

2.2 Week 1

This week, on Tuesday you will get oriented with R and R Studio, learn to process 16S rRNA amplicon data using Dada2 and start to get familiar with phyloseq objects. On Wednesday you will complete your assessment.

2.2.1 Introduction to R and R Studio

R is a freely-available command-line-based programming language originally developed in New Zealand at the University of Auckland by Ross Ihaka and Robert Gentleman in 1993. It is now the 14th most widely-used programming language in the world. It is especially well-suited for statistics and for biological programming due to more than 20,000 packages available in the CRAN and Bioconductor software repositories.

R Studio is a graphic wrapper that makes R much easier to use. In MICR336, we will use both R and R Studio to learn quantitative analysis of microbial ecology data.

The goals of Week 1 exercises are:

- To become oriented with R, R Studio, and R Markdown
- To learn how to process 16S rRNA amplicon sequencing data for analysis using the dada2 package.
- To compare the output of two different amplicon sequencing pipelines, dada2 and Qiime.
- To become familiar with phyloseq objects

For your weekly assessment, you will need to take a Blackboard-based quiz (which will require your R code to answer), and you will need to turn in the R Markdown file that you have used to generate your answers.

At the end of Week 1, you should know the answer to this question:

If you process the same sequencing data using two different bioinformatic pipelines, do they agree on which species are present and how abundant they are?

2.2.2 Setting up your class data environment

2.2.2.1 Accessing Student Desktop

For MICR336 lab, we will use R and R Studio via Student Desktop. While R and R Studio are freely available and can be easily installed on both computers (see Appendix), using Student Desktop allows us to have a standardized software environment for class and allows us to preinstall a large number of software libraries and ensure that they are working properly. You are also welcome to install R and R Studio on your own device if you want to, but you will have to troubleshoot any installation issues yourself.

Please refer to the [Appendix 2](#) for instructions on how to install R and R Studio on your own device.

Access Student Desktop on your computer by pasting the following link into a web browser:

<http://www.otago.ac.nz/studentdesktop>

2.2.2.2 Downloading the class data and templates

We use a website called 'gitlab' to store all of the data you will need for MICR336 lab (including your assessment). This ensures that the data is backed up and version controlled, and lets us easily update things as needed during the semester. Today, you need to download our lab data from gitlab. We have stored a copy of the lab manual here, as well as templates and data files.

After you are logged into Student Desktop, please do the following:

1. Open a web browser and go to the following URL:
<https://gitlab.com/morganlab/public/Micro336>

2. Download this dataset (as a zip file) to your Student Desktop account. To do this, click on the Download icon on the far right, then choose “Download zip”. The Download button on the far right (it looks like a cloud) and is next to the “Find File” button. See screenshot below (**Figure 1**).

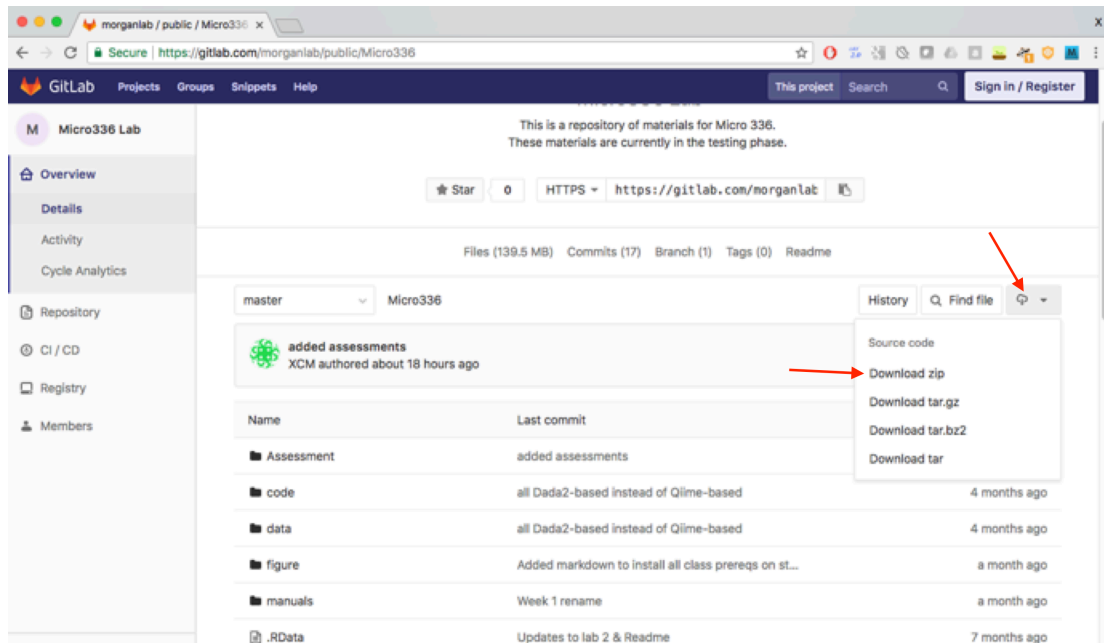


Figure 1. How to download the data from Gitlab.

3. Once you have downloaded this file, find it in your Downloads folder. Right click and select ‘7-zip’ and ‘Extract to Micro336-master’ (**Figure 2**). Once it has finished extracting, you will have a directory called ‘Micro336-master’ in your Downloads folder. Drag this folder from your Downloads folder into your Documents folder. **Make sure that you have unzipped this file and that you have placed it in Documents, and do not rename it** (All your code templates and sample code in the lab manual, which you will want to be able to copy and paste, assume that your files are located in Documents, and will not work properly if R is unable to locate your files).

When you are done with this step, the directory “Documents/Micr336-master/” should exist and should contain subdirectories “Assessments”, “data”, “code”, and “Templates”.

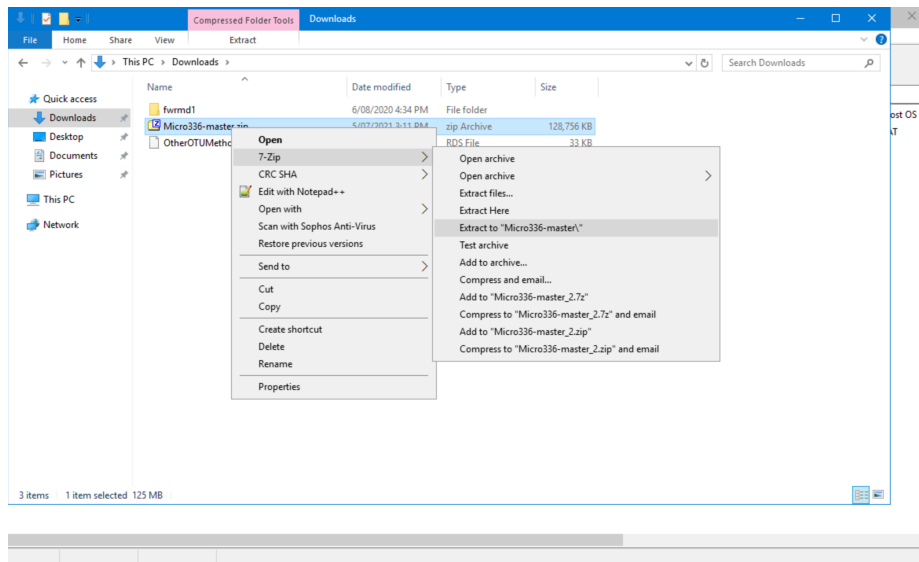


Figure 2. How to extract Micro336-master file

2.2.2.3 Opening R Studio.

Open R by browsing the Windows start menu (Applications / R Studio).

2.2.3 Using R

This class will use R for data analysis. R is a language and environment for statistical computing and graphics. The following section will review some of these concepts and introduce some new concepts for this paper.

2.2.4 A quick orientation with R Studio

When you open up R Studio, you will see something like this (**Figure 3**).

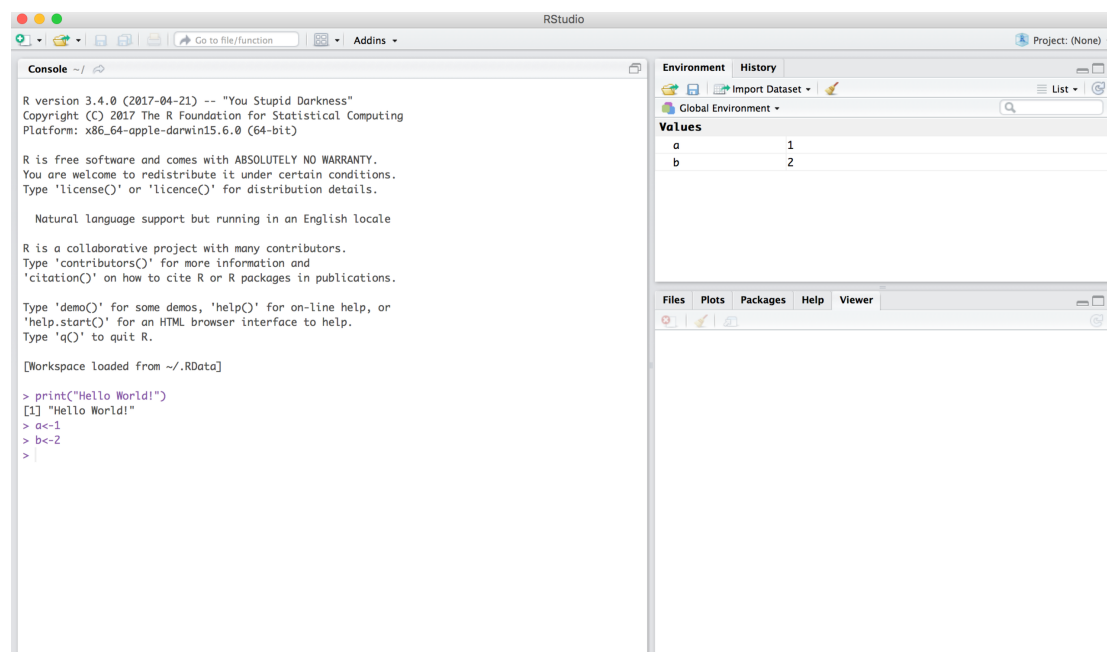


Figure 3. An overview of the R studio environment

The panel on your left is your R console. Anything you type in this window will be executed by R.

On the upper right, there are two tabs, (History) and (Environment), where you can see either the history of commands you have typed (History), or the variables you currently have in memory (Environment).

On the lower right, you have five tabs: (Plots), which will show any plots you have created; (Files) from which you can browse and open files; (Help) which you can use to view documentation; (Packages) which you can use to load R packages; and (Viewer) which is used to interface with local web content. For this class, you will most often use the Help and Plots tabs.

2.2.5 How to make an R markdown

R Markdown is a way to record the commands you use for an analysis in R, the output of R, and your own notes and comments. It therefore makes an ideal “computational lab notebook.” In Micro336, you will create R Markdowns for all assignments, and turn in your R markdown files to receive credit for your lab work.

In order to create an R markdown, you should do either of the following:

Option 1: (Recommended): Use the Micro336 RMD templates provided from Gitlab. Choose “File” followed by “Open File” from the R Studio menu. Browse to Documents/Micro336-master and open the provided template for each assignment.

Option 2: (Not recommended) Create a brand-new R markdown. This will create an example markdown file that is a bit more complicated than the Micro336 template. To do this, see the [Appendix 3](#).

After you have opened the template or created a new Markdown, you will see that your R studio now looks different. The R Markdown file is in the upper left, and your R console is on the lower left (**Figure 4**).

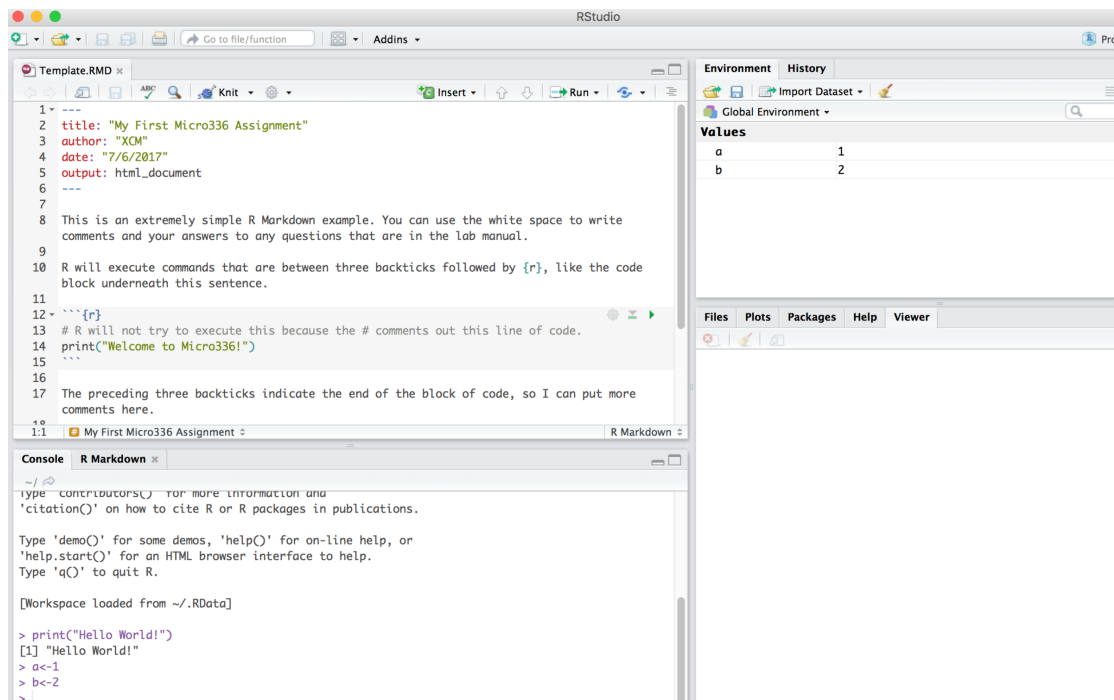


Figure 4. Making an R Markdown

An R Markdown file (RMD) has several important features you will need to understand for class.

1. White space in RMD is used for documentation (e.g. your comments).
2. Blocks of code executed by R are enclosed by backticks (```) and {r}, like this:

```
```{r}

This is a comment. R will not execute comments preceded by a number
sign.

plot(data)

```
```

In the example above, R will execute `plot(data)`, but it will ignore the comment because it is preceded by a `#`.

For class, you will be turning in your homework assignments as R markdown files. R markdown files can be knit into PDF or HTML files by pressing the “Knit” button

and choosing PDF or HTML output. HTML is more likely to work correctly, so it is recommended for class.

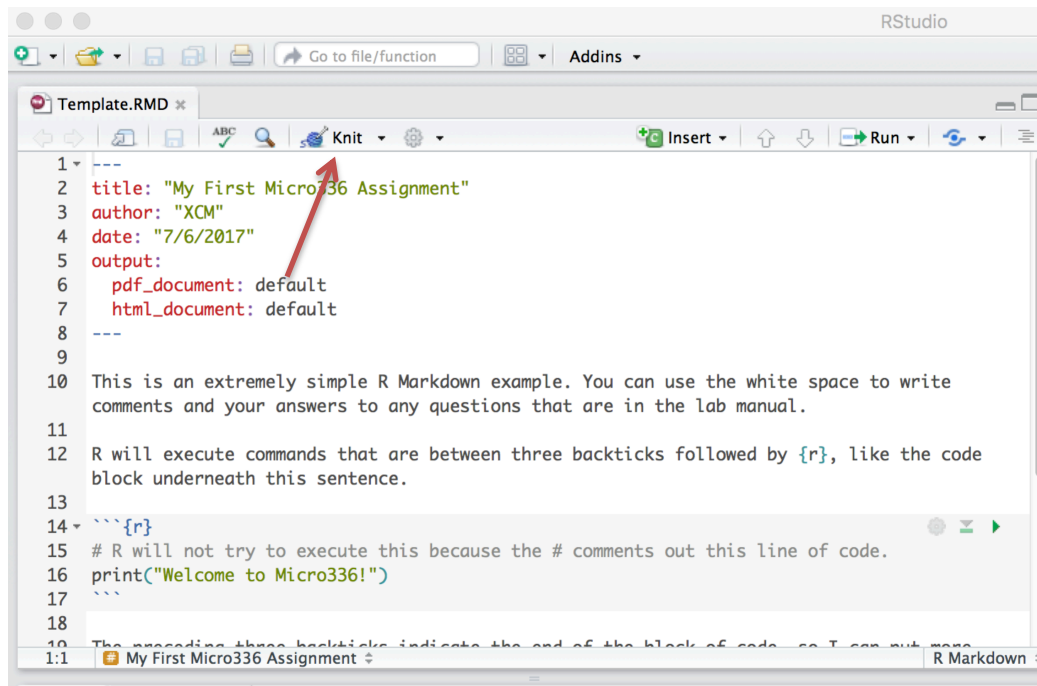


Figure 5. Knitting an R Markdown

If I knit the template (**Figure 5**), R will produce this HTML file (**Figure 6**) which nicely formats my comments, the R commands, and R's output from the commands.

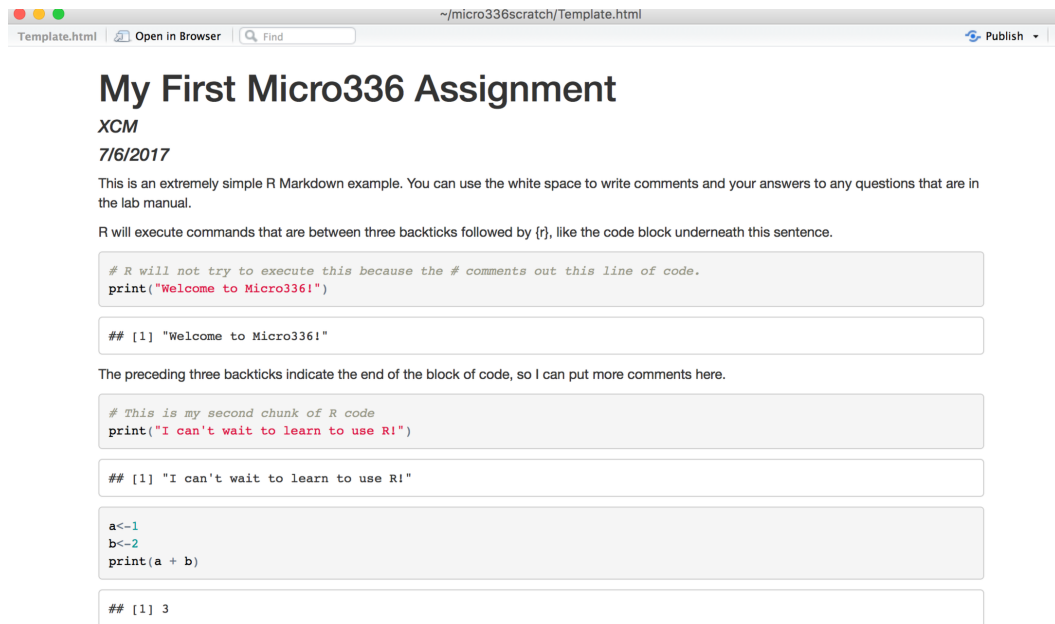


Figure 6. Example output from knitting an R markdown

This week you will begin to get familiar with some basic R commands, process some 16S rRNA data, and start using Phyloseq.

A final note: **you may find it convenient to test commands in the R console to make sure that they work correctly before pasting code that works correctly into your R markdown file.**

2.2.6 Your very first R exercises

You will use R Studio to create a Markdown file showing:

1. A few introductory R exercises
2. Your dada2 analysis.
3. Comparison of dada2, Mothur, and Qiime analysis

You will be able to use your R markdown to answer your quiz questions.

What should you turn in this week?

To get full credit for this week's work, you must take the Blackboard lab quiz (which will require you to use your Markdown and interpret your results), and you must upload your R markdown to Blackboard.

To turn in your Markdown, rename this file in the following format ("Yourlastname_yourfirstname_Micro336_Lab1.RMD". Then upload this file to our Micro336 Blackboard website, where there will be a folder for each lab assignment in the "Assignments" section.

2.2.6.1 *Introductory R Exercises*

From within R Studio, open `~/Micro336-master/Templates/Lab1.RMD`. Enter your name and date in the indicated sections on top.

When prompted by your instructor, click the first “run current chunk” button to see what happens.

```
```{r}

#R will not execute this comment because it starts with #, but it
will execute the code underneath

print ("Welcome to Micro336!")

```
```

Now, predict what will happen when you run the next chunk. Try running it to see if you predicted correctly.

```
```{r}

This is my second chunk of R code

print("I can't wait to learn to use R!")

a<-1

b<-2

print(a + b)

```
```

R can be used for simple math, plotting, and very complex calculations.

Inspect the next piece of code and predict what it does. Paste it into your RMD in “chunk 3” to test your prediction. Try modifying the code to describe and plot some other mathematical relationship.

```
shoes<-6
```

```
print(paste("I have", shoes *2, "shoes total in my house.",
collapse=""))

# Make some plots

x<-c(1:6)

plot(x, 2*x, xlab="pairs of shoes", ylab="total shoes")
```

R is fantastic for easily sorting, calculating, and plotting large quantities of data.

As an example, type the following commands into your console:

```
View(starwars)
```

How long would it take you to count how many of each species are in this data? With R, you can do it instantly with a single line of code! Try this:

```
table(starwars$species)
```

What if you want to visualize the relationship between height and weight in the Star Wars universe? Again, you can do it in a single line of code!

```
library(ggplot2)
ggplot(starwars, aes(x=mass, y=height)) + geom_point()
```

There's clearly a massive outlier in this data (it's Jabba the Hutt). Since most of our samples are human and droid, let's look at this relationship specifically and make nicer graphs. Again, with R you can do so much with a single line of code. We can

filter our data, make plots, and group by specific categories easily. It's very powerful!

```
starwars %>% filter(species %in% c("Human", "Droid")) %>% ggplot(  
  aes(x=mass, y=height, color=sex)) + geom_point() + geom_smooth() +  
  facet_wrap(~species, scales="free")
```

We could also easily perform a statistical test to determine whether male and female humans in the Star Wars universe have a difference in mean height.

```
library(rstatix)  
  
starwars %>% filter(species == "Human") %>% filter(!is.na(height)) %>%  
  t_test(height ~ sex)
```

This concludes our preview of the powerful features of R and demonstrates why it is useful for so many different kinds of data. We will now focus on using R to visualize and quantify microbial ecology data.

2.3 Using R to analyze 16S rRNA data

After we extract DNA, we send it to a sequencing facility and receive raw data, which must be processed into tables of operational taxonomic units (e.g. highly similar 16S rRNA sequences with assigned taxonomy and abundance data) in order to do microbial ecology analysis. Today, you will receive raw sequencing data exactly like you would receive from a sequencing facility. You will inspect the raw data to see what it looks like, and will use a bioinformatic tool, *dada2*, to process your raw sequencing data into a taxonomy table describing the abundances of various taxa in your data. Finally, you will compare your results to the results generated by *Mothur* and *Qiime*, two other commonly-used bioinformatic tools, to understand some strengths and weaknesses of each tool.

The specific goals of today's lab exercise are:

- 1) To process the Illumina data into a quality-filtered dataset which shows which taxa were present in each sample and at what abundance.

- 2) To compare our sequencing data to the known truth input, as well as to output generated by running Qiime and Mothur on the same samples.

At the end of this lab, you should have some idea which bioinformatic method most accurately represents the input DNA that was sequenced, and why.

We will use Dada2 to process our data for class, because it is easy to run entirely within R. Qiime and Mothur cannot be run in R, so these results have been pre-calculated for you to compare after you have processed your own data.

This schematic from Illumina (**Figure 7**) shows how the V4 hypervariable region of the 16S rRNA is amplified by PCR with the universal primers 505F and 806R. A barcode (index) is attached to the 806R primer. In paired-end Illumina sequencing, one sequencing primer hybridizes to the 515F primer and sequences in a forward direction, and the other sequencing primer binds to 806R and sequence in reverse. A third sequencing primer binds to P7 and sequences the read index (barcode) sequence. Along with the index sequences, this generates two reads (Read 1 and Read 2) which have some overlap in the middle. They will overlap less if the reads are shorter (e.g. 150 bp), and more if the reads are longer (e.g. 250 or 300 bp).

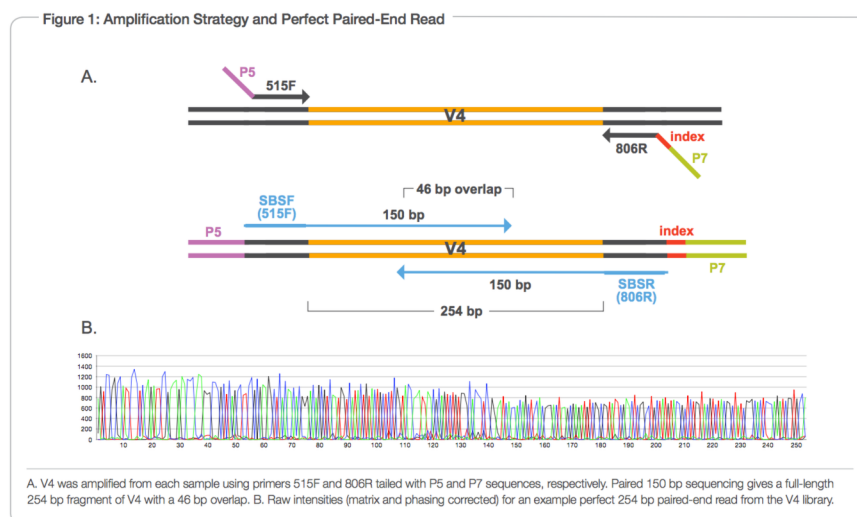


Figure 7. A schematic view of Illumina sequencing data. Source: https://www.illumina.com/documents/products/appnotes/appnote_miseq_16S.pdf

Broadly, when we process raw 16S rRNA data, we are doing the following things:

- 1) Quality controlling the reads by trimming off poor-quality data
- 2) Joining the two paired-end reads into a single sequence
- 3) Grouping similar sequences together and assigning taxonomy to the sequences
- 4) Determining the quantity of each sequence in each sample.

2.3.1 What kind of data comes back from the sequencing facility?

After you send samples to the sequencing facility, your sequencing data will be returned to you as one or more fastq files. You will often receive a total of three files comprising your entire multiplexed (combined samples) dataset (for instance, “Read1.fastq”, “Read2.fastq”, and “barcodes.fastq”. In this case, you will have to demultiplex your data according to the barcodes. Alternatively, you may receive your data already demultiplexed, in the form of many different files (for example: “sample1_R1.fastq”, “sample1_R2.fastq”, “sample2_R1.fastq”, “sample2_R2.fastq...).

2.3.2 Downloading the data we will use today

The data you will use for today's lab is located at <https://gitlab.com/morganlab/public/Micro336>.

You have already downloaded and unzipped this data.

Please read the README file inside the data subdirectory, so that you know what kind of data you are analysing today.

Use the README to answer the following question. Write down your answers – you will need to know this information later.

Question 1

(a) Describe the data you are analysing today.

(b) What was sequenced to generate these samples?

(c) How are these four samples different from one another?

Use WordPad or NotePad (**DO NOT USE MICROSOFT WORD!**) to open PE-R1.fastq, which is in the subdirectory "data".

You should see something that looks like this:

```
@M02810:149:000000000-APB17:1:1101:16793:1922 1:N:0:94
TGGGGAATATTGGACAATGGGGGGAACCCTGATCCAGCCATGCCGCGTGTGTGAAGAAGGCCTTATGGT
TGTAAGCACTTTAAGCGAGGAGGAGGCTACTTTAGTTAATACCTAGAGATAGTGGACGTTACTCGCAG
AATAAGCACCGGCTAACTCTGTGCCAGCAGCCGCGTAATACAGAGGGTGCAGACGTTAATCGGATTTA
CTGGGCGTAAAGCGTGCGTAGGCGGCTTATTAAGTCGGATGTGA
+
CDDDDCCFFFFFGGGGGGGGGGGGGGGGGGGGHHHHHHHHHHGHHHHHGGGGGGHHHEHHHHFHHHHHHGHGH
HGGGHHHHHHHHHHHHHGFGGCGGGGGGGHHHHHHHHHHHHHHHHHHHHHHGGGGGGGGGGGGGGGGGG
GGGGGGGGGGFF;>B.0:BFFFFFFFE0AF?/ACFFFFFFFB:BF0FEDDEFFD---
AAA.BFEFFFFF0:BB00.;BFFFFFF0.:.A;:AFBFAFAFFBBFF0FFFFFFF;FEB
```

A fastq file contains sequence and sequence quality information for one or more samples. A fastq file may contain information for millions of reads. Fastq files are formatted as a series of four lines per read. The first line (beginning with @) is the read identifier. The second line (TGGG...) is the actual DNA sequence of the read. The third line (+) may contain additional information. The fourth line (CDDD...) contains quality information (Q scores) for the read, but numbers have been encoded as ASCII characters instead of numbers to save space. Q scores are defined as a property that is logarithmically related to the base calling error probabilities.. $Q = -10 \log_{10} P$. The range of Q scores in the data is between 0 and 40, in which 0 is the worst possible read quality, and 40 is the best possible read quality. A Q-score of 30 represents a 1/1000 chance of sequencing error, whereas a Q-score of 20 represents a 1/100 chance of sequencing error.

Your fastq files contain tens of thousands of sequences of various quality, which you need to process.

2.3.3 Installing and loading R packages

Many R packages are not included with the base installation, but they have been preinstalled for you. You will need to load non-base libraries every time you use them, using the command “library”. Your code includes a chunk (**chunk1_setup**) needed to initialize the libraries you will need for processing and analysing your 16S rRNA data. Run this chunk now.

```
library(phyloseq)
library(dada2)
library(dplyr)
library(ggplot2)
```

You will need to reload these libraries with the “library” function before using them in every new R session.

2.3.4 Setting your working directory and sorting file paths

Before running the next piece of code, you may want to read the Appendix section on [working directories](#). This will help you understand how to point R to the correct directories where [today's data](#) is downloaded on your computer. R cannot open a file unless either it is using the same working directory as the one in which the file is located, or it has a complete file path.

Paste this code into a new chunk (**chunk2_workingdir**) to show R where to find the data you will be analysing.

```
# These commands to set the working directory and path are already
included in chunk1_setup in your template.

# Set working directory
setwd("~/Micro336-master/")

#Set path to data files
path <- "~/Micro336-master/data"
```

The next steps of processing your data are to tell R which files in your data directory are forward reads and which files are reverse reads, and to parse the sample names out of the files. Insert this piece of code into **chunk3_fileparse** in order to perform these steps.

```
#This piece of code assigns files that end with -R1.fq.gz to the list
of forward fastq files (fnFs), and files that end with -R2.fq.gz to
the list of reverse fastq files (FnRs)

fnFs <- sort(list.files(path, pattern="-R1.fq"))
fnRs <- sort(list.files(path, pattern="-R2.fq"))
```

```
# What are the actual names of our samples? Let's parse them out of
the files.

sample.names <- sapply(strsplit(fnFs, "-"), `[`, 1)

fnFs <- file.path(path, fnFs)
fnRs <- file.path(path, fnRs)
```

Insert this code into **chunk4_dadaQC** in order to have dada2 quality-control your data before determining which OTUs are present.

```
filt_path <- file.path(path, "filtered") # Place filtered files in
filtered/ subdirectory

filtFs <- file.path(filt_path, paste0(sample.names,
"_F_filt.fastq.gz"))

filtRs <- file.path(filt_path, paste0(sample.names,
"_R_filt.fastq.gz"))

#Use this command to filter and trim. It will truncate reads to an
#indicated length (informed by your RQC data), truncate poor-quality
#bases, and set the maximum acceptable error rate

out <- filterAndTrim(fnFs, filtFs, fnRs, filtRs,
                    maxN=0, maxEE=c(2,2), truncQ=2, rm.phix=TRUE,
                    compress=TRUE, multithread=TRUE)

head(out)

# Learn the profile of sequencing errors in the data

errF <- learnErrors(filtFs, multithread=TRUE)
errR <- learnErrors(filtRs, multithread=TRUE)
```

2.3.5 Making amplicons with dada2

Now that we have learned the error profile of the data, dada2 can merge sequences together and determine the unique sequences in the data after accounting for sequencing error. Insert the code below into **chunk5_runDada** in order to run this step.

```
derepFs <- derepFastq(filtFs, verbose=FALSE)
derepRs <- derepFastq(filtRs, verbose=FALSE)
# Name the derep-class objects by the sample names
names(derepFs) <- sample.names
names(derepRs) <- sample.names
dadaFs <- dada(derepFs, err=errF, multithread=FALSE)
dadaRs <- dada(derepRs, err=errR, multithread=FALSE)
mergers <- mergePairs(dadaFs, derepFs, dadaRs, derepRs,
verbose=FALSE)
```

We have now determined which are the unique sequences in the data, but we don't know which sequences are chimeras created during the PCR process. This step will compare all unique sequences in the data and remove all the sequences that are comprised of one half of one unique sequence, and one half of another unique sequence. Please insert the code below into **chunk6_chimeras** in order to remove chimeras from your data and compile your data into a table.

```
seqtab <- makeSequenceTable(mergers)
dim(seqtab)
table(nchar(getSequences(seqtab)))
seqtab.nochim <- removeBimeraDenovo(seqtab, method="consensus",
multithread=FALSE, verbose=TRUE)
dim(seqtab.nochim)
sum(seqtab.nochim)/sum(seqtab)
```

We now have a table of all the unique sequences in our data (and how many of them are present in each sample). Let's compare the number of sequences in our final dataset to the number of sequences in the original data to see how many of our sequences were lost during the processes of quality control, merging, and chimera removal. Please insert this code into **chunk7_tally** in order to create a summary of where sequences were lost during your bioinformatic pipeline. Stop here and think about how you can use this output to answer the following questions. Write down your answers – you will need to know this information later.

Question 2

- (a) Did the processed or the raw data lose a greater number of sequences during bioinformatic processing?
- (b) At which step were the most sequences lost? What happened?
- (c) How many chimeric sequences were removed?

```
getN <- function(x) sum(getUniques(x))

track <- cbind(out, sapply(dadaFs, getN), sapply(mergers, getN),
rowSums(seqtab), rowSums(seqtab.nochim))

colnames(track) <- c("input", "filtered", "denoised", "merged",
"tabled", "nonchim")

rownames(track) <- sample.names

head(track)
```

We have figured out what the unique sequences are, but right now they just look like strings of ~400 bp of DNA. Let's assign taxonomy to the sequences so that we know which taxa they represent. Insert this code into **chunk8_taxonomy** to compare each sequence to the sequences in the SILVA reference database and assign taxonomy.

2.3.6 Assigning taxonomy and analysing output

```
silva<-("~/Micro336-  
master/data/databases/silva_nr_v128_train_set.fa")  
  
taxa <- assignTaxonomy(seqtab.nochim, silva, multithread=FALSE)  
  
unname(head(taxa))
```

Congratulations – you have finally finished processing your data with dada2. We will now make it into a phyloseq object so that you can examine it using the phyloseq package. Please insert the following code into **chunk9_physeq** in order to make your data into a phyloseq object and to make barplots showing the total abundance and relative abundance of your data by phylum. Please note that by changing the “fill” parameter here, you can view your data at any taxonomic level, from kingdom to genus.

```
samples.out <- rownames(seqtab.nochim)  
subject <- samples.out  
samdf <- data.frame(Subject=subject)  
rownames(samdf) <- samples.out  
samdf$Data = c("PE", "PS", "RE", "RS")  
samdf$Method = c("dada2", "dada2", "dada2", "dada2")  
  
# Construct phyloseq object (straightforward from dada2 outputs)  
ps <- phyloseq(otu_table(seqtab.nochim, taxa_are_rows=FALSE),  
               sample_data(samdf),  
               tax_table(taxa))  
  
# What do our reads per sample & taxonomic distribution look like?  
plot_bar(ps, x="Subject", fill="Phylum")  
  
# Normalize the samples to relative abundance and replot  
ps.tx <- transform_sample_counts(ps, function(OTU) OTU/sum(OTU))  
plot_bar(ps.tx, x="Subject", fill="Phylum")
```

Now that you have your data in phyloseq / table form, you can finally begin to ask some interesting questions! Since we know the original composition of our

samples (both which species should be present and how abundant they were), we can see how well our results compare to the known truth (ie the DNA that was actually sequenced).

2.4 Comparing Dada2, Qiime, and Mothur

Insert the code below into **chunk 10_evaluate** in order to create the data objects you will need to answer these questions. You are loading a data object called “precalculated” which contains known truth, as well as precalculated Mothur and Qiime results. Then you are merging “Precalculated” with your own dada2 results which you have made today, into an object called “trio”.

```
#Load some data (truth, qiime, mothur)
precalculated<-readRDS("~/Micro336-master/data/OtherOTUMethod.rds")
sample_names(ps) = c("PE-Dada2", "PS-Dada2", "RE-Dada2", "RS-Dada2")

# merge our own data with the precalculated data
trio<-merge_phyloseq(otu_table(ps), otu_table(precalculated),
tax_table(ps), tax_table(precalculated), sample_data(ps),
sample_data(precalculated))
```

Now we’re going to ask how many unique DNA sequences and how many unique genera each method found. We will do this with the estimate_richness method, which you will use a lot for this course! Insert this code into **chunk11-richness** to evaluate, then use this analysis to answer the questions below.

```
estimate_richness(trio, measures=c("Observed"))

#Collapse data object 'trio' to genus level and recalculate
glommed<-tax_glom(trio, taxrank="Genus")
```

```
estimate_richness(glommed, measures=c("Observed"))
```

Question 3

- (a) How many unique sequences should we find?
- (b) What method found the most?
- (c) What method found the fewest?
- (d) What about for genera?

The methods clearly disagree about how many genera are present. Perhaps making a plot will help? Insert this code into **chunk12-barplots** to find out.

```
#Transform all samples to relative abundance
ps.tx <- transform_sample_counts(glommed, function(OTU) OTU/sum(OTU))

# Can we tell whether methods agree?
plot_bar(ps.tx, fill="Genus")
```

Based on the barplot, can you tell whether the methods agree on which genera are present and how abundant they are?

Clearly, a more efficient way to measure agreement or disagreement between any two samples would be useful. The piece of code below provides an example we can use to calculate agreement or disagreement between any samples. In the example below, you can change the sample you are comparing by changing the bit with `'filter(Sample=="")'` to the sample you want to compare. Insert this code into

Chunk13-intersections and modify as needed in order to answer the questions below.

```
#This changes the shape of our data to make the next steps easier
melted<-psmelt(ps.tx)

# Here, we filter our data to look only in the sample we care about
# (RS.True) and make a tally of how many times we see each genus at
# non-zero abundance in the sample. We assign this result to a
# variable called "a".

a<-melted %>% filter(Sample == "RS.True") %>% filter(Abundance > 0)
%>% group_by(Sample) %>% count(Genus)

# Here we do the same thing we just did for A, but the sample
# is "RS-Dada2" and the variable is b.
# This means we can now compare a and b easily.

b<-melted %>% filter(Sample == "RS-Dada2") %>% filter(Abundance > 0)
%>% group_by(Sample) %>% count(Genus)

#How many genera were found by dada2 and were in the real sample?
#intersect calculates things present in both sets
intersect(a$Genus, b$Genus)

#Setdiff tells you what is present in one list, but not the other
#list

#Use setdiff to determine the difference between genera found by
#dada2 and genera actually present?

#in a but not b
setdiff(a$Genus, b$Genus)

#in b but not a
setdiff(b$Genus, a$Genus)
```

```
plot_bar(ps.tx, fill="Genus")
```

Question 4 - Compare the datasets “RE.Qiime” and “RE.True”.

(a) How many genera do they both agree on?

(b) How many (and which) genera do they disagree about?

The lists are very helpful for determining which methods add (or miss) genera, but we would also like to know whether the methods measure the abundance of a genus accurately. Perhaps we can visualize this in a way that is more informative than the plot we made with chunk12. Use the code snippet below to make a few graphs to compare the abundance of each genus (as measured by each method) to the known truth (indicated by black lines). Insert these into **chunk14-finale** and use the graphs produced to answer questions below.

```
RS<-prune_samples((sample_names(ps.tx) %in% c("RS.Mothur",
"RS.Qiime", "RS-Dada2")), ps.tx)

RS_true<-prune_samples((sample_names(ps.tx) %in% c("RS.True")),
ps.tx)

RSmelt<-psmelt(RS) %>% select(Abundance, Sample, Genus)
RS_truemelt<-psmelt(RS_true) %>% select(Abundance, Sample, Genus)

RSplot<-merge.data.frame(RSmelt, RS_truemelt, by="Genus")

ggplot(RSplot, aes(x=Abundance.y, y=Abundance.x, colour=Sample.x)) +
geom_point() + scale_x_log10() + scale_y_log10() + ylab("Measured
Abundance") + xlab("True Abundance") +
scale_colour_discrete(name="Method Used") + ggtitle("Abundance of
genera when different methods are applied to sample RS") +
geom_abline(intercept=0, slope=1, linetype="dashed")
```

```

RE<-prune_samples((sample_names(ps.tx) %in% c("RE.Mothur",
"RE.Qiime", "RE-Dada2")), ps.tx)

RE.true<-prune_samples((sample_names(ps.tx) %in% c("RE.True")),
ps.tx)

REmelt<-psmelt(RE) %>% select(Abundance, Sample, Genus)
RE_truemelt<-psmelt(RE.true) %>% select(Abundance, Sample, Genus)

REplot<-merge.data.frame(REmelt, RE_truemelt, by="Genus")

ggplot(REplot, aes(x=Genus, y=Abundance.x, fill=Sample.x)) +
geom_bar(stat="identity", position="dodge") + theme(axis.text.x =
element_text(angle = 90, hjust = 1)) + geom_hline(yintercept=0.05,
linetype="dashed") + scale_fill_discrete(name="Method Used") +
ggtitle("Abundance of genera when different methods are applied to
sample RE")

```

We would also like to visualize the relationships between raw and processed outputs and known truth vs our data. Insert some of the code below into chunk14_finale in order to see how our results compare to known truths, and whether results were consistent across data QC methods. Use these plots to consider the following questions.

Question 5

(a) What should the measured abundance of every genus be in the RE community? Does any method achieve this? Is any method notably better or worse than any other?

(b) Interpret your RS graph. Is any method accurately measuring the abundance of genera? Is any method clearly superior or inferior?

(c) Based on your results, is 16S rRNA data an exact representation of actual DNA proportions in a sample?

(d) Considering all of your results today (total sequences and genera detected; extra / missing genera, properly measured abundance of taxa), which bioinformatic method performed best overall, and why do you think so?

Important capstones for the transition to next week:

2.5 Rarefaction curves

As we learned in lecture, observed alpha diversity is directly related to the number of reads in a sample. Most biological samples are quite diverse, and if we have only 100 sequencing reads, we will be able to detect less of the diversity than if we have 10,000 sequencing reads. Rarefaction curves were discussed in lecture as a way to estimate the true diversity of samples, and to determine whether or not we have sequenced deeply enough to detect most of the species that are present in the sample.

Due to the nature of the samples we sequenced today, we know exactly how many taxa should have been detected. Paste the following code into an r chunk to generate a rarefaction curve from this data. This will subsample your reads at a range of values and calculate observed alpha diversity for each sample at a range of read depths.

```
library(ranacapa)

p <- ggrare(trio, step = 1000, color = "Method", se = FALSE)

p + scale_y_log10() + scale_x_continuous(limits=c(1, 25000)) +
  facet_wrap(~Method)
```

Inspect the output graph and answer the following questions.

Question 6

- a) What is the true number of species in the data (and how many reads do you need to detect all of them?)
- b) Estimate the true number of species in the dada2 data (and how many reads you would need to detect all of them?)
- c) Estimate the true numbers of species in the Qiime and Mothur data (and how many reads you would need to detect all of them.)

2.6 Week 2 Phyloseq objects

Phyloseq is an R package developed by Paul J. McMurdie and Susan Holmes for analysis and plotting of microbiome data in R.

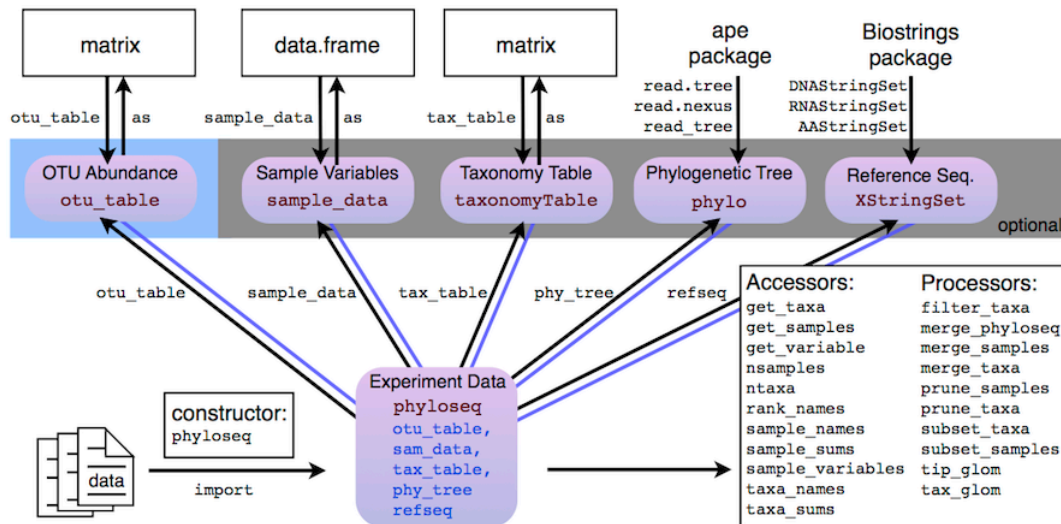


Figure 8. Construction of a *phyloseq* object

***phyloseq*: An R package for reproducible interactive analysis and graphics of microbiome census data** (2013) PLoS ONE 8(4):e61217 <http://dx.plos.org/10.1371/journal.pone.0061217>

The next section uses text from Vignettes associated with Bioconductor documentation maintained by Paul J. McMurdie. Please see the [Appendix](#) for license terms for this material. All text derived from these vignettes or other related online documentation are indicated by the citation “phylovign” and grey highlighting.

For a complete version please visit:

<https://www.bioconductor.org/packages/devel/bioc/vignettes/phyloseq/inst/doc/phyloseq-basics.html>, or go directly to <http://joey711.github.io/phyloseq/>

phyloseq allows integration of different types of datasets (termed classes) into a single ‘master’ dataset termed a *phyloseq* object (see below). The advantage of this is that all data associated with a sample (sequencing information, metadata, taxonomy, etc) are linked using a single identifier but can retain their original structure. *phyloseq* uses 3 required data classes and one non-essential object:

Required-

- (1) OTU abundance table (otu_table)
- (2) table of sample data or metadata (sample_data)
- (3) table of taxonomic descriptors or taxonomic assignments (taxonomyTable)

Non-essential: a phylogenetic tree ("phylo"-class)

Once in *phyloseq*, individual components of the *phyloseq* object can be retrieved or manipulated using commands termed 'accessors' or 'processors' respectively. *Phyloseq* also allows you to directly link with other packages within R to further manipulate or graph your data.

2.7 Getting started

2.7.1 Load *phyloseq* ³

To use *phyloseq* in a new R session, you will have to 'load' the *phyloseq* package and check the installed version:

```
require("phyloseq")  
packageVersion("phyloseq")  
[1] '1.36.0'
```

The number 1.36.0 bottom line (double hashed) indicates the current version of *phyloseq* loaded in R. Packages can also be loaded using the following command:

```
library("phyloseq")
```

If you do not have the package you will need to install it. This has already been installed during week 1. However, if you ever need to do this in another environment, please follow the instructions in the [Appendix](#).

2.8 Data analysis-HMP Case study

In the previous section you will use a publicly available dataset (HMPv35) from the Human Microbiome Project found in [the HMP QIIME Community Profiling data page](#) and code from tutorials created by the curators of phyloseq. This data is available in your Micro336 directory that you have previously downloaded (~/.Micro336-master/data/HMPv35.RData). You can load it with the following command, which will store the data as a variable named “HMPv35”. The [Appendix](#) includes instructions for how to download this file from its original source.

```
load("~/Micro336-master/data/HMPv35.RData")
```

Check your file after loading it:

```
HMPv35
```

```
phyloseq-class experiment-level object
otu_table() OTU Table:          [ 45336 taxa and 4743 samples ]
sample_data() Sample Data:      [ 4743 samples by 10 sample variables ]
tax_table()  Taxonomy Table:    [ 45336 taxa by 6 taxonomic ranks ]
phy_tree()   Phylogenetic Tree: [ 45336 tips and 45099 internal nodes ]
refseq()     DNASTringSet:      [ 45336 reference sequences ]
```

2.8.1 Investigate Some Basic Features of HMPv35

For more advanced tools for exploring the data, see [additional phyloseq documentation](<http://joey711.github.com/phyloseq/>)

Lets see what basic information we can glance from our dataset:

```
#How many taxa are found
```

```
ntaxa(HMPv35)
```

```
[1] 45336
```

```
#How many samples are found
```

```
nsamples(HMPv35)
```

```
[1] 4743
```

```
#How is my taxonomy organized (how many ranks are present)?
```

```
rank_names(HMPv35)
```

```
[1] "Rank1" "Phylum" "Class" "Order" "Family" "Genus"
```

```
#What are my sample variables
```

```
sample_variables(HMPv35)
```

```
[1] "X.SampleID"      "RSID"            "visitno"
[4] "sex"             "RUNCENTER"       "HMPbodysubsite"
[7] "Mislabeled"      "Contaminated"    "Description"
[10] "HMPbodysite"
```

```
#Above you can see bodysubsite as a variable. You can determine how
many different body subtypes there are by using 'level'
```

```
levels(sample_data(HMPv35)$HMPbodysubsite)
```

```
[1] "Anterior_nares"      "Attached_Keratinized_gingiva"
[3] "Buccal_mucosa"       "Hard_palate"
[5] "Left_Antecubital_fossa" "Left_Retroauricular_crease"
[7] "Mid_vagina"          "Palatine_Tonsils"
[9] "Posterior_fornix"    "Right_Antecubital_fossa"
[11] "Right_Retroauricular_crease" "Saliva"
[13] "Stool"               "Subgingival_plaque"
[15] "Supragingival_plaque" "Throat"
[17] "Tongue_dorsum"       "Vaginal_introitus"
```

```
#these are too fine scale so lets try
```

```
levels(sample_data(HMPv35)$HMPbodysite)
```

```
[1] "Oral" "Skin" "Stool" "Vagina"
```

```
#Let's imagine you only wish to explore a subset of your data, for
example a single bodysubsite. You can subset your data using a
variable with 'subset_samples'
```

```
HMPv35saliva <- subset_samples(HMPv35, HMPbodysubsite == "Saliva")
```

```
HMPv35saliva
```

```
phyloseq-class experiment-level object
```

```
otu_table() OTU Table: [ 45336 taxa and 290 samples ]
sample_data() Sample Data: [ 290 samples by 10 sample variables ]
tax_table() Taxonomy Table: [ 45336 taxa by 6 taxonomic ranks ]
phy_tree() Phylogenetic Tree: [ 45336 tips and 45099 internal nodes ]
refseq() DNASTringSet: [ 45336 reference sequences ]
```

Notice that you went from **4743 samples** to only **290 samples**

2.8.2 Exploring basic HMPv35 dataset features

Before you start analysing data you want to be familiar with your dataset. This means ensuring that your data is of good enough quality to support any observations made. We will start by exploring coverage (or the depth of sequencing per samples) to ensure samples have equal coverage thus allowing direct comparison between samples. We will start with the full data set and then focus on a subset for ease of and speed of analysis.

2.8.2.1 Check sequencing coverage

```
#How many taxa do we have per sample [only samples 1-10]:
taxa_sums(HMPv35)[1:10]
```

| | | | | | |
|--------------|--------------|--------------|--------------|--------------|--------------|
| OTU_97.15099 | OTU_97.13686 | OTU_97.30326 | OTU_97.26112 | OTU_97.34719 | OTU_97.12776 |
| 111 | 616 | 1469 | 114 | 161 | 12 |
| OTU_97.31377 | OTU_97.25805 | OTU_97.21215 | OTU_97.2726 | | |
| 57 | 60 | 33 | 134 | | |

It is obvious that the number of detected taxa changes per sample. To ensure that this is driven by biological/experimental reasons and not sampling bias we need to ensure equal sample coverage.

```
#How many sequences do we have per sample:
sample_sums(HMPv35)
```

| | | | | | | | |
|-----------|-----------|-----------|-----------|-----------|-----------|-----------|-----------|
| 700114607 | 700114424 | 700114712 | 700114713 | 700114554 | 700114606 | 700114335 | 700114707 |
| 2 | 24 | 5137 | 5096 | 4571 | 5247 | 6295 | 3333 |
| 700114442 | 700114610 | | | | | | |
| 4628 | 1642 | | | | | | |

Based on only ten samples it looks like coverage is very different. To confirm this we can plot the number of sequences/OTUs per sample using the package `ggplot2`, as shown in **(Figure 9)**.

```
#load library
library(ggplot2)
```

```

#determine the sum total taxa per sample, sort and make into a new
data frame

readsumsdf <- data.frame(nreads = sort(taxa_sums(HMPv35), TRUE),
sorted = 1:ntaxa(HMPv35), type = "OTU")

#repeat above for sequences per sample

readsumsdf <- rbind(readsumsdf, data.frame(nreads =
sort(sample_sums(HMPv35), TRUE), sorted = 1:nsamples(HMPv35), type =
"Samples"))

#you can manipulate attributes in a figure using standard ggplot
commands (see http://www.cookbook-r.com/Graphs/ or
http://docs.ggplot2.org/0.9.2.1/index.html).

#plot results

p <- ggplot(readsumsdf, aes(x = sorted, y = nreads)) +
  geom_bar(stat = "identity")+
  ggtitle("Total number of reads") + scale_y_log10() +
  facet_wrap(~type, 1, scales = "free")

p

```

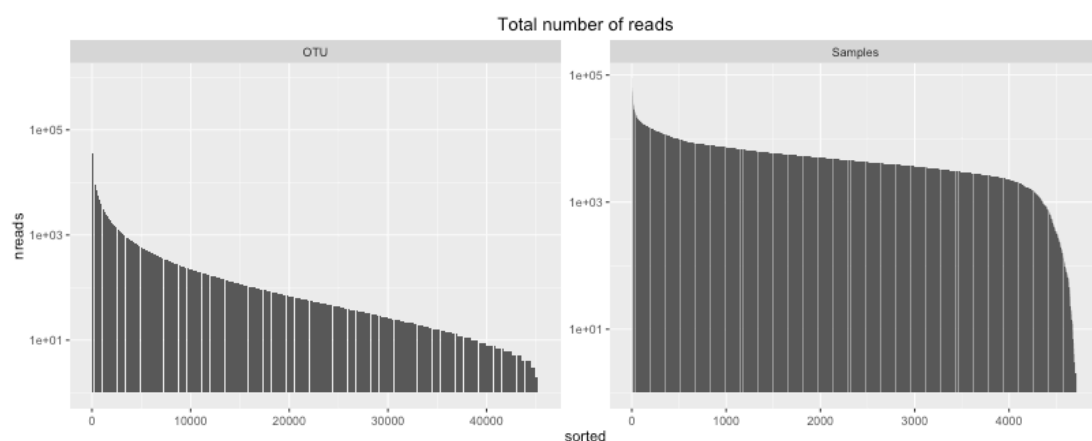


Figure 9. Distributions of OTU abundance and coverage in the HMP dataset (all samples)

The plot on the left (**Figure 9**) shows how abundant each OTU is. The one on the

right shows how coverage changes by several orders of magnitude over the samples in our dataset. This suggests that observed patterns might be driven by uneven coverage instead of experimental variables. Why does this matter? Let's see how uneven coverage could influence species detection.

```
#Calculate the number of sequences per sample
sums<-sample_sums(HMPv35)
summary(sums)
```

| Min. | 1st Qu. | Median | Mean | 3rd Qu. | Max. |
|------|---------|--------|------|---------|-------|
| 1 | 2908 | 4439 | 5419 | 6692 | 87836 |

You can see that some samples have only 1 sequence, while another has 87836, with a median of 4439. What about Observed species richness?

```
#calculate Observed species richness for all samples
alpha_summary_H35<-estimate_richness(HMPv35, measures=c("Observed"))

#Attach metadata
alpha_meta_H35<-cbind(alpha_summary_H35,
data.frame(sample_data(HMPv35)))

#Combine
alpha_meta_H35<-cbind(alpha_meta_H35, sums)

#Now we can create a visual
library(DEGreport)
library(ggrepel)
library(ggpubr)

ggplot(alpha_meta_H35, aes(x=sums, y=Observed, color=HMPbodysite)) +
  geom_point() + xlab("Reads per sample") +
  ylab("Observed richness")+
  geom_smooth(method=lm)+
  stat_cor(method = "spearman")
```

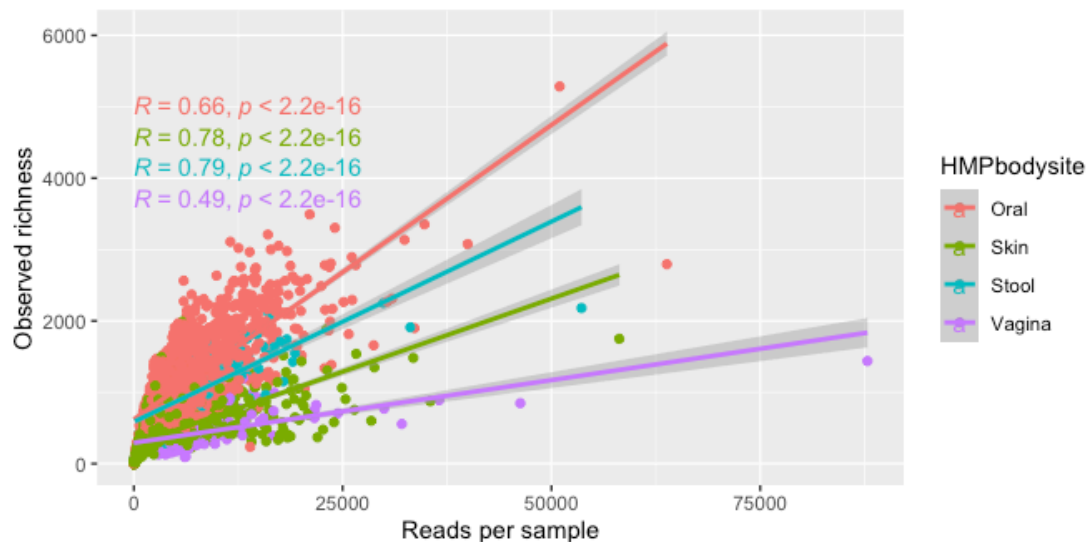


Figure 10. Detection of new species with increasing read depth. Statistics: Spearman correlation.

We can clearly see that as we sequencing deeper, we detect more species across all body sites (**Figure 10**).

```
#statistical test, Spearman correlation across all samples
cor.test(alpha_meta_H35$Observed, alpha_meta_H35$sums,
method="spearman")
```

Spearman's rank correlation rho

```
data: alpha_meta_H35$Observed and alpha_meta_H35$sums
S = 8.099e+09, p-value < 2.2e-16
alternative hypothesis: true rho is not equal to 0
sample estimates:
rho
0.5445659
```

This confirms the prior take-home, the more sequences you have the more species you detect. But since not all samples have the same sequence depth this creates issues.

To explore this further we will use the subsetted data from above (saliva only) to make it easier to handle. If we plot coverage for this subset (**Figure 11**) it looks like this:

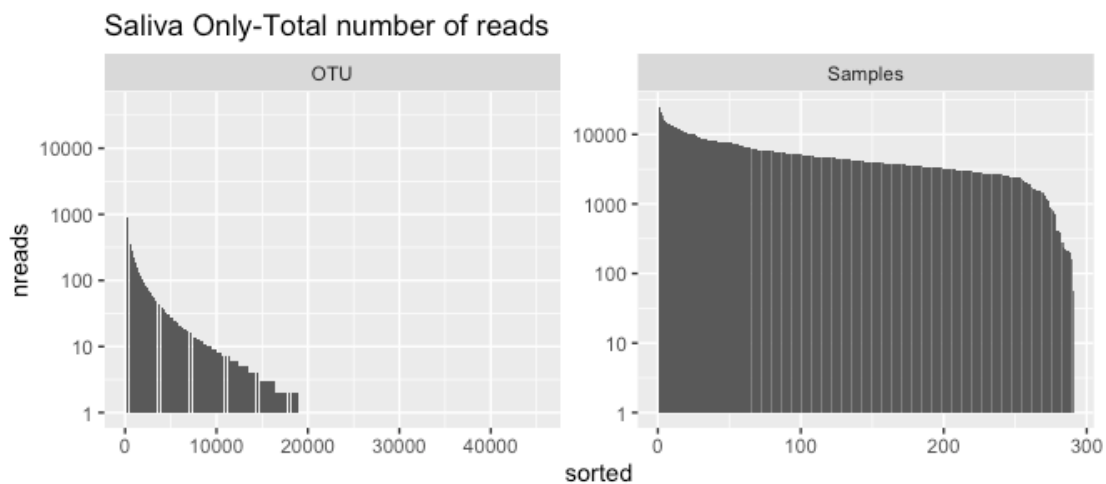


Figure 11. Distributions of OTU abundance and coverage in HMP data, saliva samples only

This means over all saliva samples coverage ranges in the hundreds to tens of thousands. Since we simply subset the data, but did not remove OTUs not found in saliva samples many OTUs show up as absent. This can impact on downstream analysis. Let's see the impact these have on alpha and beta diversity measurements.

2.8.3 Calculate alpha diversity

Alpha diversity encompasses multiple measurements that account for the diversity found within a single sample/treatment/etc. These include total numbers of distinct taxa [richness, also called 'Observed' for observed richness in phyloseq.], as well as several 'diversity' measures including Shannon, Simpson and others. While you can plot results directly, it is useful to have a record of the actual values. These can be stored in a new file as a record, or saved for further analysis in other packages by using the following commands:

```
# Calculate alpha diversity [Richness, Shannon] for both sexes using
untrimmed

alpha_summary_H35_sal<- estimate_richness(HMPv35saliva, measures =
c("Observed", "Shannon"))

# Combine results with sample_data. This is useful if we wish to plot
later against other variable or information.

alpha_meta_H35_sal <- cbind(alpha_summary_H35_sal,
data.frame(sample_data(HMPv35saliva)))
```

To visualize your results you can do a simple boxplot (**Figure 12**).

```
#Plot observed richness and Shannon diversity values for both sexes
p <- plot_richness(HMPv35saliva, x = "sex", measures=c("Observed",
"Shannon"))

#Add a boxplot using the geom function of ggplot
p + geom_boxplot()
```

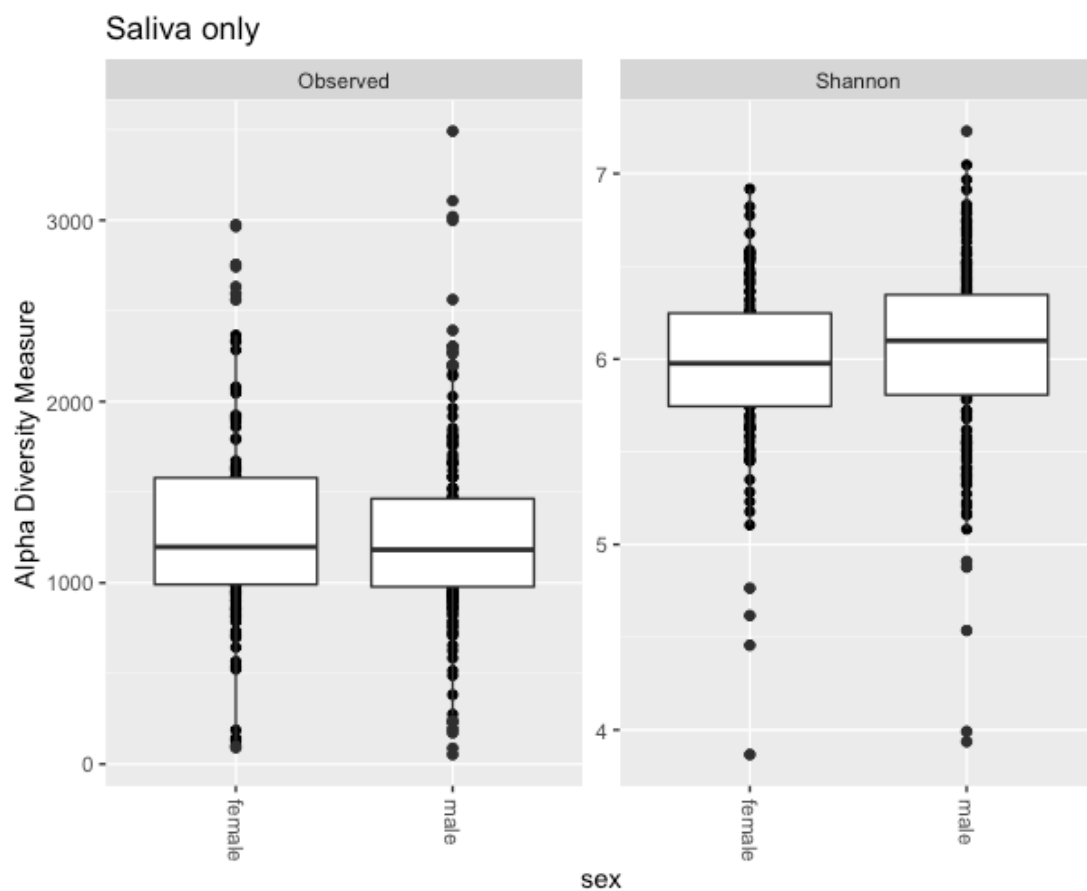


Figure 12. Boxplots of alpha diversity in HMP saliva samples

Since we know that not all samples have the same amount of sequences (**Figure 11**), let's fix that problem using rarefaction curves.

2.8.3.1 Rarefaction curves

Since the HMP dataset is very large we will start by subsetting it to a smaller

number of samples. We will focus on only "Saliva", "Stool", "Throat" subsites and only retain samples with at least 100 sequences.

```
#set the subsites we wish to use
HMPsubsample <- c("Saliva" , "Stool", "Throat")

# Subsample to only three specific subsites
HMPsubsampled<-subset_samples(HMPv35, HMPbodysubsite %in%
HMPsubsample)

#remove any samples with less than 100 sequences
pruned<-prune_samples(sample_sums(HMPsubsampled)>=100, HMPsubsampled)
pruned
```

```
phyloseq-class experiment-level object
otu_table() OTU Table: [ 45336 taxa and 913 samples ]
sample_data() Sample Data: [ 913 samples by 10 sample variables ]
tax_table() Taxonomy Table: [ 45336 taxa by 6 taxonomic ranks ]
phy_tree() Phylogenetic Tree: [ 45336 tips and 45099 internal nodes ]
refseq() DNASTringSet: [ 45336 reference sequences ]
```

Now we only have 913 samples to work with.

```
library(ranacapa)

# Calculate rarefaction curves, then plot and color by subsite
p <- ggrare(pruned, step = 1000, color = "HMPbodysubsite", se =
FALSE,parallel = TRUE)+

  scale_y_log10()+

  scale_x_continuous(limits=c(1, 25000))

p
```

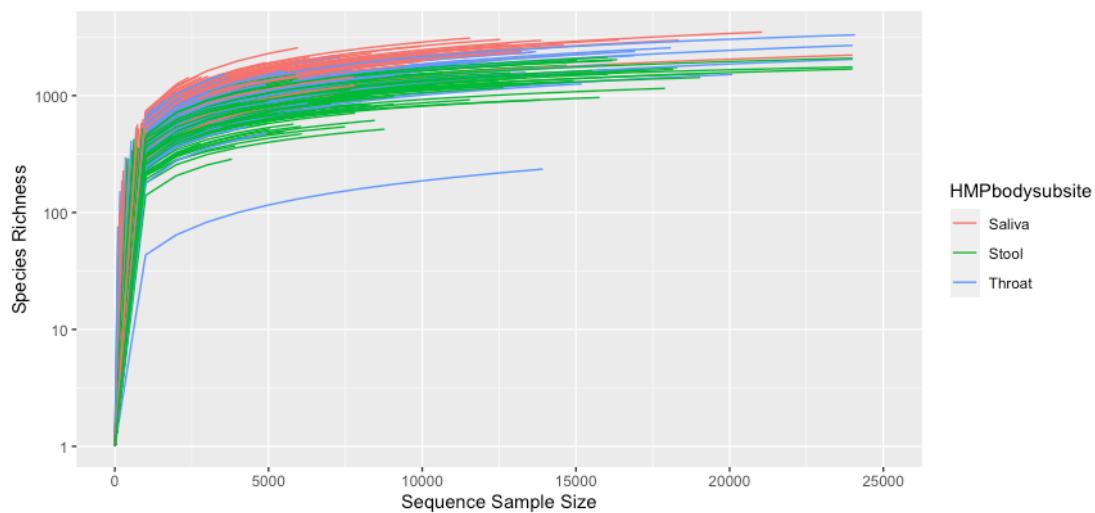


Figure 13. Rarefaction curves showing species detected along levels of sampling for all HMPbodysubsites.

We can see that the length of the lines (representing number of sequences per sample) is not consistent for all samples (**Figure 13**). This is important since the more we sequence the more species we can detect affected our interpretation of richness in a sample. To see better how it affects across subsites, lets facet.

```
#to facilitate visualization due to different sampling depth lets facet
p+facet_wrap(~HMPbodysubsite)
```

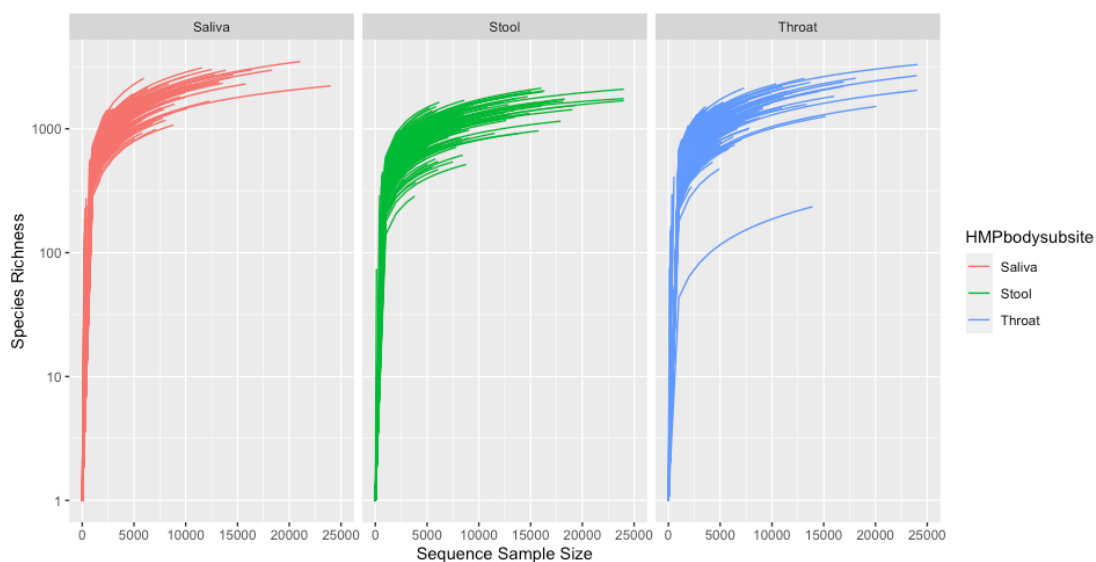


Figure 14. Rarefaction curves faceted by HMPbodysubsite

Three things are clearer now:

- 1) Observed richness clearly increases as read depth increases in all subsites
- 2) Not all samples have the same sequencing depth
- 3) Species detection has not been saturated (not all species have been detected i.e. the curve is not flat yet)

To allow comparison of alpha diversity under equivalent condition we need to subsample to an equal sequencing depth. This can be achieved by rarefying our dataset.

2.8.3.2 “Rarefy” Abundances to Even Depth

In order to normalize the coverage for each sample we will rarefy (randomly subsample) all data to the same level. This is done by subsampling to a determined depth. We can look at our coverage plot and determine what depth we wish to use. This needs to be a number large enough to allow for detection of patterns, but low enough to maximize the number of samples we will be able to analyze. Our previous plot gives a good overview, but it is hard to determine what sequence depth is the best to use.

For this example, an arbitrary number (1000) was chosen to reduce the size of the dataset in order to increase the speed of downstream analyses. In your own research you should choose a value that best suits your dataset/study (i.e. it does not have to be 1000). Once rarefied we will redo all prior alpha diversity analyses and compare results. Since this is a random sampling we will use a ‘seed’. It is useful to set a seed when you subsample so your result is reproducible. It allows the process to always start from the same point instead of a new one each time.

```
#Rarefy to even sequencing depth (1000 sequences)
H35_sal_rare <- rarefy_even_depth(HMPv35saliva, 1000, rngseed=TRUE)
```

If we re-plotted Figure 11 the right panel should be flat (consistent across all samples) across the 1000 sequence mark. If we summarize the alpha diversity results and compare them to our raw dataset you can see large changes in the generated values.

```
> summary(alpha_summary_H35_sal)      > summary(alpha_summary_H35_sal_rare)
  Observed      Shannon      Observed      Shannon
Min.   : 53.0   Min.   :3.867   Min.   :250.0   Min.   :4.486
1st Qu.: 980.8   1st Qu.:5.783   1st Qu.:401.0   1st Qu.:5.324
Median :1184.5   Median :6.062   Median :444.0   Median :5.505
Mean   :1286.1   Mean   :6.008   Mean   :444.4   Mean   :5.509
3rd Qu.:1515.0   3rd Qu.:6.304   3rd Qu.:489.0   3rd Qu.:5.720
Max.   :3492.0   Max.   :7.229   Max.   :653.0   Max.   :6.251
```

These changes had a small impact on the observed median based on sex (Figure 15).

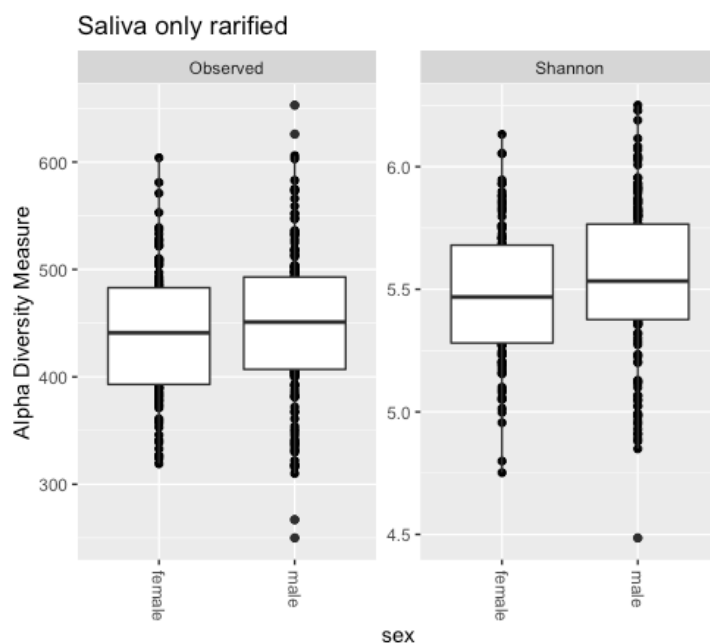


Figure 15. Alpha diversity of HMP saliva samples after rarefaction

In order to confirm if the observed change is statistically significant we can perform a t-test using the following code:

```
#Use alpha summary to calculate means and do a t-test
t.test(Observed ~ sex, data = alpha_meta_H35_sal)
t.test(Observed ~ sex, data = alpha_meta_H35_sal_rare)
```

| | |
|---|--|
| <pre>> t.test(Observed ~ sex, data = alpha_meta_H35_sal) Welch Two Sample t-test data: Observed by sex t = 0.70655, df = 243.9, p-value = 0.4805 alternative hypothesis: true difference in means between group female and group male is not equal to 0 95 percent confidence interval: -85.08906 180.27605 sample estimates: mean in group female mean in group male 1314.202 1266.608</pre> | <pre>> t.test(Observed ~ sex, data = alpha_meta_H35_sal_rare) Welch Two Sample t-test data: Observed by sex t = -1.0792, df = 257.49, p-value = 0.2815 alternative hypothesis: true difference in means between group female and group male is not equal to 0 95 percent confidence interval: -24.523769 7.160405 sample estimates: mean in group female mean in group male 439.2072 447.8889</pre> |
|---|--|

If we compare the results pre and post rarefaction it does not change the results, as it is not significant ($p > 0.05$). While this is the case in this scenario, for other comparisons rarefaction and removal of subsets of the data can have significant effects and should be accounted for in your analysis.

2.8.3.3 Non-parametric test

To confirm the above results we will use the non-parametric Kruskal-Wallis test, or one-way ANOVA on ranks.

In order to confirm if the observed change is statistically significant we can perform a t-test using the following code:

```
#confirm results using non-parametric test

library(microbiome)

library(ggpubr)

kruskal.test(Observed ~ sex, data = alpha_meta_H35_sal_rare)
```

```
> kruskal.test(Observed ~ sex, data = alpha_meta_H35_sal_rare)

Kruskal-Wallis rank sum test

data: Observed by sex
Kruskal-Wallis chi-squared = 1.5028, df = 1,
p-value = 0.2202
```

When we test for differences in observed species there is no significant effect ($p > 0.05$). An effective way to show this within your visual is to use the `stat_compare_means` command.

```
##create boxplot with stats in the figure
```

```
p +
  geom_boxplot() +
  stat_compare_means(method = "kruskal.test", paired = TRUE)
```

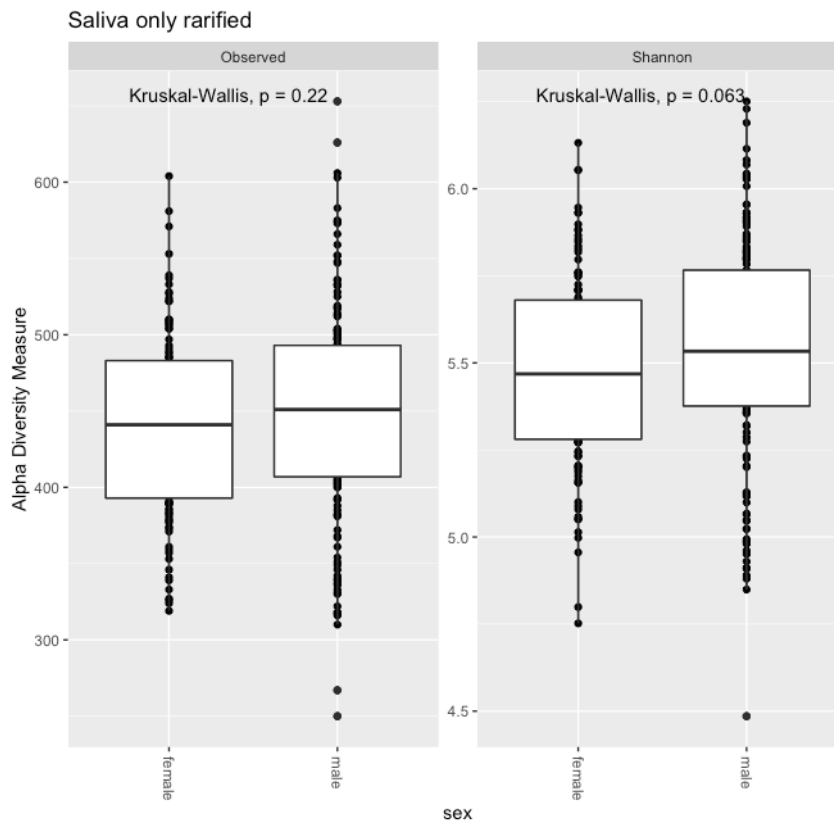


Figure 16. Refer to Figure 15. Plot re-drawn now showing results for `kruskal.test` for changes based on global mean.

The `stat_compare_means` command will give a global p value across all categories. If you want to compare specific combinations you can provide a list of pairwise comparisons you want.

```
##create boxplot with stats in the figure
my_comparisons <- list( c("female", "male"))

p <- plot_richness(H35_sal_rare, x = "sex", measures=c("Observed",
"Shannon")) +

  geom_boxplot() +
```

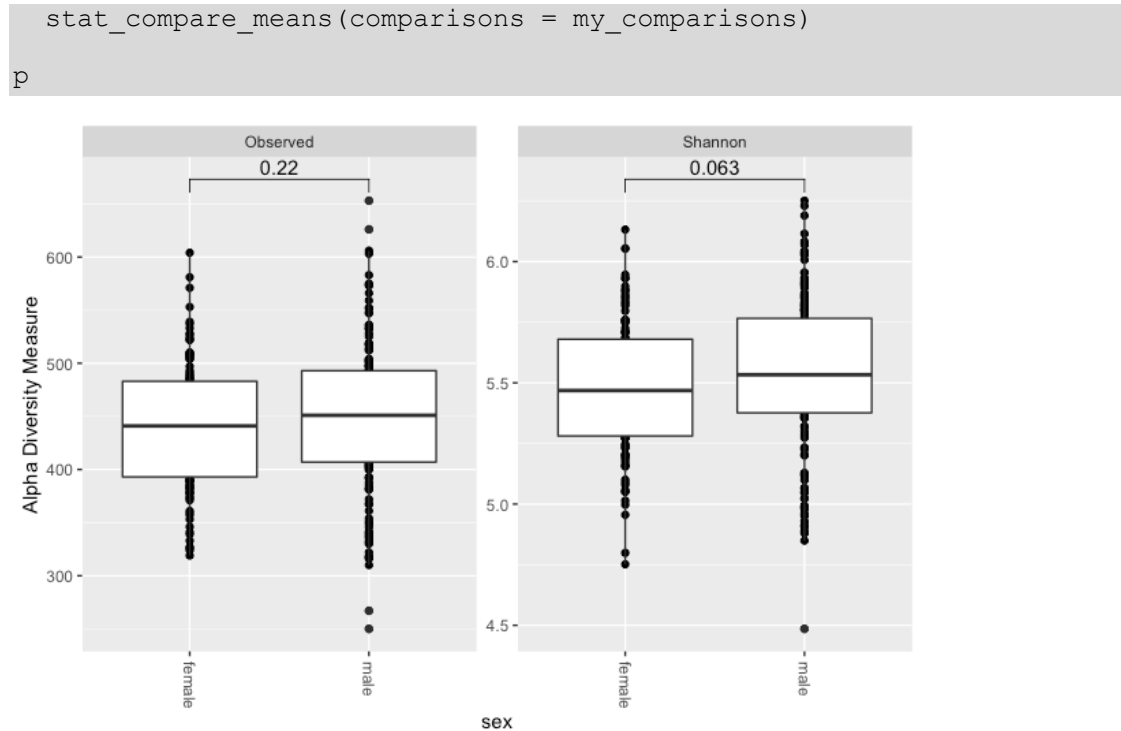


Figure 17. Refer to Figure 16. Plot re-drawn now showing results specific comparisons between categories.

2.8.4 Compare beta diversity

Now that we know what the alpha diversity is, we can compare these across samples to determine the beta diversity. We will use NMDS plotting of Bray Curtis distances to visualize this. Detailed explanation can be found in: <https://sites.google.com/site/mb3gustame/dissimilarity-based-methods/nmds> A sample code is below:

For NMDS plots it's important to set a seed since the starting positions of samples in the algorithm is random.

```
#Ordinate data using Non-metric multidimensional scaling (NMDS) on
Bray-Curtis dissimilarity (distances)

set.seed(1)

NMDS.ord <- ordinate(HMPv35saliva, "NMDS", "bray")

#By calling the newly created file we can get the stress value of the
plot
```

```
NMDS.ord
```

```
> NMDS.ord
```

```
Call:
```

```
metaMDS(comm = veganify0TU(physeq), distance = distance)
```

```
global Multidimensional Scaling using monoMDS
```

```
Data: wisconsin(sqrt(veganify0TU(physeq)))
```

```
Distance: bray
```

```
Dimensions: 2
```

```
Stress: 0.1985491
```

```
Stress type 1, weak ties
```

```
No convergent solutions - best solution after 20 tries
```

```
Scaling: centring, PC rotation, halfchange scaling
```

```
Species: expanded scores based on 'wisconsin(sqrt(veganify0  
TU(physeq)))'
```

NMDS ordinations should always indicate their stress values as a measure of quality. *Clarke 1993* suggests the following guidelines for acceptable stress values: <0.05 = excellent, <0.10 = good, <0.20 = usable, >0.20 = not acceptable. Stress values at or above 0.2 are deemed suspect and a stress value approaching or exceeding 0.3 indicates that the ordination is arbitrary. Our values are lower than 0.2 so not ideal, but not bad. Another way to look at it is through a Shepard's stress plot (Figure 18).

You will need to load the vegan library for this:

2.8.4.1 Stressplot

```
library(vegan)
```

```
stressplot(NMDS.ord)
```

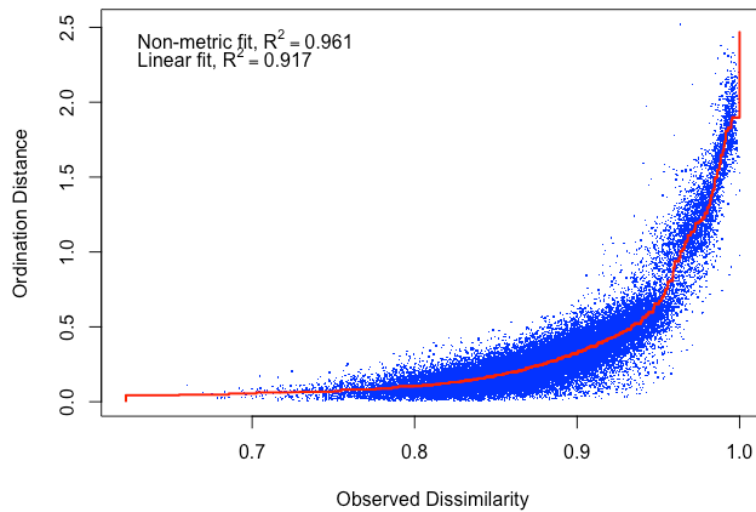


Figure 18. Stressplot for NMDS ordination

If there is a large amount of scatter (i.e. a poor fit), the ordination is not representative of the original distances and any plot to visualize the data cannot be trusted. In this case it is good enough to proceed.

Let's make an NMDS plot using our data and see how it looks:

```
sampleplot<-plot_ordination(HMPv35saliva, NMDS.ord, type = "samples",
color = "sex")

sampleplot
```

Raw data

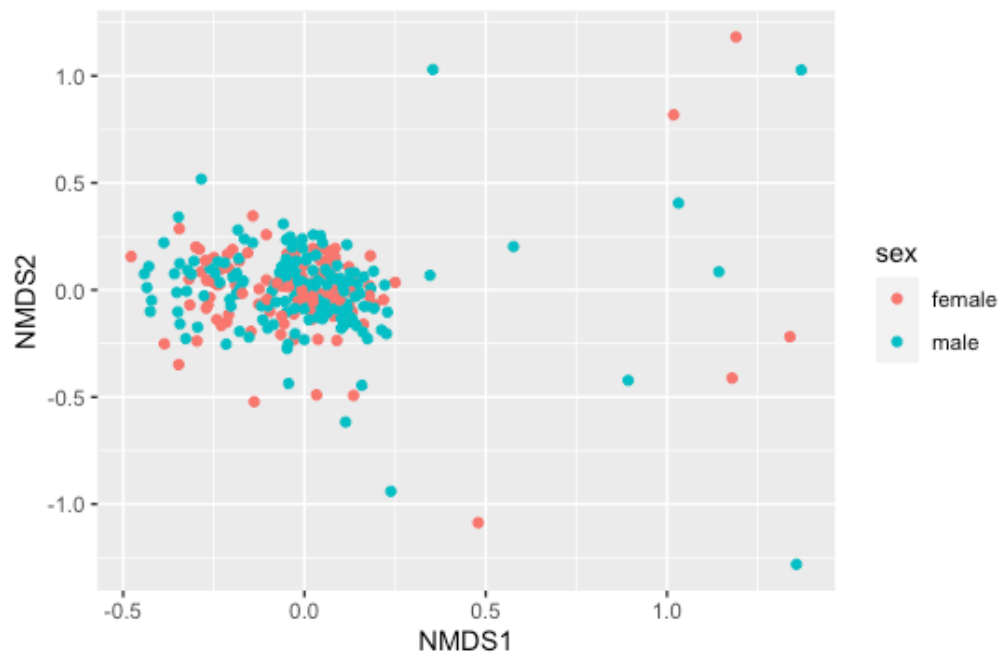


Figure 19. NMDS plot based on raw (unrarified) data comparing samples based on sex.

If we repeat the process using our rarefied data:

Rarified data

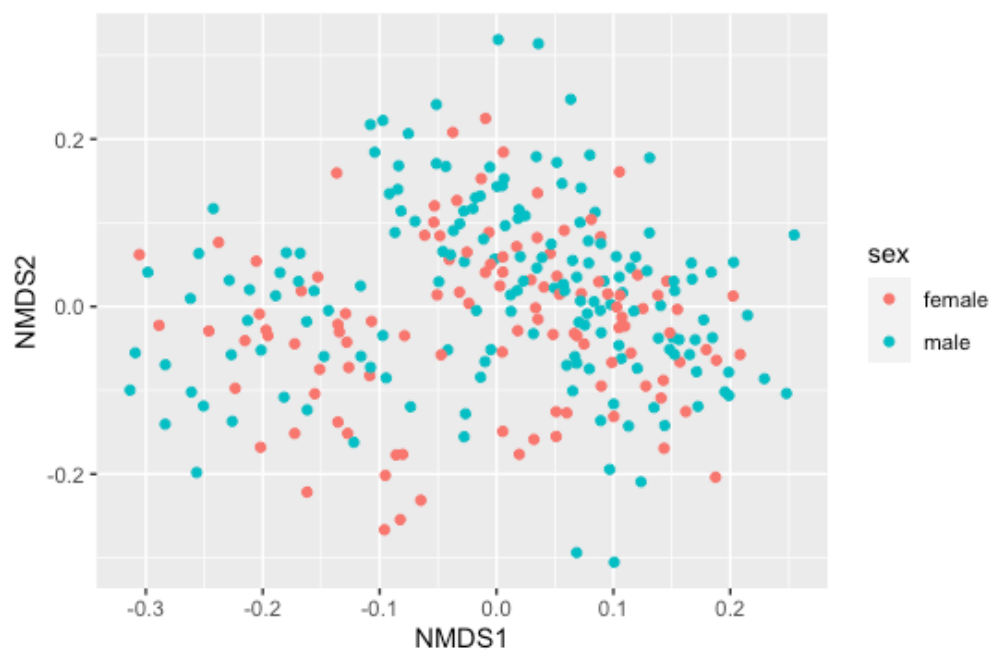


Figure 20. NMDS plot based on rarified data comparing samples based on sex.

The two plots clearly show the effect rarefaction has had. From this point forward we will only work with the rarefied data.

2.8.5 Identifying differences in community composition

In order to create a dataset with stronger patterns we will re-subset the original HMPv35 dataset based on two body sites (Saliva and Stool). Afterwards we will rarefy to 1000 sequences as before.

```
HMPsvS <-subset_samples(HMPv35, HMPbodysubsite == "Saliva" |  
HMPbodysubsite == "Stool")  
  
HMPsvS_rare <-rarefy_even_depth(HMPsvS, 1000,rngseed=TRUE)  
  
HMPsvS_rare
```

```
phyloseq-class experiment-level object  
otu_table() OTU Table:      [ 23587 taxa and 578 samples ]  
sample_data() Sample Data:  [ 578 samples by 10 sample variables ]  
tax_table()  Taxonomy Table: [ 23587 taxa by 6 taxonomic ranks ]  
phy_tree()   Phylogenetic Tree: [ 23587 tips and 23424 internal nodes ]  
refseq()     DNASTringSet:   [ 23587 reference sequences ]
```

If you run a t-test on the alpha diversity measures it confirms that a significant change is present. To see if these changes reflect a change in community composition we can use an NMDS plot.

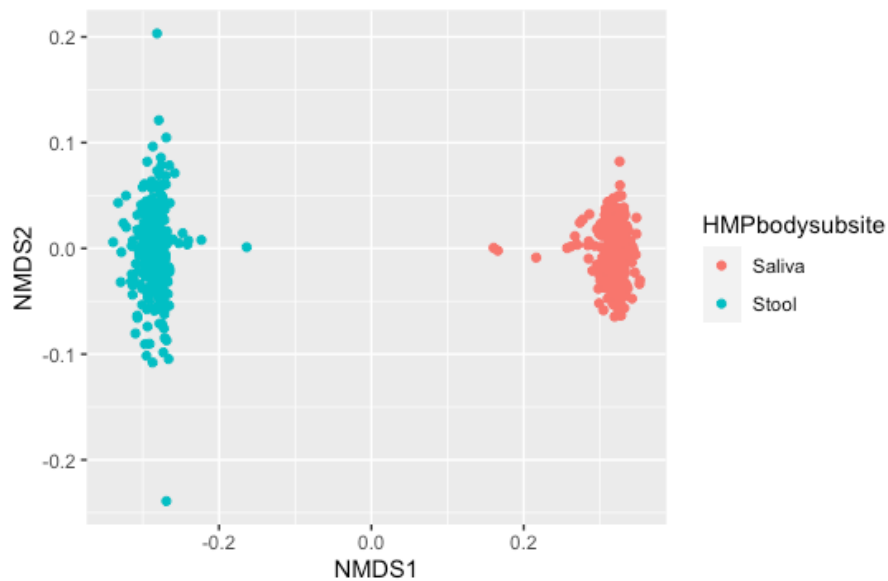


Figure 21. NMDS ordination of HMP saliva and stool samples

The NMDS plot clearly shows grouping based on sites. To confirm that these observed groupings are real we can use an ANOSIM test.

2.8.6 ANOSIM: The ANALysis Of SIMilarity (ANOSIM) test¹

The ANOSIM test is similar to an Analysis of variance (ANOVA) test used to analyze the differences among group means. However, in an ANOSIM we use a dissimilarity matrix rather than raw data. Using a matrix helps compare different types of non-parametric data. To perform the test use:

```
#Select variable to test for grouping
HMPbodysubsite_group <- get_variable(HMPSvS_rare, "HMPbodysubsite")

#Perform ANOSIM
HMPbodysubsite_ano <- anosim(distance(HMPSvS_rare, "bray"),
HMPbodysubsite_group)
```

¹ <https://sites.google.com/site/mb3gustame/hypothesis-tests/anosisim>

```
#Is the observed grouping statistically significant?
```

```
HMPbodysubsite_ano$signif
```

```
##[1] 0.001
```

```
#How strong is the grouping?
```

```
HMPbodysubsite_ano$statistic
```

```
##[1] 0.9991916
```

ANOSIM confirms a strong and statistically significant grouping based on body site, confirming that these two communities are different. It is always good to double check these observations. A secondary test that complements ANOSIM is the NPMANOVA.

2.8.7 NPMANOVA (ADONIS): non-parametric multivariate analysis of variance²

The NPMANOVA, performed in the *vegan* package using the *adonis()* function, is a test for analysis of variance using distance matrices. To perform it:

```
#Create a data frame using your sample_data
```

```
df <-as(sample_data(HMPSvS_rare), "data.frame")
```

```
#Calculate your Bray distance matrix
```

```
d <-distance(HMPSvS_rare, "bray")
```

```
#Perform your ADONIS test
```

```
bodysiteadonis <-adonis(d ~ HMPbodysubsite, df)
```

```
#Check results
```

```
bodysiteadonis
```

²<https://sites.google.com/site/mb3gustame/hypothesis-tests/manova/npmanova>

```

Call:
adonis(formula = d ~ HMPbodysubsite, data = df)

Permutation: free
Number of permutations: 999

Terms added sequentially (first to last)

      Df SumsOfSqs MeanSqs F.Model    R2 Pr(>F)
HMPbodysubsite  1    38.37  38.370  104.45 0.1535 0.001 ***
Residuals     576   211.60   0.367      0.8465
Total         577   249.97              1.0000
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

```

The results suggest that while there is a definite change in community composition that is statistically significant, the strength of the change is not very big. Perhaps only a small portion of the community is changing with many overlapping OTUs, or there is a lot of variance across samples within a bodysubsite.

2.8.8 Working with continuous data

So far we have shown you examples of processing datasets using categorical data (variables that have discrete groups like body site). However, many experiments also collect continuous data (data that has a gradient or can be measured on an infinite scale). To explore how to use the already covered techniques using this type of data we will import a new HMP derived dataset that focuses solely on vaginal samples. The dataset has already been rarefied and is ready for analysis.

Note: Because the actual pH values of the HMP samples are not included among publicly-available metadata, biologically-realistic values based on Nugent score have been simulated for these samples for learning purposes.

```

#import new dataset

HMP_simPH<-readRDS("~/Micro336-master/data/HMP-simPH.rds")

HMP_simPH

```

```

phyloseq-class experiment-level object
otu_table() OTU Table:      [ 5312 taxa and 388 samples ]
sample_data() Sample Data:  [ 388 samples by 11 sample variables ]
tax_table()  Taxonomy Table: [ 5312 taxa by 6 taxonomic ranks ]

```

2.8.8.1 Compare alpha diversity using continuous data

You can use the same `estimate_richness` commands to generate information on alpha diversity. However, plotting and statistics used to test patterns will change. Here we will focus on a pH gradient in vaginal samples using the variable "ph2". We will then use a Spearman's correlation to test how well pH corresponds to changes in alpha diversity.

```
#Plot observed richness and Shannon diversity for all pH
p <-plot_richness(HMP_simPH, x = "ph2", measures=c("Observed",
"Shannon"))

library(DEGreport)

#Plot results and fit a line using a linear model [lm], and provide
the statistical result for a Spearman correlation
p +
  geom_point() +
  geom_smooth(method=lm) +
  geom_cor(method = "spearman")
```

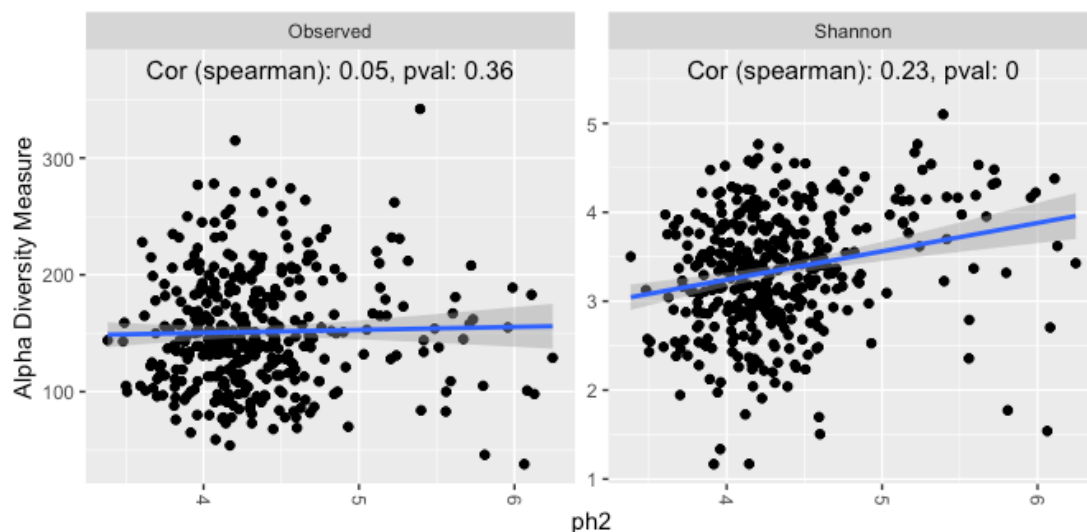


Figure 22. Changes in alpha diversity (Observed and Shannon) along a pH gradient for vaginal samples with results for Spearman correlation.

We can see a weak but positive correlation for Shannon, but no statistically significant trend for observed. We can confirm this and get the specific results using a direct test.

```
#Non-parametric test: Spearman correlation test

ph_Shannon<-cor.test(alpha_meta_HMP_simPH$Shannon,
alpha_meta_HMP_simPH$ph2, method="spearman",exact = FALSE)

ph_Shannon

ph_Observed<-cor.test(alpha_meta_HMP_simPH$Observed,
alpha_meta_HMP_simPH$ph2, method="spearman",exact = FALSE)

ph_Observed
```

2.8.8.1 Compare beta diversity using continuous data

We will use the same code we did before to create an NMDS plot and color by pH.

```
NMDS.ph <- ordinate(HMP_simPH, "NMDS", "bray")
NMDS.ph

#stressplot
stressplot(NMDS.ph)

sampleplot<-plot_ordination(HMP_simPH, NMDS.ph, type = "samples",
color = "ph2")

sampleplot
```

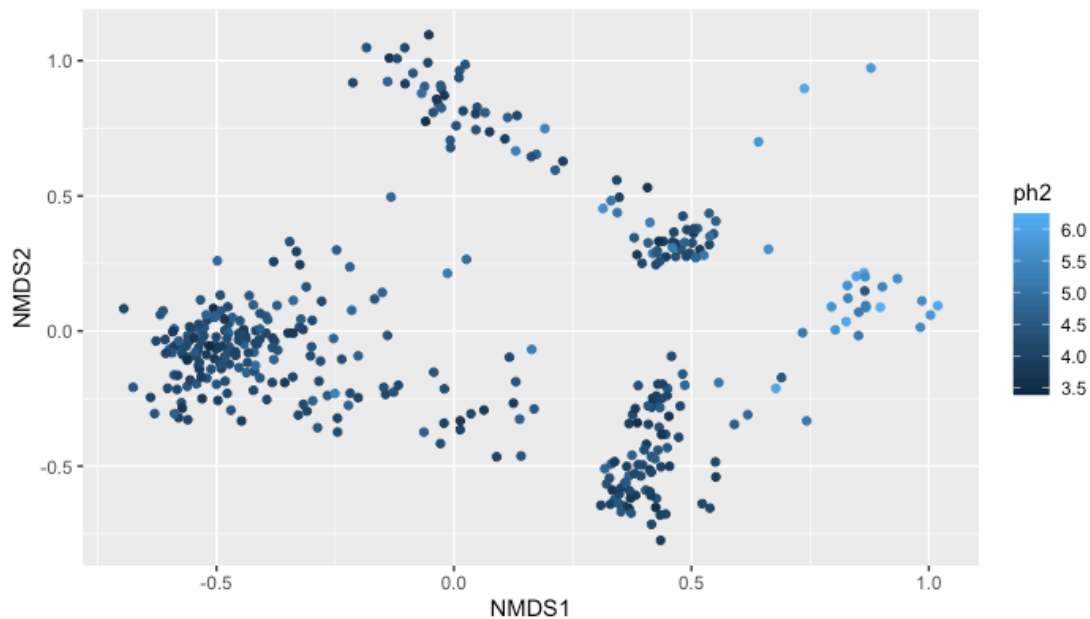


Figure 23. NMDS plot based vaginal samples showing relationship to pH.

You can see a slight trend along NMDS1 where pH value increase along the axis. We can test this by extracting the values for NMDS1 and plotting against pH directly.

```
#Extract NMDS1 values from ordination and create a new dataframe
coord<-as.data.frame(scores(NMDS.ph))

#Attach SampleID's so we can link to metadata
coord$X.SampleID <- rownames(coord)

#Attach metadata
coord_meta<-merge(coord, alpha_meta_HMP_simPH, by = "X.SampleID")

library(gthor)

#use new data to make a plot and add some statistics
ggplotRegression(lm(NMDS1~ ph2, data=coord_meta))
```

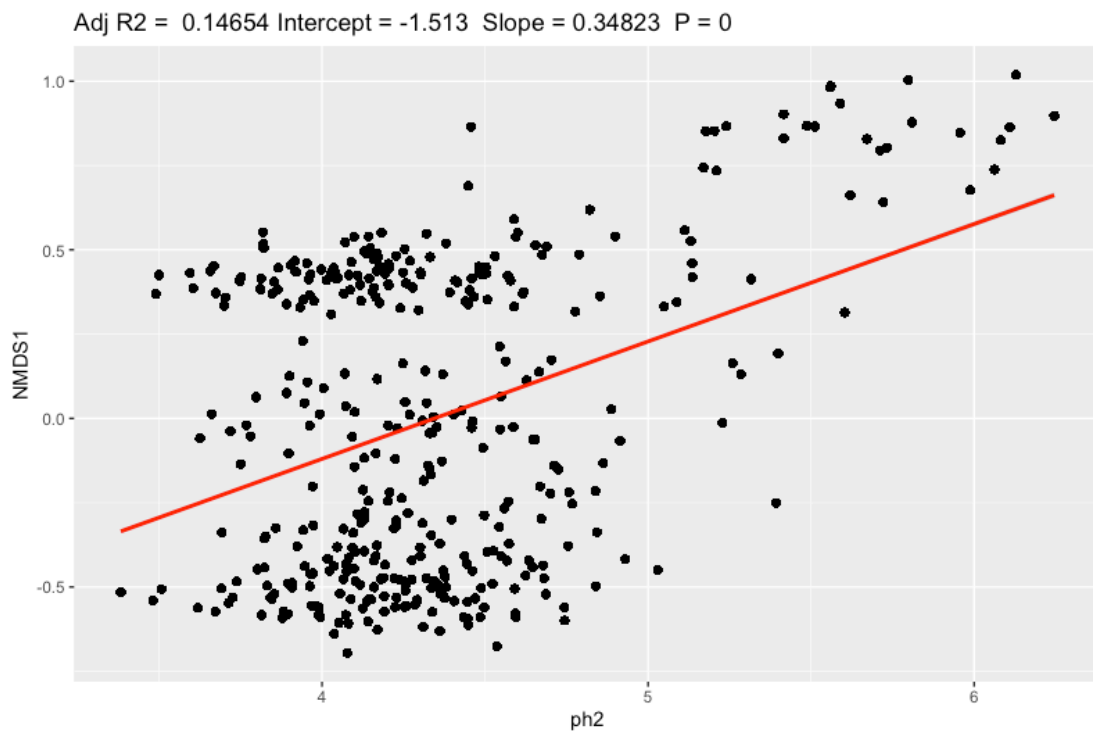


Figure 24. Correlation between NMDS1 (from Figure23) and pH based on a linear model.

We can see that a significant positive correlation exists between NMDS1 and pH. We can confirm this directly by regressing pH against NMDS coordinates using a linear model.

```
#confirm exact values directly using a linear regression
nmbs1_ph_lm<-lm(NMDS1~ pH2, data=coord_meta)

summary(nmbs1_ph_lm)
```

You could also use a different correlation (e.g. Spearmans) to perform a similar test.

```
#confirm linear model with a Spearmans correlation
cor.test(coord_meta$pH2, coord_meta$NMDS1, method="spearman")
```

```

Spearman's rank correlation rho

data: coord_meta$ph2 and coord_meta$NMDS1
S = 4.4388e+14, p-value < 2.2e-16
alternative hypothesis: true rho is not equal to 0
sample estimates:
rho
0.2193986

```

2.8.8.2 Other methods for findings correlation between Beta diversity and continuous data

A useful tool for testing correlations between microbiome beta diversity and continuous metadata is the Mantel test. This is a test of the [correlation](#) between two [matrices](#). It means we can compare the "distance" or dissimilarity matrices created from separate datasets. This creates a unitless comparison that is useful when the scale of change may be very different between the datasets.

```

#First let's combine our data at the Genus level to make the process
run faster. This is not required but will make the analysis run
faster for this example.

HMP_simPH_glom<-tax_glom(HMP_simPH, taxrank="Genus")

#this is taking the site x species matrix (out_table) and determining
distances between each set of sites sampled

HMP_simPH_bray <- phyloseq::distance(HMP_simPH_glom, "bray")

#ph matrix

#this makes a matrix with differences in pH between each sample
df_ph <- as(sample_data(HMP_simPH_glom), "data.frame")
ph.dist <- dist(df_ph$ph2, method = "euclidean")
dist.matrix <- as.matrix(ph.dist)

#We can now run the Mantel test

mantel(HMP_simPH_bray, ph.dist, method = "spearman")

```

```

Mantel statistic based on Spearman's rank correlation rho

Call:
mantel(xdis = HMP_simPH_bray, ydis = ph.dist, method = "spearman")

Mantel statistic r: 0.452
Significance: 0.001

Upper quantiles of permutations (null model):
  90%   95%  97.5%   99%
0.0405 0.0542 0.0665 0.0833
Permutation: free
Number of permutations: 999

```

We can see there is a significant, positive correlation between community composition and pH in vaginal samples. We can reuse the same ADONIS test for comparing variance.

```

#test using ADONIS

df<-as(sample_data(HMP_simPH_glom), "data.frame")
d <-phyloseq::distance(HMP_simPH_glom, "bray")
pHadonis <-adonis(d ~ ph2, df)
pHadonis

```

```

Call:
adonis(formula = d ~ ph2, data = df)

Permutation: free
Number of permutations: 999

Terms added sequentially (first to last)

      Df SumsOfSqs MeanSqs F.Model    R2 Pr(>F)
ph2      1    12.616  12.6164  250.97 0.39401 0.001 ***
Residuals 386    19.404   0.0503      0.60599
Total    387    32.021              1.00000
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

```

2.8.9 Week 3 Exploring community composition

2.8.9.1 Stacked bar plots (relative abundance)

Now that we know there are changes in community structure, let's see how we can explore changes in specific portions of the community. A common way to explore community composition is through stacked bar plots. These provide quick access to the data, but have limited value for comparison or quantitative analysis and we will explore better ways to present results later on.

Stacked bar plots are only useful if we can tell the different colours apart and link them to their assigned variables. To reduce the number of taxa plotted we will remove phyla represented less than 1% (relative abundance) in each sample. Let's start by comparing Saliva samples by sex.

```
library(plyr)
library(dplyr)
library(scales)

# This command is an example of a 'pipe', or a sequence of commands
# combined (by the use of this pipe symbol '%>%' into a single command
# that automatically calls all of them. I will explain each portion.

H35_sal_rare_over1 <- H35_sal_rare %>% ##use this file for analysis
  tax_glom(taxrank = "Phylum") %>% ##collapse all data at the phylum
  level
  transform_sample_counts(function(x) {x/sum(x)} ) %>% ##convert
  counts to relative abundance
  psmelt() %>%      ##melt (convert to a format [data.frame]
  compatible with ggplots) to long format
  filter(Abundance > 0.01) %>% ##prune out phyla below 1% in each
  sample
  arrange(Phylum) ##here we sort phylum data based on abundance
```

```
#Now you have your data ready for plotting in ggplots

d<- ggplot(H35_sal_rare_over1,aes(x=sex,y=Abundance,fill=Phylum)) +
  geom_bar(stat="identity",position="fill") +
  scale_y_continuous() +
  ylab("Relative Abundance")

d
```

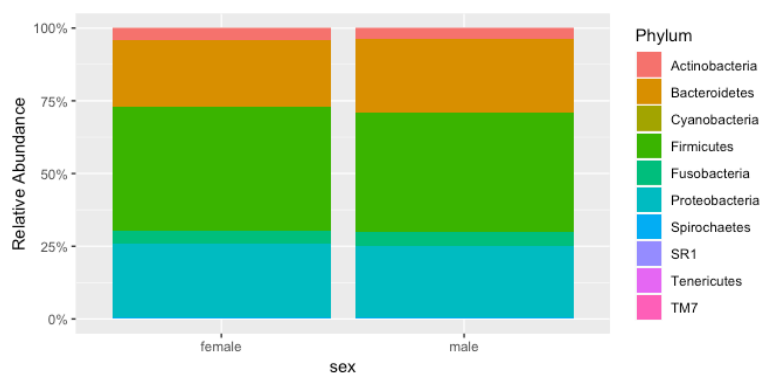


Figure 25. Stacked barplot showing mean relative abundance of phyla

The results don't change much, but the data is layered with additional information that may have an effect. Let's see how adding a time component affects results by looking at 'visitno'.

2.8.9.2 Stacked bar plots (facetting)

To do this we will replot the graph and split the plots based on 'sex' and replace the variables on the x-axis with 'visitno'. Splitting the plot into multiple plots is accomplished by 'facetting' (either `facet_grid` or `facet_wrap`):

```
ggplot(H35_sal_rare_over1,aes(x=visitno,y=Abundance,fill=Phylum)) +
  facet_grid(sex~.) +
  geom_bar(stat="identity",position="fill") +
  scale_y_continuous(labels = percent_format())
```

You can re-use the same commands as before to complete the plot.

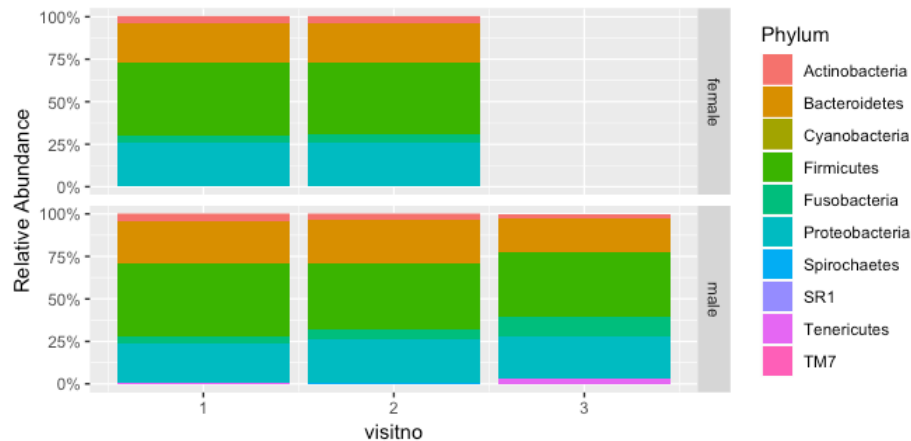


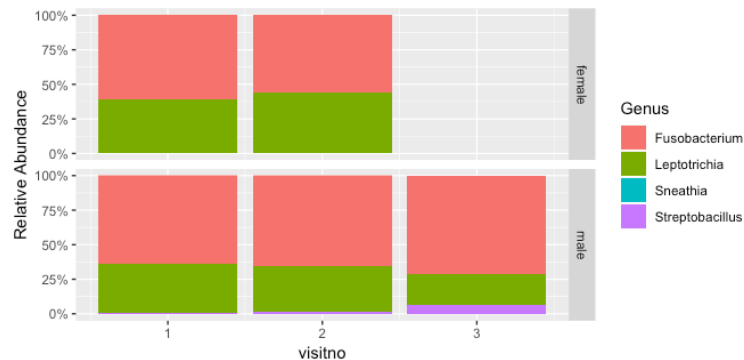
Figure 26. Stacked barplot showing mean relative abundance of phyla, faceted by sex

You can see that within male patients there is an increase in Fusobacteria and Tenericutes. Let's focus on the Fusobacteria and see who is changing. For this we first need to subset our data to only this group then repeat our plotting as before (see **Stacked bar plots (facetting)**) replacing Phylum for Genus in the code.

```
#Subset by Fusobacteria
HPR.fuso <- subset_taxa(H35_sal_rare , Phylum == "Fusobacteria")

# prune out genera with no hits in each sample and re-use prior code
HPR.fuso.RA <- HPR.fuso %>%
  tax_glom(taxrank = "Genus") %>%
  transform_sample_counts(function(x) {x/sum(x)} ) %>%
  psmelt() %>%
  filter(Abundance > 0.00) %>%
  arrange(Genus)
```

You can see that changes are driven by *Streptobacillus* (**Figure 27**). However, this is relative abundance so we do not get a sense of how abundant these are. For this



we need absolute abundance (sequences per sample) to get a better idea of what is happening. Let's use the original subset data without converting to relative abundance.

Figure 27. Faceted barplot, shown at genus level

In this example changes are subtle, so let's look at data with stronger patterns to learn how to identify the most important community drivers. Let's go back to the rarefied dataset comparing stool vs saliva samples.

2.8.10 Differential pattern analysis

The results suggest that some microbial groups are changing between Stool and Saliva. Let's explore taxa are changing by using a Kruskal-Wallis test. First we need to create a melted data frame from our phyloseq object which includes relative abundance & taxa glommed at Class level. We glom at Class level to allow us to better see the plots and speed up the analysis for the sake of speed. Under normal circumstances we may wish to do the test at lower levels (genus, species, OUT or ASV) so we get precise results for each organisms. We will first look at the whole dataset before identifying the key groups changing

```
HMPsvS_rare_df<-HMPsvS_rare %>%
  tax_glom(taxrank = "Class") %>%
  transform_sample_counts(function(x) {x/sum(x)} ) %>%
  psmelt()
```

```
d<-
ggplot(HMPSvS_rare_df,aes(x=HMPbodysubsite,y=Abundance,fill=Phylum))+
  geom_bar(stat="identity",position="fill") +
  scale_y_continuous()+
  ylab("Relative Abundance")
```

d

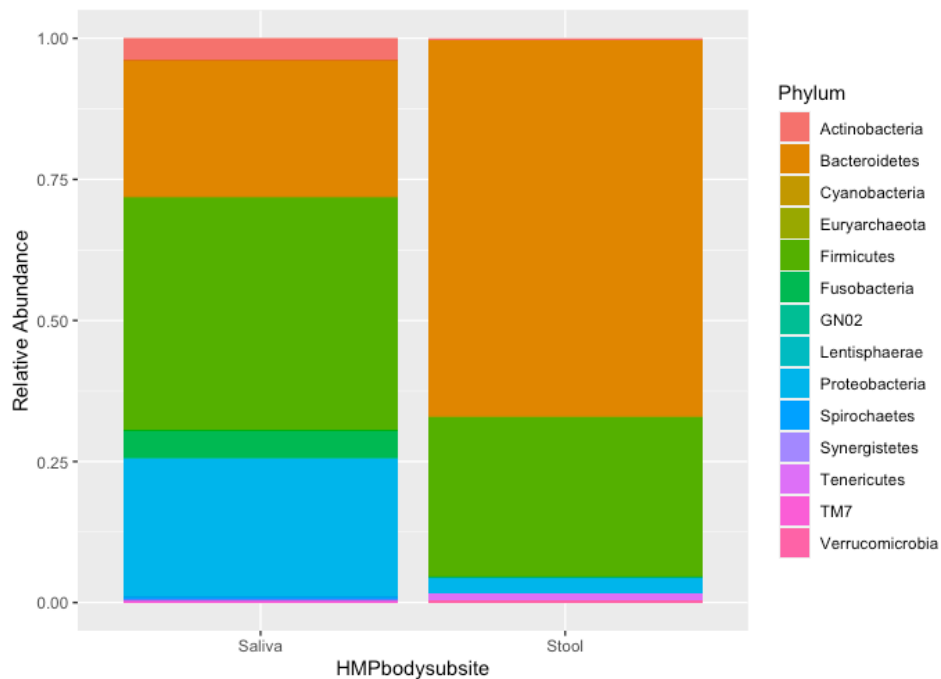


Figure 28. Stacked barplot showing mean relative abundance of phyla after glomming data at Class level.

The taxonomic profiles look very different, but we don't know which changes are statistically significant and what are the specific organisms changing. Let's identify which groups are actually statistically different using the rstatix package.

```
library(rstatix)
# do KW test
KW_taxa<-HMPSvS_rare_df%>%
  group_by(Class) %>%
  kruskal_test(Abundance ~ HMPbodysubsite) %>%
  adjust_pvalue(method="fdr") %>%
```

```

filter(p.adj < 0.05)

# keep only taxa with significant results
only_sig_taxa<-HMPsvS_rare_df %>%
  filter(Class %in% KW_taxa$Class)

# plot
ggplot(only_sig_taxa, aes(x=HMPbodysubsite, y=Abundance,
fill=Phylum)) +

  geom_boxplot() +

  facet_wrap(Phylum~Class,scales="free_y")+

  stat_compare_means(method = "kruskal.test", paired = FALSE)

```

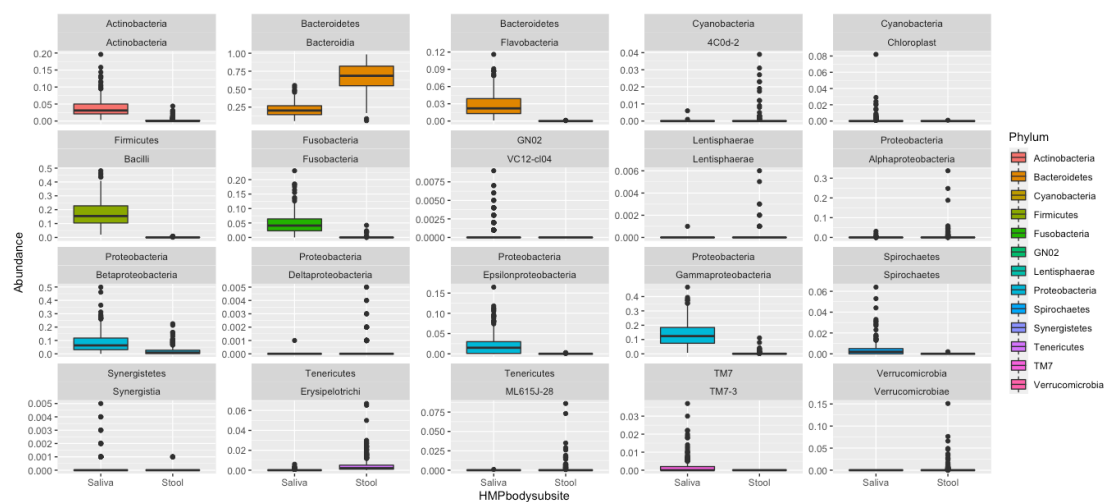


Figure 29. Differentially abundant taxa between stool and saliva by Class and Phylum. Taxa identified using the kruskal_test.

We can now see which Class are different, and what Phylum they belong to.

We can also use other methods to perform this sort of test. Below we will cover the edgeR package as an alternative.

2.8.11 Differential pattern analysis using edgeR package

Load libraries needed

```
library("edgeR")
library(reshape2)
library(RColorBrewer)
library(grid)
library(empiricalFDR.DESeq2)
library("DESeq2")
library(PathoStat)
```

We will be using the edgeR package. EdgeR is used for differential expression analysis of RNA-seq expression profiles with biological replication. It implements a range of statistical methodology based on the negative binomial distributions, including empirical Bayes estimation, exact tests, generalized linear models and quasi-likelihood tests. While designed for RNA seq data, the statistical tests within the package can be used for microbiome data analysis as well. We will exploit these stats capability by modifying our data file to match the inputs requirements of edgeR, using the phyloseq_to_edgeR function within the PathoStat library.

NOTE: In class we will be using a rarefied dataset to reduce the time and computing power needed to show you the process, but in general for methods like edgeR and DeSeq2 large datasets with deep sequencing are best.

```
#convert the phyloseq data object into an edgeR "DGE" data object,  
called dge.  
  
dge <- phyloseq_to_edgeR(HMPSvS_rare, group = "HMPbodysubsite")
```

Now that our data is correctly formatted we can carry out our statistical analysis. Here we will use a test similar to a t-test called an 'exact test'. Like a t-test, the analysis determines the statistical significance in observed differences in means between two groups. Other tests are available, but we will not use them here. To perform the exact test:

```
et <- exactTest(dge)
```

To help us with downstream processing let's do a few changes. First we need to perform a false discovery rate (FDR) correction. In datasets like ours where the number of variables (OTUs) is large, the chance of 'false positives' is high. We account for this by performing p value adjustments using various methods. In this example we will use the Benjamini-Hochberg procedure (BH) and add adjusted values to our results (et). Then we will create a new table (tt) by using the command 'topTags'. This command extracts the top differentially expressed tags in a data frame for a given pair of groups, ranked by p-value or absolute log-fold change. Afterwards we process the results further to focus on only the most significant portions.

```
# Perform FDR, extract values from test results, create new table,
and sort by p-value

tt <- topTags(et, n = nrow(dge$table), adjust.method = "BH", sort.by =
"PValue")

res <- tt@.Data[[1]]

alpha <- 0.05

sigtab <- res[(res$FDR <= alpha), ]

sigtab <- cbind(as(sigtab, "data.frame"),
as(tax_table(HMPSvS_rare)[rownames(sigtab), ], "matrix"))

#Subset by most significant only (p value <0.05 and at least a 2-fold
change in abundance)

sigtab_2fold <- subset(sigtab, FDR < 0.05 & logFC >= 2 | logFC <= -2
& FDR < 0.05)
```

2.8.11.1 Plot exactTest results

Let's plot significantly different Phyla between Stool and Saliva:

```

#Remove any rows with no Phylum assignment
sigtabgen <-subset(sigtab_2fold, !is.na(Phylum))

#Order by Phylum
x <-tapply(sigtabgen$logFC, sigtabgen$Phylum, function(x) max(x))
x <-sort(x, TRUE)

#Turn Phylum into a factor
sigtabgen$Phylum <-factor(as.character(sigtabgen$Phylum), levels =
names(x))

#Plot
ggplot(sigtabgen, aes(x = Phylum, y = logFC)) +
geom_point(size = 6) +
theme(axis.text.x = element_text(angle = -90, hjust = 0, vjust =
0.5)) +
scale_fill_brewer() +
coord_flip()

```

The plot shows that while OTUs in some Phyla (Fusobacteria) are associated with only one site, most Phyla have examples of OTUs present in both sites. This could mean a site preference at lower taxonomic levels.

Let's explore this by plotting significantly different (p less than 0.05, more than 2-fold change) Genera between Saliva and Stool.

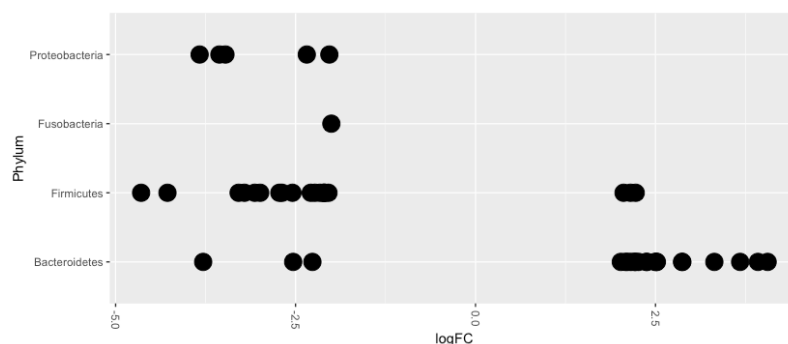


Figure 30. EdgeR results - taxa differentially abundant between body sites

```
#subset original phyloseq object to include only significant taxa
sigtabgen <-subset(sigtab_2fold, !is.na(Genus))
# Genera order
x <-tapply(sigtabgen$logFC, sigtabgen$Genus, function(x) max(x))
x <-sort(x, TRUE)

sigtabgen$Genus <-factor(as.character(sigtabgen$Genus), levels =
names(x))

#make plot
ggplot(sigtabgen, aes(x = Genus, y = logFC, color = Phylum)) +
  geom_point(size = 6) +
  theme(axis.text.x = element_text(angle = -90, hjust = 0, vjust =
0.5)) +
  scale_fill_brewer() +
  coord_flip()
```

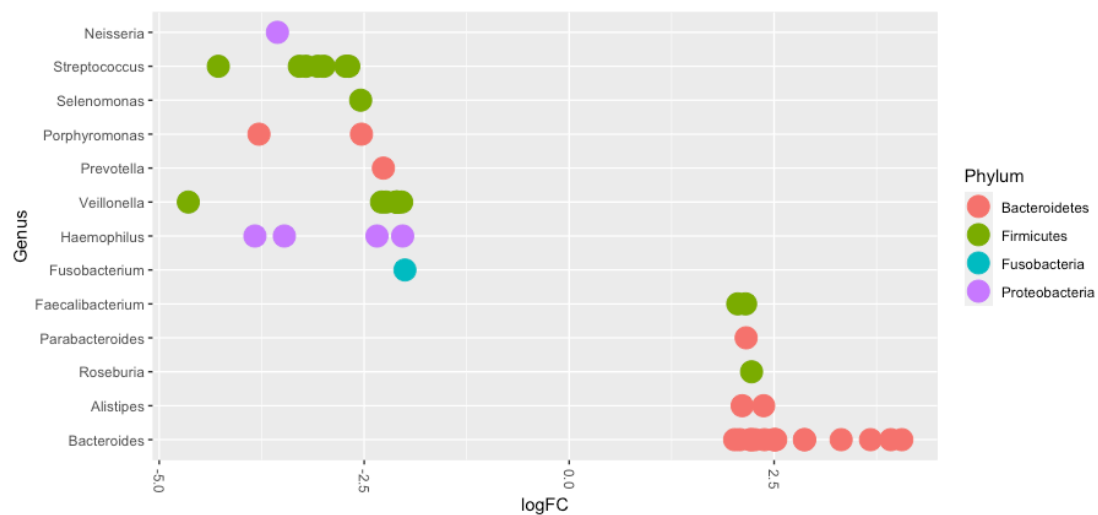


Figure 31. EdgeR results: genera differentially abundant between body site, coloured by phylum

As you can see organisms belonging to the genus *Haemophilus* are more than 2-fold more common in Saliva, than stool, while the opposite is true for

Faecalibacterium. To allow quantitative (visual comparison) lets subset the full data by significantly different OTUs only and plot the absolute values.

2.8.12 R coded subsetting of important OTUs

It is always easier, and more reproducible, to hard code data manipulations. This avoids copy/pasting errors between users. To do this use:

```
#Rename OTU ID and add to tax_table if you have not done this before
tax_table(HMPSvS_rare) <- cbind(tax_table(HMPSvS_rare),
OTU=taxa_names(HMPSvS_rare))

#Make exact test results with OTU names into data frame
sigtabgen_OTUs <- data.frame(OTU = rownames(sigtabgen), sigtabgen)

#Make a new file with only the names of the OTUs you wish to keep.
keepTaxa_ET <- sigtabgen_OTUs$OTU

#Check that your names are OK.
print(keepTaxa_ET)

#Use the subset_taxa function and your new taxa list to create a
phyloseq file with only your OTUs of interest
ET_2fold <- subset_taxa(HMPSvS_rare, OTU %in% keepTaxa_ET)
```

Once you have generated your subset data, you will need to melt it for plotting using ggplots.

```
#Melt phyloseq file into a data frame
dat_onlysigOTU <- psmelt(ET_2fold)

#Order data based on Phylum abundance
```

```
dat_onlysigOTU <- dat_onlysigOTU[order(dat_onlysigOTU$Phylum),]
```

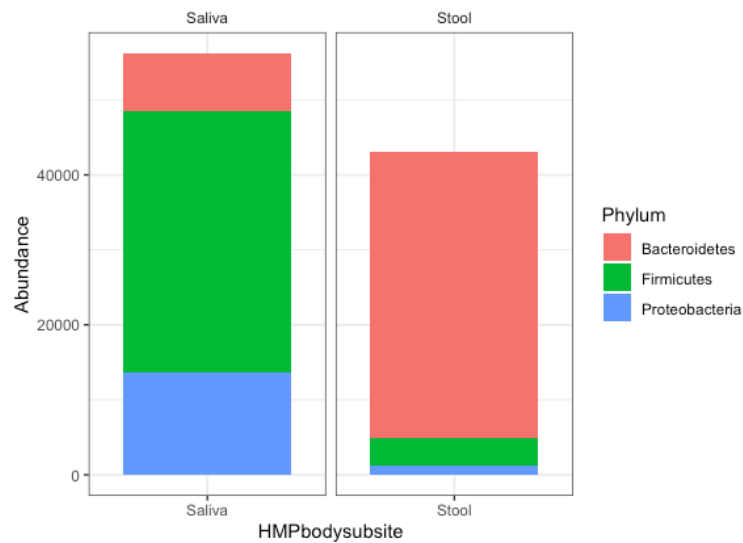


Figure 32. Differentially abundant taxa between stool and saliva by Phylum. Taxa identified using the exactTest.

While the above plot allows you to see major differences, it is difficult to interpret because it's compositional and combines multiple samples together, as well as removing the ability to compare the same group between subsites. A clearer way to present the data would be to use boxplots by Genera and order by Phylum.

```
#show differences by Genera
ggplot(dat_onlysigOTU, aes(x=HMPbodysite, y=Abundance, fill=Phylum))+
  geom_boxplot() +
  facet_wrap(~Phylum+Genus) +
  scale_y_log10()
```

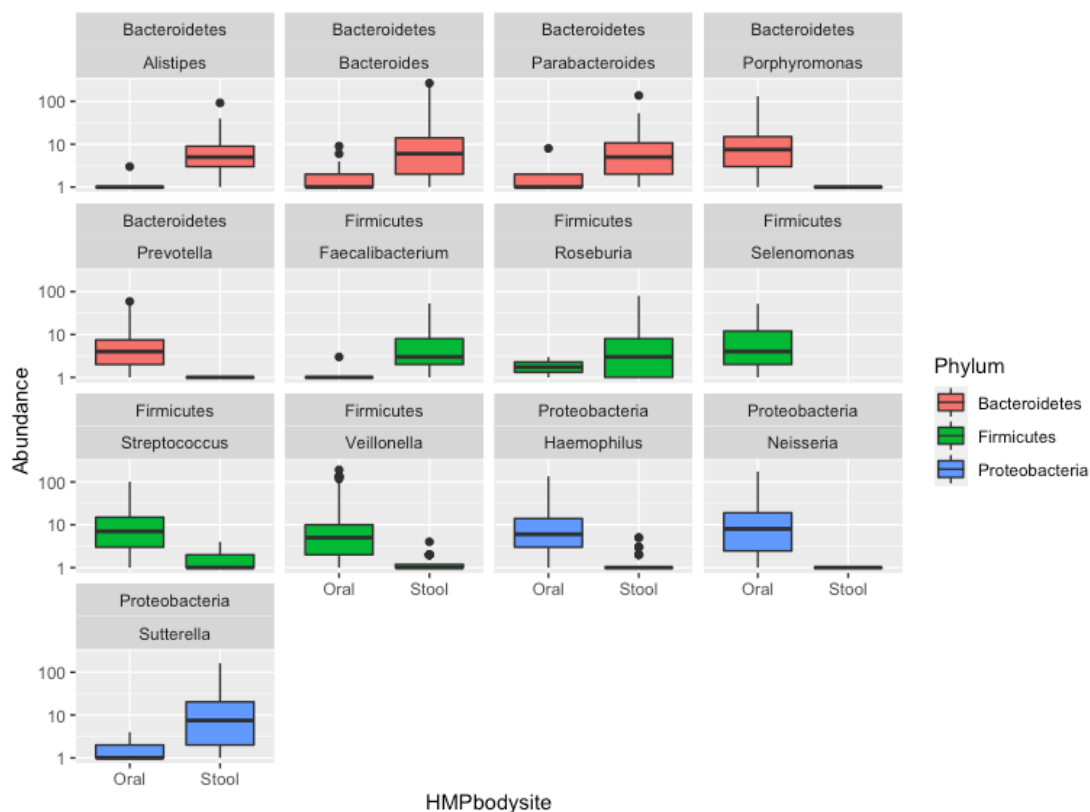


Figure 33. Differentially abundant taxa between stool and saliva by Genus and Phylum. Taxa identified using the `kruskal_test`.

2.8.13 Plotting taxonomic changes along continuous data

To show you how to analyse taxonomic changes along continuous data we will again use our `HMP_simPH` phyloseq object composed of vaginal samples. While boxplots and barcharts are normally used for categorical data these are not suitable for continuous data. Instead we use line charts to show trends along the gradient, or scatter plots to show how the data is changing.

Here are two examples.

```
# Lets make the analysis run faster by glomming at Phylum level,
melting our object and filtering any low abundance phyla.
```

```
HMP_simPH_over1 <- HMP_simPH %>%
  tax_glom(taxrank = "Phylum") %>%
  transform_sample_counts(function(x) {x/sum(x)} ) %>%
  psmelt() %>%
```

```

filter(Abundance > 0.01) %>%
  arrange(Phylum)

#We can now plot the data
ggplot(HMP_simPH_over1, aes(x=ph2, y=Abundance, color=Phylum)) +
  geom_point() +
  scale_y_continuous() +
  facet_wrap(Phylum~., scales = "free_y") +
  geom_smooth(method=lm) +
  ylab("Relative Abundance") +
  stat_cor(method = "spearman")

```

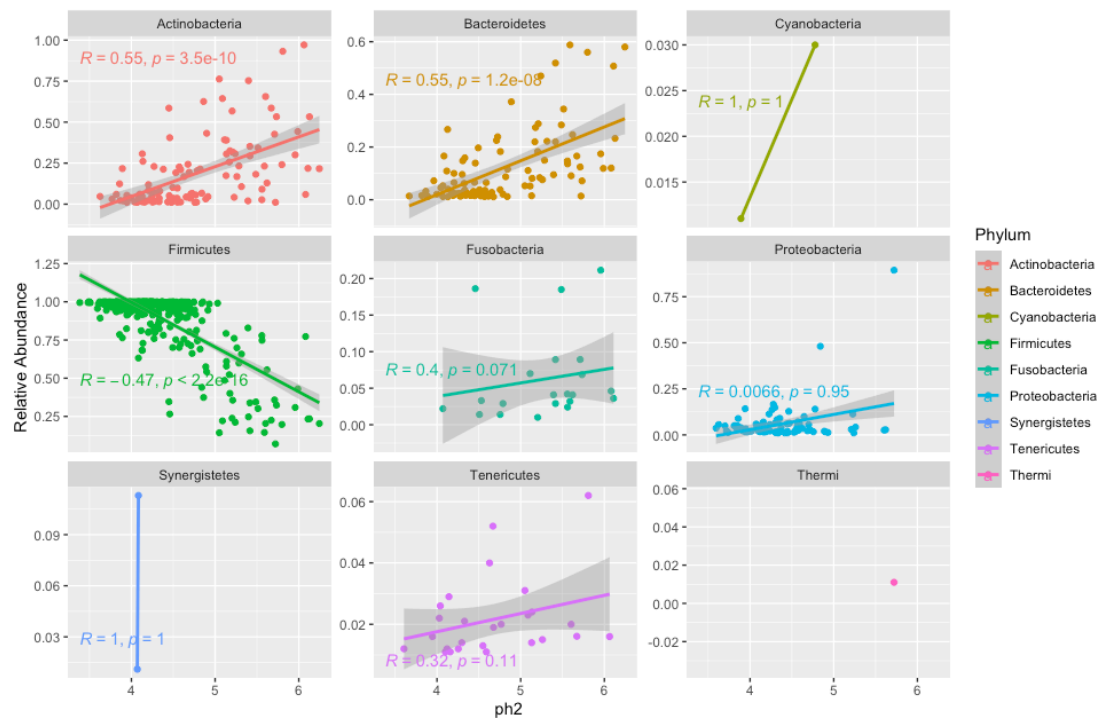


Figure 34. Taxonomic (Phylum level) changes along a pH gradient for vaginal samples with results for Spearman correlation.

To retrieve the statistics for the linear fit we can use the same code as before

```

library(lme4)
Phyla_ph_lm<-lmList(Abundance ~ ph2 | Phylum, HMP_simPH_over1)

```

```
Phyla_ph_lm
```

We can do the same for the Spearmans correlation using this:

```
#And we can get the Spearmans results like this
```

```
ddply(HMP_simPH_over1, .(Phylum), summarise, "corr" = cor(ph2,
Abundance, method = "spearman"))
```

| | Phylum | corr |
|---|----------------|--------------|
| 1 | Actinobacteria | 0.547306826 |
| 2 | Bacteroidetes | 0.546548894 |
| 3 | Cyanobacteria | 1.000000000 |
| 4 | Firmicutes | -0.467013247 |
| 5 | Fusobacteria | 0.402340133 |
| 6 | Proteobacteria | 0.006582197 |
| 7 | Synergistetes | 1.000000000 |
| 8 | Tenericutes | 0.318890787 |
| 9 | Thermi | NA |

Notice that the p values are not shown. To calculate and show those we can use another command, but the test is sensitive to groups (taxa) with too few data points so we first need to remove those.

```
#We can get the full info here, but need to remove taxa with too few
data points first
```

```
df<-HMP_simPH_over1 %>%
```

```
filter(Phylum %in% c("Actinobacteria", "Proteobacteria",
"Tenericutes", "Firmicutes")) %>%
```

```
group_by(Phylum)
```

```
#use the new dataframe to do your Spearmans test by each group/taxa
```

```
Taxa_spearman<-df %>%
```

```
nest(data = -Phylum) %>%
```

```
mutate(corr=map(data,~cor.test(.x$ph2, .x$Abundance, method =
"sp")))) %>%
```

```
mutate(tidied = map(corr, tidy)) %>%
```

```
unnest(tidied) %>%
```

```
select(-data, -corr)
```

Taxa_spearman

| | Phylum
<chr> | estimate
<dbl> | statistic
<dbl> | p.value
<dbl> | method
<chr> | alternative
<chr> |
|---|-----------------|-------------------|--------------------|------------------|---------------------------------|----------------------|
| 1 | Actinobacteria | 0.547 | <u>108856.</u> | 3.55e-10 | Spearman's rank correlation rho | two.sided |
| 2 | Firmicutes | -0.467 | <u>14061828.</u> | 2.64e-22 | Spearman's rank correlation rho | two.sided |
| 3 | Proteobacteria | 0.00658 | <u>116707.</u> | 9.51e-1 | Spearman's rank correlation rho | two.sided |
| 4 | Tenericutes | 0.319 | <u>1992.</u> | 1.12e-1 | Spearman's rank correlation rho | two.sided |

And we can reuse a modified code from before to only plot the significant Phyla.

```
# do tests

Sp_pH_taxa<-ddply(HMP_simPH_over1, .(Phylum), summarise, "corr" =
cor(ph2, Abundance, method = "spearman"))%>%

  filter(corr > 0.5)

# make a list of the significant genera

only_sig_taxa<-HMP_simPH_over1 %>%

  filter(Phylum %in% Sp_pH_taxa$Phylum)

# plot

ggplot(only_sig_taxa,aes(x=ph2,y=Abundance,color=Phylum))+

  geom_point() +

  scale_y_continuous()+

  facet_wrap(Phylum~.,scales = "free_y")+

  geom_smooth(method=lm)+

  ylab("Relative Abundance")+

  stat_cor(method = "spearman")
```

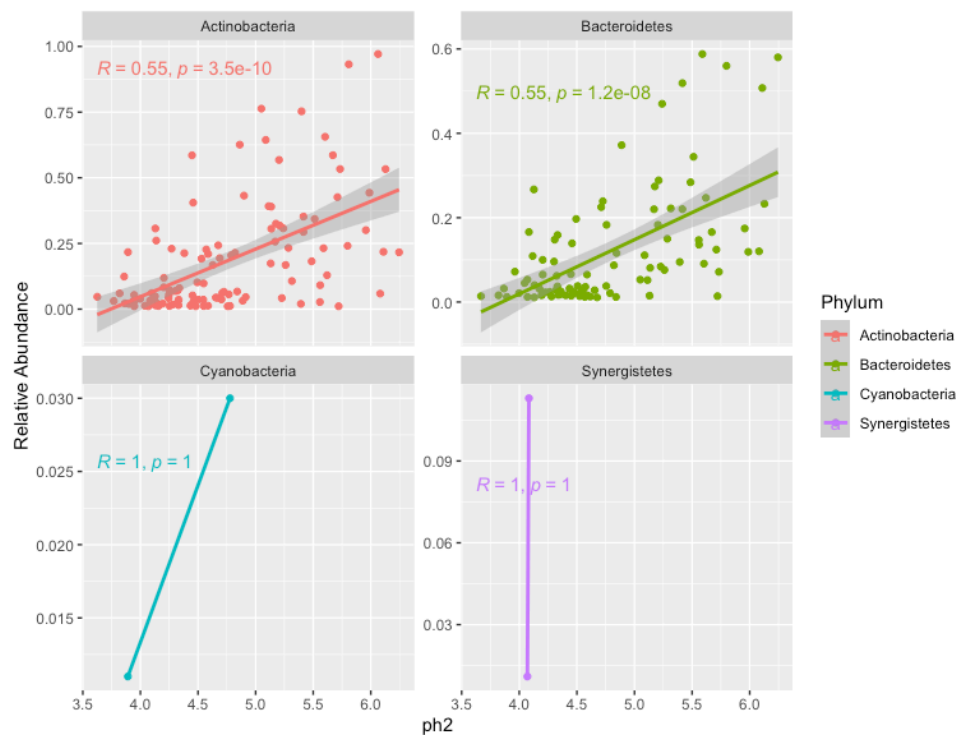


Figure 35. Taxonomic (Phylum level) changes along a pH gradient for vaginal samples with only significant Phyla (corr > 0.5) plotted. Statistic results for Spearman correlation shown.

Finally, we can also use the edgeR package with a custom linear model.

```
HMP_simPH_phylglom <- HMP_simPH %>%
  tax_glom(taxrank = "Phylum")

dge2 <- phyloseq_to_edgeR(HMP_simPH_phylglom, group = "sex")

# Make our sample data easy to access
a <- sample_data(HMP_simPH_phylglom)

# Set up our test design: we're going to model the abundance of each
# Phylum as a function of pH
design <- model.matrix(~a$ph2)

# let edgeR normalize data and apply the design
```

```

x <-calcNormFactors(dge2, method="RLE")
x <-estimateGLMCommonDisp(dge2, design)
x <-estimateGLMTrendedDisp(dge2, design)
x <-estimateGLMTagwiseDisp(dge2, design)

# the statistical test we're applying is a quasi-likelihood negative
binomial generalized log linear model
fit <- glmQLFit(x, design)
qlf <- glmQLFTest(fit, coef=1)

# Extract values from test results
tt <-topTags(qlf, n = nrow(dge2$stable), adjust.method = "BH", sort.by
= "PValue")
res <-tt@.Data[[1]]
alpha <-0.05
sigtax <-res[(res$FDR <= alpha), ]
sigtax <-cbind(as(sigtax, "data.frame"),
as(tax_table(HMP_simPH_phylglom)[rownames(sigtax), ], "matrix"))

#subset by most significant only (p value <0.05 and at least a 2-fold
change in abundance)
sigtax_2fold <- subset(sigtax, FDR < 0.05 & logFC >= 2 | logFC <= -2
& FDR < 0.05)
head(sigtax_2fold)

#Make a new file with only the names of the taxa you wish to keep.
keepTaxa_pH <- sigtax_2fold$Phylum

#Check that your names are OK.
print(keepTaxa_pH)

#Use the subset_taxa function and your new taxa list to create a
phyloseq file with only your Phylum of interest
ph_2fold <- subset_taxa(HMP_simPH, Phylum %in% keepTaxa_pH)

```

```
#Melt phyloseq file into a data frame
dat_onlysigPhyla <- ph_2fold %>%
  tax_glom(taxrank = "Genus") %>%
  transform_sample_counts(function(x) {x/sum(x)} ) %>%
  filter_taxa(function(x){sum(x > 0) > 30}, prune = TRUE) %>%
  psmelt() %>%
  filter(Abundance > 0.01) %>%
  arrange(Phylum)

ggplot(dat_onlysigPhyla,aes(x=ph2,y=Abundance,color=Phylum))+
  geom_point() +
  scale_y_continuous()+
  facet_wrap(Phylum+Genus~.,scales = "free_y")+
  geom_smooth(method=lm)+
  ylab("Relative Abundance")+
  stat_cor(method = "spearman")
```

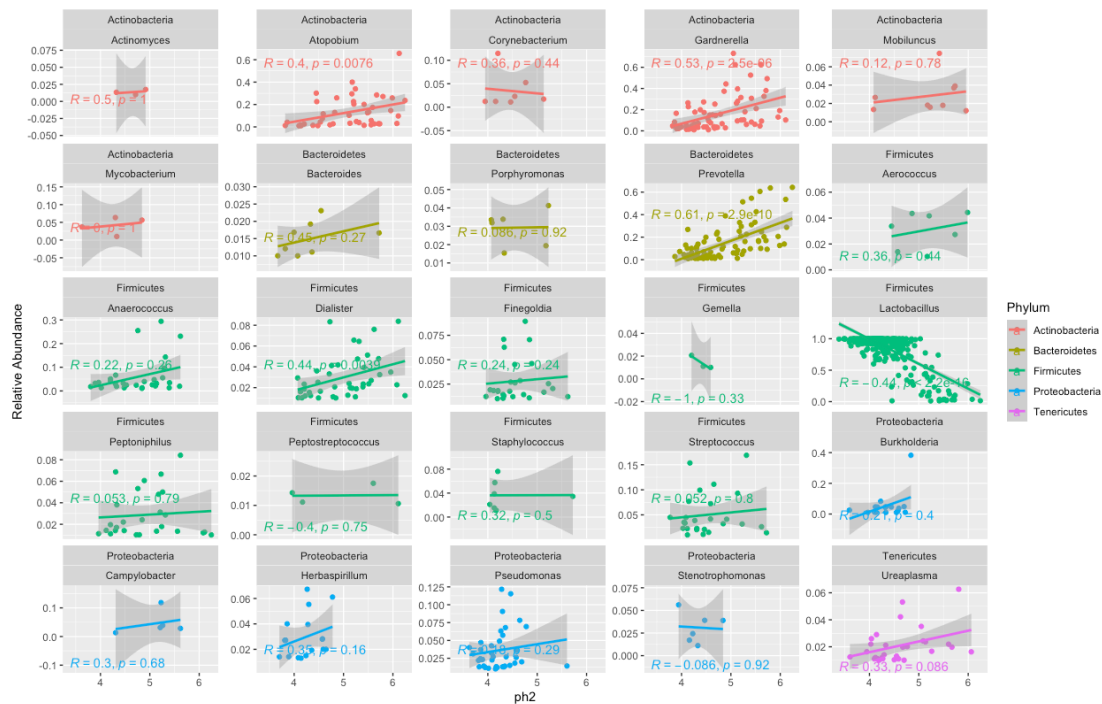


Figure 36. Differentially abundant taxa between stool and saliva by Genus and Phylum. Taxa identified using a custom linear model. Plots show the results of a separate Spearman correlation.

3 Capstone: Your assessment

For your assessment, you will hand in a report based on your analysis of microbiome data provided at the beginning of the class. Students will be divided into groups with each group working on datasets from independent studies. Within a group, individuals will be tasked with analysing specific aspects of the datasets. A template will be provided for students to provide results based on their specific dataset and research focus.

This is an individual report with each student working separately on different research questions and will be assessed based on 5 report sections each representing 20% of the report marks:

1. Background (1-2 pages)
2. Methods and results for alpha diversity measurements
3. Methods and results for beta diversity measurements
4. Methods and results for taxonomic comparisons
5. Discussion and synthesis (2-3 pages)

Each section will be assessed based on the following criteria:

| Report assessment criteria: | Excellent | Good | Neutral | Fair | Poor | Unable to judge |
|--|-----------|------|---------|------|------|-----------------|
| Background | | | | | | |
| Methods documentation | | | | | | |
| Clarity of writing/ease of understanding | | | | | | |
| Clear statement of purpose | | | | | | |
| Clear presentation of results | | | | | | |
| Clear synthesis and interpretation | | | | | | |
| Completeness | | | | | | |

4 Appendix

4.1 Installing Software for Class

4.1.1 How to install R and R Studio

4.1.1.1 Using *R* and *R Studio* on the Student Desktop

If you use Otago University's Student Desktop (install instructions are available here: <https://blogs.otago.ac.nz/studentit/student-desktop/student-desktop-own-device/>), R and R Studio are pre-installed in this environment.

You can run Student Desktop either as stand-alone or within your browser.

You can log into student desktop at: <http://www.otago.ac.nz/studentdesktop>

You will need to run the following on your R studio console on your student in order to install the packages needed for class

```
source("https://gitlab.com/morganlab/public/Micro336/raw/master/code/lab_prereqs.R")
```

4.1.1.2 *You can install R on your own computer*

R is an open-source statistical software package which is freely available at <https://cran.r-project.org>. At this URL, you can download pre-compiled binaries for Windows, Linux, or Mac. Follow the links to download the appropriate binary for your computer.

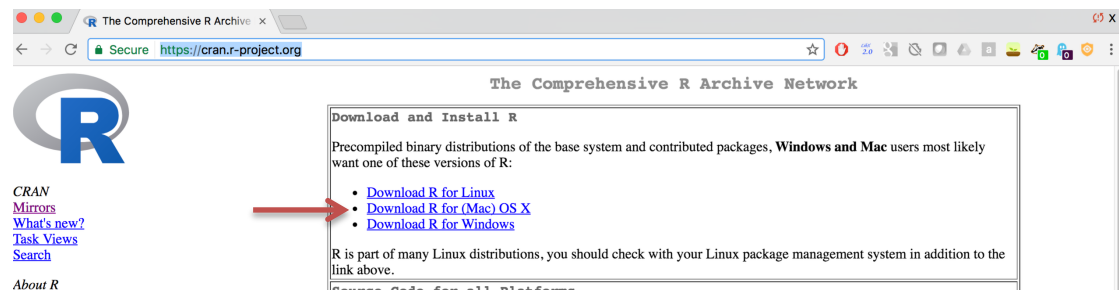


Figure 37. How to download R

Follow the instructions below for Mac or Windows.

For Mac:

Choose the package appropriate for your operating system version. You can see your operating system version by choosing the Apple menu and then selecting “About this Mac”.

The image is a composite of three screenshots illustrating the steps to download R on a Mac.

Top Screenshot: Shows the Apple menu with the path: Apple menu > About This Mac.

Middle Screenshot: Shows the 'About This Mac' window. A red arrow points to 'OS X El Capitan' (Version 10.11.6). Other details include: MacBook Pro (Retina, 13-inch, Early 2015), Processor: 2.9 GHz Intel Core i5, Memory: 16 GB 1867 MHz DDR3, Startup Disk: Macintosh HD, Graphics: Intel Iris Graphics 6100 1536 MB, and Serial Number: C02QD10EFVH8.

Bottom Screenshot: Shows the CRAN website (https://cran.r-project.org). A red arrow points to the 'R 3.4.0.pkg' download link under the 'Files:' section. The page also displays the R logo and various navigation links on the left sidebar.

Text from the CRAN website:

please use the normal precautions with downloaded executables.

As of 2016/03/01 package binaries for R versions older than 2.12.0 are only available from the [CRAN archive](#) so users of such versions should adjust the CRAN mirror setting accordingly.

R 3.4.0 "You Stupid Darkness" released on 2017/04/21

Important: since R 3.4.0 release we are now providing binaries for OS X 10.11 (El Capitan) and higher using non-Apple toolkit to provide support for OpenMP and C++17 standard features. Please read the corresponding note below.

Please check the MD5 checksum of the downloaded image to ensure that it has not been tampered with or corrupted during the mirroring process. For example type

```
md5 R-3.4.0.pkg
```

in the *Terminal* application to print the MD5 checksum for the R-3.4.0.pkg image. On Mac OS X 10.7 and later you can also validate the signature using

```
pkgutil --check-signature R-3.4.0.pkg
```

Files:

R 3.4.0 binary for OS X 10.11 (El Capitan) and higher, signed package. Contains R 3.4.0 framework, R.app GUI 1.40 in 64-bit for Intel Macs, Tcl/Tk 8.6.6 X11 libraries and Tinfo 5.2. The latter two components are optional and can be omitted when choosing "custom install", they are only needed if you want to use the `tcltk` R package or build package documentation from sources.

Note: the use of X11 (including `tcltk`) requires [XQuartz](#) to be installed since it is no longer part of OS X. Always re-install XQuartz when upgrading your OS X to a new major version.

Important: this release uses Clang 4.0.0 and GNU Fortran 6.1, neither of which is supplied by Apple. If you wish to compile R packages from sources, you will need to download and install those tools - see the [tools](#) directory.

Figure 38. Downloading R on a Mac. Choose the correct version for your OS.

For Windows:

Choose the “base” directory and then “Download R for Windows”.

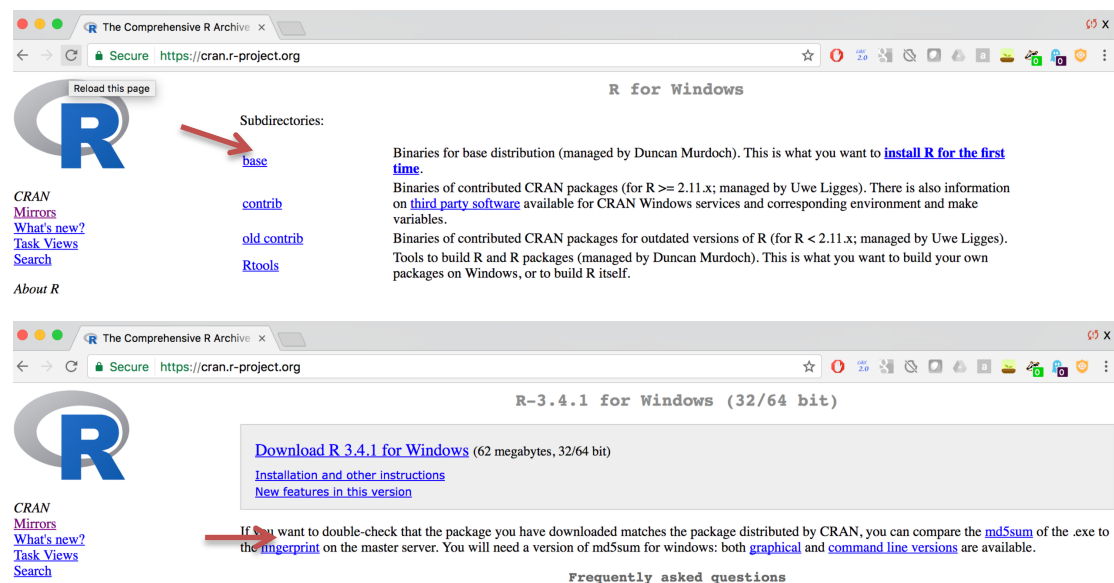


Figure 39. Downloading R on Windows.

4.1.1.3 Downloading R Studio

R studio is a graphic environment that complements R and allows easy construction of the Markdown files required for class.

You can download R Studio at

<https://www.rstudio.com/products/rstudio/download/>

Click the Download button that is under Select “R Studio Desktop – Open Source License” (you will have to scroll down), and choose the appropriate installer for your operating system at the bottom of the page.

The screenshot shows the RStudio website's download page. At the top, there are navigation links for 'Products', 'Resources', 'Pricing', 'About Us', and 'Blogs'. Below this, a table lists different license options: AGPL, Commercial, and AGPL, each with a 'Download' button. A 'Have Questions?' section with a 'Talk to the Sales Team' button is also present. The main section is titled 'RStudio Desktop 1.0.143 — Release Notes' and includes a note about R requirements. Below this, a table titled 'Installers for Supported Platforms' lists various operating systems and their corresponding installer sizes and dates. A red arrow points to this table. Below the table, there is a 'Source Code' section.

| Installers | Size | Date | MD5 |
|--|---------|------------|------------------------------------|
| RStudio 1.0.143 - Windows Vista/7/8/10 | 81.9 MB | 2017-04-19 | 76bb84296b9202759b3eb1de555a231 |
| RStudio 1.0.143 - Mac OS X 10.6+ (64-bit) | 71.2 MB | 2017-04-19 | c7f16d855428b225a20225d1a431954b4 |
| RStudio 1.0.143 - Ubuntu 12.04+/Debian 8+ (32-bit) | 80.5 MB | 2017-04-19 | 21ca14b2fcd61a2361aead2a7f3d0313d |
| RStudio 1.0.143 - Ubuntu 12.04+/Debian 8+ (64-bit) | 92.1 MB | 2017-04-19 | 75761aee20915d8d415d562b3771fbee |
| RStudio 1.0.143 - Fedora 19+/Redhat 7+/openSUSE 13.1+ (32-bit) | 84.7 MB | 2017-04-19 | 2c356d4ee0d67ad4042e0194a7b3e53 |
| RStudio 1.0.143 - Fedora 19+/Redhat 7+/openSUSE 13.1+ (64-bit) | 85.7 MB | 2017-04-19 | 7ab5fcd249331d6b6e493c6c8a7ac36d68 |

| Zip/Tarballs | Size | Date | MD5 |
|--|----------|------------|------------------------------------|
| RStudio 1.0.143 - Windows Vista/7/8/10 | 117.6 MB | 2017-04-19 | 24ae8674ba0c0e5310c8172f2e5f12e |
| RStudio 1.0.143 - Ubuntu 12.04+/Debian 8+ (32-bit) | 86.2 MB | 2017-04-19 | c4444d8159463b9e7a930ba4c071350e0 |
| RStudio 1.0.143 - Ubuntu 12.04+/Debian 8+ (64-bit) | 93.2 MB | 2017-04-19 | c92ecfdb80432a5532e23a22630918b |
| RStudio 1.0.143 - Fedora 19+/Redhat 7+/openSUSE 13.1+ (32-bit) | 85.4 MB | 2017-04-19 | 197280b1c10daee0baef889a14d9a58f |
| RStudio 1.0.143 - Fedora 19+/Redhat 7+/openSUSE 13.1+ (64-bit) | 86.6 MB | 2017-04-19 | 0ba5fcd249331d6b6e493c6c8a7ac36d68 |

Figure 40. Downloading R Studio

4.2 Creating an R Markdown (not from the class template).

1. In R Studio, from the “File” menu, select “New File” and “R Markdown”.

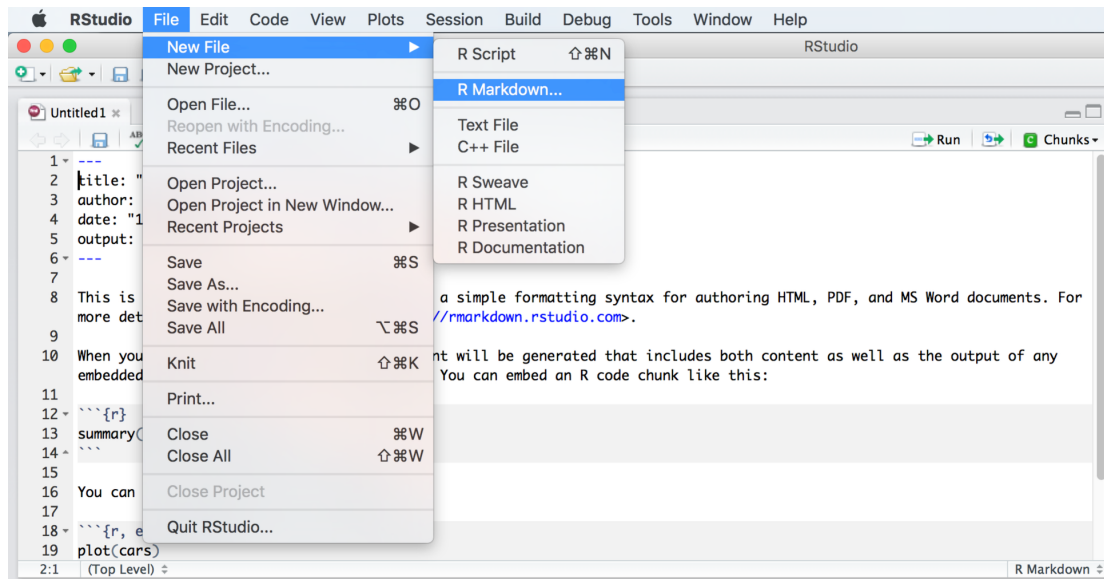


Figure 41. Making a new R Markdown

2. Give your Markdown file a name, and choose “HTML” as the output format. Then click “OK”.

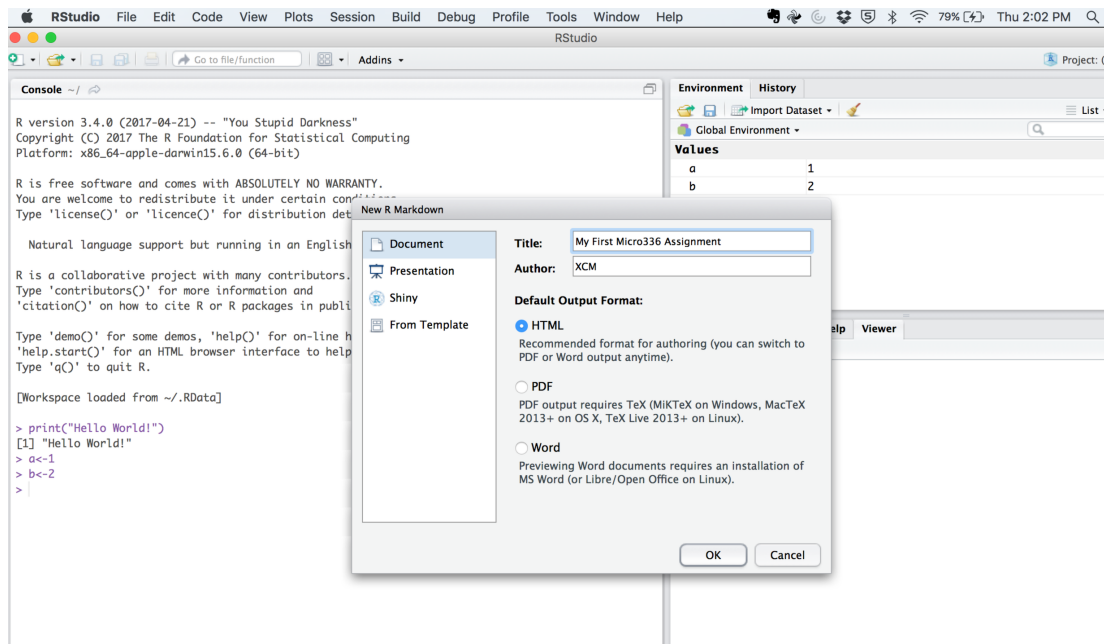


Figure 42. Naming your R Markdown.

4.3 Some simple R exercises

4.3.1 Beginning to use R

The next section extensively uses text from a tutorial written by Murray Cazdown. Please see the [Appendix 4](#) for license terms for this material. All text chunks derived from this tutorial is indicated by the citation “Cazdown” and are in shaded gray boxes. This section reviews some key concepts that were introduced in your R orientation session.

4.3.2 A conceptual view of R – The Big Picture

In R, you can assign many different types of data (numbers, characters, tables of numbers and / or characters) to variables. R has many different functions (commands) to manipulate and analyze your data. All analysis in R for this class will consist, fundamentally, of loading your data into variables, then using R functions to manipulate, analyze, and plot your data.

For example, in the following code, data is assigned to x and y, and then the function “plot” is used to plot the data.

```
```{r}
#Assign data to X
x<-c(1:10)
print(x)
Assign data to Y
y<-c(11:20)
print(y)
#Apply the "plot" function to x and y
plot(x, y)
```
```

This piece of code, in R markdown, would produce the following output:

```
#Assign data to X
x<-c(1:10)
print(x)

## [1] 1 2 3 4 5 6 7 8 9 10
# Assign data to Y
y<-c(11:20)
print(y)

## [1] 11 12 13 14 15 16 17 18 19 20
#Apply the "plot" function to x and y
plot(x, y)
```

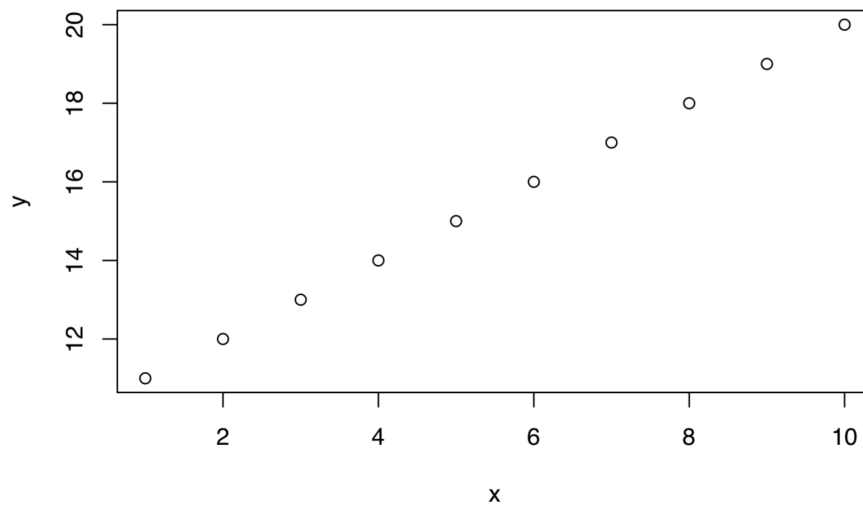


Figure 43. Output from example R commands

I could also execute this code by typing the commands in the console. It would look like this in the console, and would produce the same plot in the “plots” tab in R studio.

```
Console R Markdown ✕
~/
> x<-c(1:10)
> print(x)
[1] 1 2 3 4 5 6 7 8 9 10
> y<-c(11:20)
> print(y)
[1] 11 12 13 14 15 16 17 18 19 20
> plot(x, y)
>
```

Figure 44. Example: using the console

I can apply any function I like to x and y – mean, median, or even a T-test!

```
Console R Markdown ✕
~/
> mean(x)
[1] 5.5
> median(y)
[1] 15.5
> t.test(x, y)

Welch Two Sample t-test

data: x and y
t = -7.3855, df = 18, p-value = 7.503e-07
alternative hypothesis: true difference in means is not equal to 0
95 percent confidence interval:
-12.844662 -7.155338
sample estimates:
mean of x mean of y
5.5 15.5

> |
```

Figure 45. Example: applying a statistical function to data

The important thing to understand is that **we are using functions (commands) on variables (data) to get outputs.**

4.3.3 Getting help in R Studio ²

You will sometimes want to use the Help menu so you can read the documentation about how to use an R function. There are several different ways you can do this:

1. In the console, you can type `??command` or `help(command)` and the documentation for the command will appear
2. You can also use the R studio Help menu to bring up help documentation (**Figure 12**).
3. You can also refer to the R manual <https://cran.r-project.org/doc/manuals/r-release/R-intro.html>
4. Or www.google.com with "R command" or "R thing I want to do"

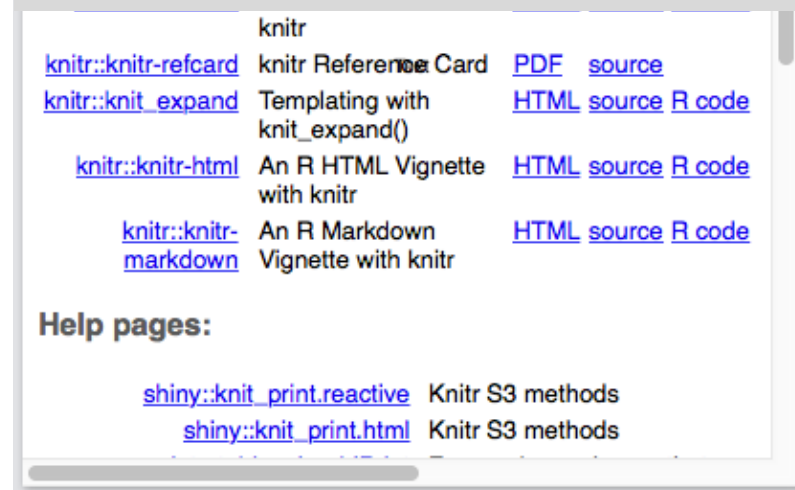


Figure 46. Using the R Studio help menu

4.3.4 Understanding Variables in R ²

Variables are used in R to store things: numbers, letters, strings, lists, or even what other variables contain

It is good practice to name your variable so that it describes what it contains

Variables can be called anything you like so long as you follow these rules:

- Variable names must start with a letter

- Variable names cannot contain symbols other than underscore or period (`_` or `.`)
- Upper or lower case letters are allowed.
- Numbers are allowed except at the start.
- Variable names cannot be reserved words.

Examples of variable names:

`mydata`

`Mydata`

`myData`

`my_data`

R has an assignment operator (`<-`) that you can use to assign values to variables. For example, type this into your prompt to assign the value “5” to the variable “a”.

```
a<-5
```

To view what a variable contains, just enter the variable name into the R console prompt.

Variables can be overwritten by assigning something different to them. Variables only contain the last item assigned. In addition to assignment, R has basic arithmetic operators (addition (+), subtraction (-), multiplication (*), division (/), and exponentiation (^). You can also use parenthesis for clarity in all mathematical operations.

```
# A little practice using basic geometry
```

```
a <- 2
```

```
b <- 3
```

```
a <- a * 2
```

```
hypotenuse <- a^2 + b^2
```

```
not_hypotenuse <- (a+b)^2
```

```
hypotenuse <- hypotenuse^(0.5)
```

What values do a, b, not_hypotenuse, and hypotenuse now contain?

4.3.5 Reading and Writing Data ²

One of the most important things you will need to learn to do is to read data files into R and store them as variables.

Reading

Just as we can store single values into variables, we can store entire datasets in a variable. Generally the data we want to read in will be in either tab- or comma-separated format. We have to specify three things: 1) the filename 2) whether the file has a header and 3) the field separating character (\t = tab)

If you don't assign the output from read.delim to a variable it will just output the entire file into your prompt.

```
# for tab separated data
mydata <- read.delim(file = "dummy_data.txt", header= TRUE, sep =
"\t", stringsAsFactors = FALSE)

# for comma separated data
mydata <- read.csv(file = "dummy_data.csv", header = TRUE)
```

Writing Data

Writing files is very similar to reading data. Again, you need to specify the filename, whether there are column and row names in the data, and the separating character to use.

```
write.table(data = mydata, file = "mydata_out.txt", sep = "\t",  
col.names = TRUE, row.names = FALSE, quote = FALSE )
```

4.4 Source of R tutorials

4.4.1 Murray Cazdow vignettes

Portions of sections “R tutorial” and “Getting Started with R” originate from tutorials written by Murray Cazdow:

https://github.com/murraycazdow/R_tutorials

They are used under the MIT License:

The MIT License (MIT) Copyright (c) 2015 Murray Cadzow, University of Otago Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions: The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software. THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

4.4.2 Bioconductor Vignettes

Portions of section “Working with *Phyloseq*” originate from:

<https://www.bioconductor.org/packages/release/bioc/vignettes/phyloseq/inst/doc/phyloseq-analysis.html>

They are used under the AGPL-3 License:

By default open-access materials contained in this directory are copyright protected.

Open-Source Exemption:

The only exception to this copyright is the source-code (computer-interpretable code), but not textual explanations of the source code, some of which may already be included in past, pending or future publications by the authors and should be considered copyrighted. To be clear, most of the exempted source code material is present in Sweave (.Rnw), R-markdown (.Rmd), markdown (.md), and R-script (.R) files, and is hereby open-licensed according to AGPL-3 for free use and re-use for any purpose, with no guarantee of valid or useful performance by the authors (although we are trying very hard). Any LaTeX, or any other code primarily intended to produce text documents, is not exempted and should be considered copyright protected.

Intent of authors:

We intend this copyright to allow for free-of-charge open-access to the included materials, but not republication or redistribution without explicit permission from the authors.

4.5 Additional R Concepts

4.5.1 The Working Directory ²

In R everything you do is in reference to what is known as your 'working directory'. This is the folder on your computer that R will read and write data to by default

You can find your current working directory with the command

```
getwd()
```

Likewise you can change your working directory with

```
setwd(dir = "path/to/directory")
```

or in RStudio in the Session menu -> Set Working Directory

4.5.2 How to install R packages manually

the following instructions show how to install R packages manually or directly in the terminal. For a manual install use the Toolbar:

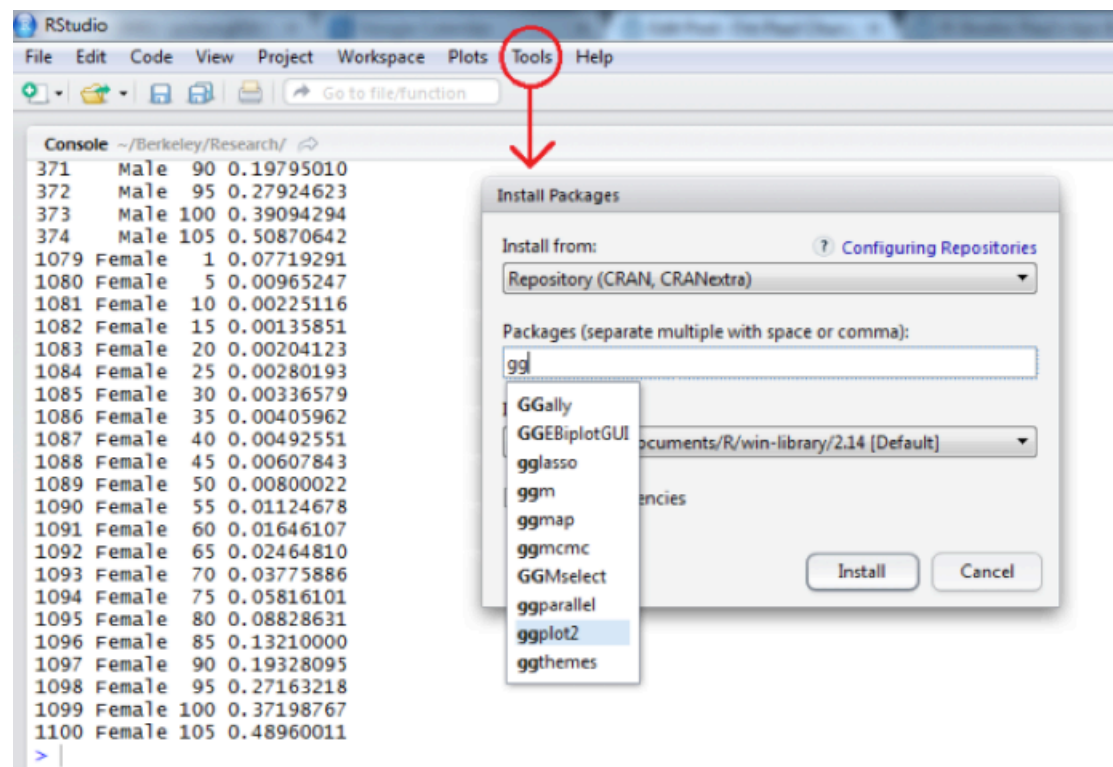


Figure 47. Installing an R package from the gui.

Alternatively, in the terminal:

```
source("https://bioconductor.org/biocLite.R")
biocLite("phyloseq")
```

4.5.3 How to download HMPv35 R data from original source

Create a temporary local file where there large file will be downloaded and read-from.

```
HMPv35 <- tempfile()

otu_url <- "https://raw.githubusercontent.com/joey711/phyloseq-demo/gh-
pages/HMPv35.RData"

download.file(otu_url, HMPv35)
```

If you have problems doing this and get an error saying:

Error in load(HMPv35) : empty (zero-byte) input file

Download the file directly to your downloads folder by copy pasting the url into a web browser. Then you can load it directly from that folder by:

```
HMPv35 <- "/Users/sergiomorales/Downloads/HMPv35.RData"

load(HMPv35)
```

You can also save this file to a different location so you do not need to import again.

```
save(HMPv35, file="your/file/path/HMPv35.RData")
```

4.6 Hypothesis testing in categorical and continuous metadata

In [section 6.3.3](#), you learned to test whether observed alpha diversity was different based on sex (eg male vs female). This is an example of hypothesis testing on a categorical metadata – in the HMP data, the sex of all patients is one of two categories. In contrast, a variable such as age or weight is a continuous metadata.

4.6.1 Categorical hypothesis testing

Testing the hypothesis that sex affects salivary Shannon alpha diversity:

H_0 : There is no relationship between Shannon diversity and sex

$H_{\text{alternative}}$: There is a relationship between Shannon alpha diversity and sex

Here, we can use a t-test (two categories, parametric (assumes data is normally distributed) or a Mann-Whitney U test (AKA Wilcoxon rank-sum test) (non-parametric – does not assume data is normally distributed) to test the null hypothesis. If we had more than two categories to test, we would use either an ANOVA (parametric) or a Kruskal-Wallis (non-parametric) test to test the null hypothesis that there is no difference in means (parametric) or that the distributions of both populations are equal (non-parametric).

Let's see how to apply all four of these tests to test this particular null hypothesis, using the `alpha_meta_H35_sal` object created in section 6.3.3

#T test

```
t.test(alpha_meta_H35_sal$Shannon ~ alpha_meta_H35_sal$sex)
```

Welch Two Sample t-test

```
data:  alpha_meta_H35_sal$Shannon by alpha_meta_H35_sal$sex
t = -1.3733, df = 265.64, p-value = 0.1708
alternative hypothesis: true difference in means is not equal to 0
95 percent confidence interval:
 -0.19277719  0.03435782
sample estimates:
mean in group female    mean in group male
      5.961410           6.040619
```

Interpretation: Based on the T-test, we do not reject the null.

#Mann Whitney test

```
wilcox.test(alpha_meta_H35_sal$Shannon ~ alpha_meta_H35_sal$sex)
```

Wilcoxon rank sum test with continuity correction

data: alpha_meta_H35_sal\$Shannon by alpha_meta_H35_sal\$sex

W = 8996, p-value = 0.09355

alternative hypothesis: true location shift is not equal to 0

Interpretation: Based on Mann-Whitney test, we do not reject the null

#Anova

```
result<-aov(alpha_meta_H35_sal$Shannon ~ alpha_meta_H35_sal$sex)
```

```
summary(result)
```

| | Df | Sum Sq | Mean Sq | F value | Pr(>F) |
|-------------------------|-----|--------|---------|---------|--------|
| alpha_meta_H35_sal\$sex | 1 | 0.44 | 0.4403 | 1.835 | 0.177 |
| Residuals | 288 | 69.10 | 0.2399 | | |

Interpretation: Based on ANOVA, we do not reject the null

#Kruskal-Wallis test

```
kruskal.test(alpha_meta_H35_sal$Shannon ~ alpha_meta_H35_sal$sex)
```

Kruskal-Wallis rank sum test

data: alpha_meta_H35_sal\$Shannon by alpha_meta_H35_sal\$sex

Kruskal-Wallis chi-squared = 2.8145, df = 1, p-value = 0.09341

Interpretation: Based on Kruskal-Wallis test, we do not reject the null

4.6.2 Continuous data hypothesis testing

Sometimes you will need to answer different questions that require non-categorical statistical tests – for example, “Is there a significant relationship between pH and observed alpha diversity?”. We’ll show you another example we can test in the `alpha_meta_H35_sal` object.

H_0 : There is no relationship between observed alpha diversity and Shannon diversity.

$H_{\text{alternative}}$: There is a relationship between observed alpha diversity and Shannon diversity.

#Parametric test: Pearson correlation test

```
cor.test(alpha_meta_H35_sal$Shannon, alpha_meta_H35_sal$Observed,  
method="pearson")
```

```
Pearson's product-moment correlation
```

```
data: alpha_meta_H35_sal$Shannon and alpha_meta_H35_sal$Observed
```

```
t = 14.654, df = 288, p-value < 2.2e-16
```

```
alternative hypothesis: true correlation is not equal to 0
```

```
95 percent confidence interval:
```

```
0.5822062 0.7149219
```

```
sample estimates:
```

```
cor
```

```
0.6535593
```

Interpretation: Observed diversity and Shannon entropy are highly correlated

#Non-parametric test: Spearman correlation test

```
cor.test(alpha_meta_H35_sal$Shannon, alpha_meta_H35_sal$Observed,
method="spearman")
```

Spearman's rank correlation rho

data: alpha_meta_H35_sal\$Shannon and alpha_meta_H35_sal\$Observed

S = 1366900, p-value < 2.2e-16

alternative hypothesis: true rho is not equal to 0

sample estimates:

rho

0.6637101

Interpretation: Observed alpha diversity and Shannon entropy are highly correlated

4.6.2.1 Using EdgeR with categorical metadata

In this example (as shown in [section 6.3.9](#) of the lab manual), for each OTU in the dataset, we are using the binomial test to testing the null hypothesis that there is no difference in the frequency of the observed OTU between either category.

#Two categories

```
dge = phyloseq_to_edgeR(HMPSvs_rare, group = "HMPbodysubsite")
et = exactTest(dge)
tt = topTags(et, n = nrow(dge$table), adjust.method = "BH", sort.by =
"PValue")
head(tt$table)
```

| Rank1 | Phylum | Class | Order | Family |
|------------------|---------------|---------------|-------------|-----------------|
| Genus | logFC | | | |
| OTU_97.42864 | Root | Firmicutes | Clostridia | Clostridiales |
| Veillonellaceae | Veillonella | -4.678247 | | |
| OTU_97.45365 | Root | Firmicutes | Bacilli | Lactobacillales |
| Streptococcaceae | Streptococcus | -4.258746 | | |
| OTU_97.43343 | Root | Bacteroidetes | Bacteroidia | Bacteroidales |
| Bacteroidaceae | Bacteroides | 4.023568 | | |

```

OTU_97.40451 Root Bacteroidetes Bacteroidia Bacteroidales
Bacteroidaceae Bacteroides 3.890675

OTU_97.44941 Root Proteobacteria Gammaproteobacteria Pasteurellales
Pasteurellaceae Haemophilus -3.862739

OTU_97.43147 Root Bacteroidetes Bacteroidia Bacteroidales
Porphyromonadaceae Porphyromonas -3.732534

logCPM PValue FDR
OTU_97.42864 9.447860 0 0
OTU_97.45365 9.053691 0 0
OTU_97.43343 9.071287 0 0
OTU_97.40451 8.923910 0 0
OTU_97.44941 8.773278 0 0
OTU_97.43147 8.625102 0 0

```

The outcome of this statistical test (shown in `tt$table`, obtained by running code in section 6.3.9) is a list of OTUs for which the null hypothesis has been rejected – ie they appear more frequently in one category than in the other category. The `logFC` value indicates how relatively enriched they are in category A relative to category B.

4.6.2.2 Using *EdgeR* with continuous metadata

When we are testing relationships between OTUs and continuous metadata, we're testing a hypothesis that as a metadata increases or decreases, the abundance of an OTU also increases (reject null), or the abundance of the OTU does not change (accept null). In the case of a variable like pH, we might expect that for every 0.5 units of pH, we would expect a doubling of the abundance of the organism.

Here is an example of code that can test this hypothesis. This code uses the HMP data for its example, but HMP data does not have any continuous metadata. Therefore, I will attach Shannon diversity (calculated in `alpha_meta_H35_sal`) to the `HMPv35saliva` phyloseq object so we can test whether any OTUs are associated with Shannon entropy.

Code to attach Shannon diversity to HMPv35Saliva sample data

```
sample_data(HMPv35saliva)[,10] = alpha_meta_H35_sal$Shannon
```

```
# Rename the metadata column in the sample data table
colnames(sample_data(HMPv35saliva))[10] = "Shannon"
```

Now that we've created a phyloseq object with a continuous metadata, we can convert it to an edgeR-compatible object. Note that I am still using (any) of my categorical metadata as a grouping variable – if I use Shannon here, I will get an error.

```
dge = phyloseq_to_edgeR(HMPv35saliva, group = "HMPbodysubsite")

# Make our sample data easy to access
a<-sample_data(HMPv35saliva)

# Set up our test design: we're going to model the abundance of each OTU as
a

# function of Shannon diversity
design<-model.matrix(~a$Shannon)

# let edgeR normalize data and apply the design
x = calcNormFactors(dge, method="RLE")
x = estimateGLMCommonDisp(dge, design)
x = estimateGLMTrendedDisp(dge, design)
x = estimateGLMTagwiseDisp(dge, design)

# the statistical test we're applying is a quasi-likelihood negative
binomial # generalized log linear model
fit <- glmQLFit(x, design)
qlf <- glmQLFTest(fit, coef=1)
```

qlf can then be filtered and visualized exactly like the results of the exact test in the previous example.

Interpretation: Taxa associated significantly with Shannon diversity here would indicate that for every unit increase in Shannon diversity, there is a corresponding increase in log abundance of the taxa.

What is quasi-likelihood negative binomial generalized log linear model? We want to ask whether any OTUs tend to increase or decrease along with Shannon diversity. If we apply a Spearman test, we're just asking whether samples with highest Shannon tend to be the samples with highest abundance of that OTU. If we apply a quasi-likelihood negative binomial GLLM, we can account for two features of microbiome data that make it very complicated:

Quasi-likelihood: a statistical method that better models data with high variance

Negative binomial: Adds a component to the model to account for the fact that some OTUs are really common and some OTUs are sparse (appear in very few samples)

Generalized log linear model : accounts for sparsity and high variance when asking (for example) whether an OTU tends to be more abundant in a sample with higher Shannon diversity.

5 Glossary

| Term | Description |
|----------------------|---|
| 16s rRNA gene | A gene encoding the component of the 30S small subunit of a prokaryotic ribosome. It is the most common phylogenetic marker used for community analysis. See molecular clock. |
| ADONIS | Or non-parametric multivariate analysis of variance (NPMANOVA). Provides a way to test statistically whether there is a significant difference in the observed variance (think variation in observed samples) between two sample clusters. |
| Alpha diversity | Represents the mean species diversity. These include total numbers of distinct taxa [richness, also called 'Observed' for observed richness in phyloseq.], as well as several 'diversity' measures including Shannon, Simpson and others. |
| ANOSIM | or ANALYSIS OF SIMilarity test. Provides a way to test statistically whether there is a significant difference between two or more groups of sampling units. It uses a dissimilarity matrix rather than raw data to determine if two sample clusters overlap (share the same community). |
| Beta diversity | Represents the diversity in microbial community between different environments (difference in taxonomic abundance profiles from different samples). Compares the species composition between two samples. |
| Bray-Curtis distance | Dissimilarity matrix generated from abundance or read count data. Differences in microbial abundances between two samples are summarized using values from 0 to 1. 0 means both samples share the same species at exact the same abundances. 1 means both samples have completely different species abundances. |
| Dada2 | An open-source software package that denoises and removes sequencing errors from Illumina amplicon sequence data. |
| dataframe | A table or a two-dimensional array-like structure in which each column contains values of one variable and each row contains one set of values from each column. |

| Term | Description |
|---------------------|---|
| <i>Exact Test</i> | Or Fisher's exact test. Performs a comparison of means (exact tests) to determine if differences in the means between two groups of negative-binomially distributed counts exist. |
| Fastq file | is a text-based file storing both a biological sequence (usually nucleotide sequence) and its corresponding quality scores. |
| FDR | Or false discovery rate correction. Procedures designed to control for the expected proportion of "discoveries" (rejected null hypotheses [low p values]) that are false (incorrect rejections). |
| Jaccard distance | Dissimilarity matrix generated from presence or absence of species (does not include abundance information). Differences in microbial composition between two samples are summarized using values from 0 to 1. 0 means both samples share exact the same species. 1 means both samples have no species in common. |
| matrix | A collection of data elements arranged in a two-dimensional rectangular layout. |
| microbial community | A community of microscopic organisms. These can include bacteria, archaea, eukaryotes and viruses. |
| molecular clock | A gene sequences that can be used to track the evolution of organisms. The most commonly used is the 16S gene. |
| Mothur | Is an open-source bioinformatics pipeline for performing microbiome analysis from raw DNA sequencing data. |
| Nanodrop | A spectrophotometer that uses extremely low volumes. |
| NMDS | Or non-metric multidimensional scaling. A type of ordination technique for multidimensional scaling. The goal of NMDS is to group data points into clusters of similar points based on a series of variables. By doing that you collapse information from multiple dimensions (e.g, from many OTUs) into just a few, so that they can be visualized and interpreted in 2-d plots. |
| otu_table | In phyloseq, it represents your OTU abundance table. |
| P value | Or probability value, or significance. A number between 0 and 1 and interpreted in the following way: A small p-value (typically ≤ 0.05) indicates strong evidence against the null |

| Term | Description |
|-------------------------------|---|
| | hypothesis, so you reject the null hypothesis. For example if comparing mean richness, $p < 0.05$ indicate the richness is significantly different. |
| Phred quality score (Q score) | <p>The most common metric used to assess the Base calling accuracy of a sequencing platform. Q scores are defined as a property that is logarithmically related to the base calling error probabilities (P)².</p> <p>$Q = -10 \log_{10} P$</p> <p>For example, if Phred assigns a Q score of 30 (Q30) to a base, this is equivalent to the probability of an incorrect base call 1 in 1000 times.</p> |
| phyloseq | An R package that allows you to import, store, analyze, and graphically display complex phylogenetic sequencing data that has already been clustered into Operational Taxonomic Units (OTUs), especially when there is associated sample data, phylogenetic tree, and/or taxonomic assignment of the OTUs. |
| phyloseq object | A file formatted for phyloseq that can include an OTU table (which contains the OTU counts for each sample), the sample data matrix (containing the metadata for each sample), the taxonomy table (the predicted taxonomy for each OTU), the phylogenetic tree, and the OTU representative sequences. |
| Pipeline | Also called a workflow. A collection of commands, (or transformations/tasks) run in series. |
| psmelt function | A command to create a dataframe from a phyloseq object. |
| Qiime | Is an open-source bioinformatics pipeline for performing microbiome analysis from raw DNA sequencing data. |
| Qubit fluorimeter | A fluorometer designed to accurately measure DNA, RNA, and protein quantity. |
| R Chunk | A code chunk is a runnable piece of R code. |
| R Functions | The fundamental building block of R. Each function represents a command. |
| R Library | The directory where packages are stored. |

| Term | Description |
|--------------------------|---|
| R Packages | The fundamental units of reproducible R code. They include reusable R functions, the documentation that describes how to use them, and sample data. They are collection of commands. |
| rank_names | In phyloseq, it represents the taxonomic levels in your tax_table. |
| Rarefaction | A technique used to subsample the read counts of each sample to a common read depth. |
| sample_data | In phyloseq, it represents your sample data or metadata. |
| SILVA reference database | Is an open-source resource that provides comprehensive, quality checked and regularly updated databases of aligned small (16S / 18S, SSU) and large subunit (23S / 28S, LSU) reference sequences. |
| tax_table | In phyloseq, it represents your table of taxonomic descriptors or taxonomic assignments. |
| UniFrac distance | Dissimilarity matrix generated from sequence distances (phylogenetic tree) based on the fraction of branch length that is shared between two samples or unique to one or the other sample. Differences in microbial composition between two samples are summarized using values from 0 to 1. 0 means both samples share exact the same species. 1 means both samples have no species in common. |
| unweighted UniFrac | See unifrac distance. Purely based on sequence distances (does not include abundance information). |
| weighted UniFrac | See unifrac distance. Branch lengths are weighted by relative abundances (includes both sequence and abundance information). |

6 References

1. Yilmaz, P. *et al.* Minimum information about a marker gene sequence (MIMARKS) and minimum information about any (x) sequence (MIXS) specifications. *Nature Biotechnology* **29**, 415-20 (2011).
2. Cazdow, M. R tutorials. Vol. 2017 (2017).
3. McMurdie, P.J. & Holmes, S. Phyloseq vignette. (2018).